

The Backend Engineer's Library

Cloud-Native Supply Chain Security

Volume C06

Contents

Container Images: OCI Format, Layers, and Attack Surface

Container Registries: Architecture, Trust, and Threats

Base Image Strategy: Minimal, Distroless, Hardened

Image Scanning and Vulnerability Management

Image Signing and Verification in Kubernetes

Admission Control and Policy Engines: OPA and Kyverno

Kubernetes Delivery Chains: Helm, Operators, GitOps

Infrastructure as Code Supply Chain Risks

Serverless, Managed Services, and the Cloud Provider Chain

A Cloud-Native Supply Chain Reference Architecture

Chapter 1 — Container Images: OCI Format, Layers, and Attack Surface

What this chapter covers. You run containers every day. You `docker build`, you `docker push`, you `kubectl apply`, and something that started life as a `Dockerfile` ends up as processes on a node in a cluster. This chapter is about what that *something* actually is on the wire and on disk — the **OCI image** — because you cannot secure an artifact whose structure you treat as opaque. An image is not a binary blob and not a filesystem snapshot; it is a small, precisely specified graph of **content-addressed JSON and tar blobs**, and nearly every attack and defense in the rest of Book 6 is a statement about that graph. Signing (Book 5) signs a node in it. Scanning (Chapter 4) enumerates what a node contains. Admission control (Chapter 6) checks a node's identity before it runs. Registries (Chapter 2) store the graph and hand out its edges. This chapter builds the format from the bottom up — manifest, config, layers, index — is precise about **digests versus tags** and **compressed-blob digests versus uncompressed diff IDs**, walks the layer/overlay model and the secret-leak it enables, maps the image attack surface that frames the rest of the book, and ends at the **OCI referrers model**, the mechanism by which the entire signing/SBOM/provenance ecosystem hangs off an image by its digest.

Learning goals — after this chapter you should be able to:

- Name the **OCI specifications** (Image Format, Distribution, Runtime) and explain the Docker-to-OCI standardization and why interoperability matters.
- Describe an image precisely as a **manifest + config + layers**, all **content-addressed by SHA-256**, and state what each part contains and what the manifest digest *is* (the image's identity — the thing you sign and deploy by).
- Distinguish the **manifest digest** (of the compressed layer blob) from the config's **diff_id** (of the uncompressed tar), and explain why both exist.
- Explain the **image index / manifest list** and how one tag resolves to a per-platform manifest.

- Explain the **overlay filesystem** layer model, why **nothing is ever deleted across layers**, and how secrets leak into images that a later `RUN rm` cannot remove.
- Use **multi-stage builds** to leave build-time secrets and junk behind.
- Map the **image attack surface**: base OS packages, app dependencies, added tooling, embedded secrets — and the tampering vectors that content-addressing does and does not defeat.
- Explain the **mutable-tag / immutable-digest** distinction as a TOCTOU problem and why fleets deploy by digest.
- Explain the **OCI 1.1 Referrers API** and the `subject` field, and the older cosign `.sig` tag convention, as the way signatures, SBOMs, and provenance attach to an image.

The OCI specifications

Before 2015 there was no *standard* container image; there was **Docker's** image format and its registry protocol, which everyone reverse-engineered because Docker was the only game in town. That is a fragile way to run an industry. In 2015 Docker, CoreOS, and others formed the **Open Container Initiative (OCI)** under the Linux Foundation, and Docker **donated** its image format and runtime as the seed. OCI turned a de-facto format into three published, versioned specifications:

- **Image Format Specification** — what an image *is*: the manifest, config, layer, and index media types and their JSON schemas, and the content-addressing rules. This is the spec this chapter is mostly about.
- **Distribution Specification** — how images move: the HTTP registry API (the `/v2/` API) for pushing and pulling manifests and blobs by tag or digest, including (as of v1.1) the Referrers API. Chapter 2 is built on this.
- **Runtime Specification** — how an *unpacked* image is run as a container: the on-disk bundle (a root filesystem plus a `config.json`) and the lifecycle a runtime like `runc` or `crun` executes. Largely out of scope for supply chain; we touch only the image-config fields that become runtime posture.

The relationship to Docker is worth stating plainly because it still confuses people: **the Docker image format and the OCI image**

format are two dialects of the same thing, differing almost entirely in media-type strings. `docker build` today produces OCI-compatible artifacts; containerd, CRI-O, Podman, Buildah, Kaniko, BuildKit, `crane`, `skopeo`, and every cloud registry speak OCI. When we say "the image" we mean the OCI artifact, and where Docker's legacy media types differ we note it.

Why standardization is not a footnote: **interoperability is a security property, not only a convenience**. Because the format is a published spec rather than one vendor's implementation, a signature produced by cosign, an SBOM produced by Syft, a scan produced by Trivy, and an admission check enforced by Kyverno all operate on the *same* bytes with the *same* digest and mean the same thing across GitHub, GitLab, ECR, GCR, Harbor, and a bare `registry:2`. A closed format would force each tool to re-derive identity, and identity is the one thing a supply chain cannot afford to have two opinions about.

What an image actually is

Strip away the tooling and an OCI image is **four kinds of object**, all identified by the SHA-256 of their own bytes (Book 5, Chapter 1 — Cryptographic Foundations, on content addressing):

Object	Media type (OCI)	Content	Role
Manifest	<code>application/vnd.oci.image.manifest.v1+json</code>	JSON: one config descriptor + ordered list of layer descriptors	The image <i>identity</i> . Its own digest is <code>sha256:...</code> — the thing you sign, pin, and deploy.
Config	<code>application/vnd.oci.image.config.v1+json</code>	JSON: env, entrypoint/cmd, user, workdir, <code>rootfs.diff_ids</code> , <code>history</code>	Runtime configuration + the ordered list of layer <i>identities</i> + build history.

Object	Media type (OCI)	Content	Role
Layer	application/vnd.oci.image.layer.v1.tar+gzip	gzipped tar of filesystem changes (a "diff")	The actual bytes: files added, changed, or whited-out relative to the layer below.
Index	application/vnd.oci.image.index.v1+json	JSON: list of manifest descriptors with platform	Multi-platform: one name → per-arch/OS manifests. Optional.

Everything is glued together by **descriptors**. A descriptor is the OCI spec's universal pointer: a small JSON object with a `mediaType`, a `digest`, and a `size`. It is a content-addressed edge in the graph — it says "there is an object of *this* type, with *this* SHA-256, of *this* many bytes," and nothing more. The single top-level digest transitively commits to every byte of the image, because the manifest names the config and layers by digest, and the config names each layer's uncompressed identity by digest. Change one byte anywhere and the top digest changes. That is the whole integrity argument, and we will lean on it repeatedly.

```

flowchart TD
    IDX["Image Index  
vnd.oci.image.index.v1+json  
sha256:INDEX"]
    M_AMD["Manifest amd64  
sha256:MANIFEST_A"]
    M_ARM["Manifest arm64  
sha256:MANIFEST_B"]
    CFG["Config JSON  
sha256:CONFIG  
env, entrypoint, user,  
rootfs.diff_ids, history"]
    L0["Layer 0 blob  
tar+gzip  
sha256:LAYER0"]
    L1["Layer 1 blob  
tar+gzip  
sha256:LAYER1"]
    L2["Layer 2 blob  
tar+gzip  
sha256:LAYER2"]

    IDX -->|"platform amd64"| M_AMD
    IDX -->|"platform arm64"| M_ARM
    M_AMD -->|"config descriptor"| CFG
    M_AMD -->|"layers[0]"| L0
    M_AMD -->|"layers[1]"| L1
    M_AMD -->|"layers[2]"| L2

```

The manifest

The manifest is the root of a *single-platform* image. Here is a real one, fetched with `crane` (the go-containerregistry CLI), trimmed:

```
$ crane manifest cgr.dev/chainguard/nginx:latest | jq .
```

```
{
  "schemaVersion": 2,
  "mediaType": "application/vnd.oci.image.manifest.v1+json",
  "config": {
    "mediaType": "application/vnd.oci.image.config.v1+json",
    "digest": "sha256:7f3c1b9e0a...c21",
    "size": 2841
  },
  "layers": [
    {
      "mediaType": "application/vnd.oci.image.layer.v1.tar+gzip",
      "digest": "sha256:b5d2f8e4a1...9ac",
      "size": 3125478
    },
    {
      "mediaType": "application/vnd.oci.image.layer.v1.tar+gzip",
      "digest": "sha256:0e91ab77c3...f10",
      "size": 41522
    }
  ]
}
```

Three things to internalize. First, the manifest is *small* — a few kilobytes of pointers — and it contains **no image bytes**, only descriptors. Second, the layer order is **significant**: layers apply bottom-to-top and later layers override earlier ones. Third, and most important, the **manifest's own digest is the image's identity**:

```
$ crane digest cgr.dev/chainguard/nginx:latest
sha256:4b2c9e6f1a3d8c0b5e7a2f9d4c1b8e6a3f0d7c2b9e5a1f8d4c0b7e3a6f2d9c1b
```

That `sha256:4b2c...` is what you sign in Book 5, Chapter 3, what you pin in a Deployment, and what an admission controller checks in Chapter 6. It is not the digest of a layer or of the config; it is the digest of the manifest JSON, and by transitivity of the whole image.

Docker's legacy dialect uses `application/vnd.docker.distribution.manifest.v2+json` for the manifest, `application/vnd.docker.container.image.v1+json` for the config, and `application/vnd.docker.image.rootfs.diff.tar.gzip` for layers. Structurally identical; only the strings differ. A registry and runtime dispatch on these `mediaType` values to decide how to parse each object, which is why getting them right matters for interoperability.

The config

The config blob is the image's *behavior and provenance-of-assembly*. Fetch it by the digest the manifest gave:

```
$ crane config cgr.dev/chainguard/nginx:latest | jq .
```

```
{
  "created": "2026-05-14T09:12:44Z",
  "architecture": "amd64",
  "os": "linux",
  "config": {
    "User": "65532",
    "Env": ["PATH=/usr/sbin:/usr/bin:/sbin:/bin"],
    "Entrypoint": ["/usr/sbin/nginx"],
    "Cmd": ["-g", "daemon off;"],
    "WorkingDir": "/",
    "ExposedPorts": { "8080/tcp": {} }
  },
  "rootfs": {
    "type": "layers",
    "diff_ids": [
      "sha256:1f0d7c2b9e...5a1",
      "sha256:6a3f0d7c2b...9e5"
    ]
  },
  "history": [
    { "created": "2026-05-14T09:12:40Z", "created_by": "ADD file:...
in /", "comment": "buildkit" },
    { "created": "2026-05-14T09:12:44Z", "created_by": "RUN apk add
nginx", "comment": "buildkit" }
  ]
}
```

The `config` object holds the runtime knobs: `Entrypoint`/`Cmd`, `Env`, `WorkingDir`, `ExposedPorts`, and `User`. These become the container's default behavior — we return to their security implications below.

`rootfs.diff_ids` is the subtle part and a common source of confusion. It is the ordered list of **layer identities** — but these are **not** the same digests the manifest lists. **The manifest's layer digest is the SHA-256 of the compressed (gzipped) blob; the config's `diff_id` is the**

SHA-256 of the *uncompressed* tar. Two digests for the same layer, for two different jobs:

- The **manifest digest** identifies the blob *as stored and transferred* — the registry deduplicates and serves it under that digest, and gzip determinism is not guaranteed, so this is the "on-the-wire" identity.
- The **diff_id** identifies the layer's *content* independent of how it was compressed — the runtime computes the rootfs from `diff_ids`, and it is stable even if you recompress. The **chain ID** (a running hash of `diff_ids`) is what overlay snapshotters use to name a stack of applied layers.

You will trip over this exactly once — when a layer digest in `docker history` or a scan report does not match the manifest — and then never again. Keep the rule: *manifest lists compressed digests, config lists uncompressed diff_ids*.

`history` is the config's record of **how each layer was made**: roughly one entry per Dockerfile instruction, with `created_by` holding the command. Entries with `empty_layer: true` correspond to instructions like `ENV`, `WORKDIR`, `USER`, `ENTRYPOINT` that change configuration without adding a layer, so `history` is usually longer than `diff_ids`. This is the record `docker history` renders — and, as we will see, a place secrets go to be discovered.

The layers

A layer is a **tar archive of filesystem changes** relative to the layers beneath it — a *diff*, not a full filesystem. If a build step adds `/usr/bin/app` and modifies `/etc/nginx/nginx.conf`, the layer's tar contains exactly those two paths. Deletions are encoded with **whiteout** entries: to delete `/etc/secret` a layer contains a zero-length file `/etc/.wh.secret`, and to mask an entire directory it contains an *opaque whiteout* `/dir/.wh..wh..opq`. The whiteout is an instruction to the union filesystem — "hide this path from the layers below" — and it is central to the secret-leak problem later.

Layers are **content-addressed, deduplicated, and shared**. Two images built **FROM** the same base reference the *same* base-layer blobs by the *same* digest; the registry stores each blob once and a node caches each blob once. A 10 MB app on a 120 MB base costs 10 MB to push and pull if the base is already present. This sharing is the economic engine of

containers at fleet scale — and, as the distributed-systems lens will show, also a **blast-radius multiplier**.

Multi-platform images: the index

A single manifest is one architecture and OS. Real registries serve `nginx:latest` to amd64 laptops and arm64 nodes from **one tag**, using an **image index** (Docker calls it a *manifest list*):

```
$ crane manifest --platform all nginx:1.27 | jq
'{mediaType, manifests: [.manifests[] | {digest, platform}]}'
```

```
{
  "mediaType": "application/vnd.oci.image.index.v1+json",
  "manifests": [
    { "digest": "sha256:aa11...", "platform": { "architecture":
"amd64", "os": "linux" } },
    { "digest": "sha256:bb22...", "platform": { "architecture":
"arm64", "os": "linux" } },
    { "digest": "sha256:cc33...", "platform": { "architecture":
"s390x", "os": "linux" } }
  ]
}
```

The index is itself a content-addressed object with its own digest, and each entry is a descriptor pointing to a **per-platform manifest** carrying a `platform` field. When a runtime pulls a tag that resolves to an index, it selects the manifest matching its own `architecture/os` and pulls *that*. An index also serves as the container for referrers responses and, increasingly, for bundling an image with its attestations — the same structure, reused. When you pin `nginx@sha256:INDEX_DIGEST` you are pinning the *index*, which transitively commits to every per-platform image under it.

How images are built: the layer model

`docker build` (or BuildKit, Buildah, Kaniko) walks a `Dockerfile` and turns instructions into layers. Roughly: each filesystem-mutating instruction — `RUN`, `COPY`, `ADD` — produces a new layer whose tar is the diff that instruction caused; metadata-only instructions — `ENV`, `WORKDIR`, `USER`, `ENTRYPOINT`, `CMD`, `LABEL` — produce empty-layer history entries and mutate the config instead.

```

FROM alpine:3.20           # base layers (shared)
RUN apk add --no-cache nginx # layer: nginx package tree
COPY nginx.conf /etc/nginx/ # layer: one config file
COPY app /usr/bin/app       # layer: the binary
USER 65532                  # config change, empty layer
ENTRYPOINT ["/usr/bin/app"] # config change, empty layer

```

The build **cache** keys each step on the instruction *and* the state of everything it consumed; a step is reused only if it and every step before it are unchanged. This is why **layer order determines cache efficiency**: put the stable, slow layers (dependency install) *before* the volatile ones (your source), so a code change re-runs only the cheap tail. Order also matters for **reproducibility** (Book 4, Chapter 2 — Hermetic and Reproducible Builds): non-deterministic steps early in the file poison the cache and the digest for everything after them.

The overlay filesystem at runtime

At runtime the layers are not re-tarred; they are **stacked by a union filesystem** — on Linux almost always **overlayfs**. Each layer is unpacked once into its own directory (named by chain ID) and marked read-only. overlayfs presents a single **merged** view where upper layers shadow lower ones, and gives the container a thin **writable upper layer** (upperdir) for changes it makes while running; the image layers are the read-only lowerdir s.

```

flowchart TB
    subgraph merged ["Merged view – the container rootfs"]
        direction TB
        RW["Writable container layer (upperdir)"]
        RW --> L3
        L3 --> L2
        L2 --> L1
    end
    L3["Layer 2 (ro): /usr/bin/app"]
    L2["Layer 1 (ro): /etc/nginx/nginx.conf"]
    L1["Layer 0 (ro): base OS filesystem"]
    classDef ro fill:#eef,stroke:#557
    class L1,L2,L3 ro

```

Two consequences follow directly and are the crux of image security:

1. **Layers are read-only and immutable once built.** A container writing to `/etc/secret` writes to its own upperdir; the image's layers are untouched. Nothing a running container does changes the image.

2. **Deletion is a masking operation, not an erasure.** When a build step deletes a file, the tooling writes a *whiteout* in the new layer that hides the path in the merged view. **The bytes remain in the earlier layer's blob.** The merged filesystem doesn't show them; the image still ships them.

Nothing is ever deleted: the secret-leak

This is the single most important image-hygiene fact, and it burns teams constantly. Consider:

```
FROM node:20
COPY id_rsa /root/.ssh/id_rsa      # layer N: private key added
RUN git clone git@github.com:acme/private.git \
  && rm /root/.ssh/id_rsa          # layer N+1: "removes" the key
```

The author believes the key is gone: it is not in the running container's filesystem, and `ls /root/.ssh` shows nothing. But the image is a *stack of layers*, and **layer N still contains `id_rsa` in full**. Anyone who pulls the image can list the layers and extract the blob:

```
$ crane export acme/app:latest - | tar -tf - | grep -c id_rsa  #
merged view: 0, masked by whiteout
0
$ crane blob acme/app@sha256:LAYER_N | tar -xzf - root/.ssh/id_rsa  #
the raw layer: still there
$ cat root/.ssh/id_rsa
-----BEGIN OPENSSH PRIVATE KEY-----
...the secret, fully intact...
```

`docker history` makes the leak trivial to *locate* even without extraction, because the config's history records the commands verbatim:

```
$ docker history --no-trunc acme/app:latest
IMAGE          CREATED BY                                     SIZE
sha256:...     RUN git clone ... && rm /root/.ssh/id_rsa     412
MB
sha256:...     COPY id_rsa /root/.ssh/id_rsa                 3.4
kB  <-- the key layer
sha256:...     FROM node:20                                  1.1
GB
```

```

flowchart LR
    subgraph img["Image blobs (what ships and is pullable)"]
        LN["Layer N"]
        ADDS /root/.ssh/id_rsa
        SECRET PRESENT
        LN1["Layer N+1"]
        whiteout /root/.ssh/.wh.id_rsa
        hides it in merged view
    end
    MV["Merged rootfs"]
    id_rsa: NOT visible
    ATT["Attacker: crane blob layer N"]
    -> extracts id_rsa intact
    LN --> MV
    LN1 --> MV
    LN --> ATT
    style LN fill:#fdd,stroke:#a33
    style ATT fill:#fdd,stroke:#a33

```

The rule: **RUN rm (or docker rmi, or deleting in a later COPY) never removes anything from the image.** It only adds a whiteout. Anything a secret ever touched a layer — a `.npmrc` with a token, a cloud credential file, a `.env`, a build SSH key, a `pip` config with an index password — is recoverable by anyone who can pull the image, forever, regardless of what later layers do. The correct responses are: never let the secret land in a layer (mount it as a **build secret** — `RUN --mount=type=secret` in BuildKit — so it exists only during that step and is never committed), and use multi-stage builds to leave build-time material behind entirely.

Multi-stage builds

A multi-stage build (Book 4, Chapter 2) uses several `FROM` stages in one `Dockerfile` and ships only the *last* stage's layers. Earlier stages — with compilers, source, package caches, SSH keys, and the entire toolchain — are built, used, and **discarded**; only files explicitly `COPY --from`'d into the final stage survive into the image:

```
# Stage 0: build — has the toolchain, source, and secrets. NOT shipped.
FROM golang:1.23 AS build
WORKDIR /src
COPY . .
RUN --mount=type=secret,id=gh_token \
    GOFLAGS=-mod=mod go build -o /out/app ./cmd/app

# Stage 1: final — inherits none of stage 0's layers.
FROM gcr.io/distroless/static:nonroot
COPY --from=build /out/app /app
USER 65532
ENTRYPOINT ["/app"]
```

The final image contains the `distroless/static` base and one file, `/app`. It has **no shell, no package manager, no compiler, no source, and no build key** — none of stage 0's layers are in its manifest, so none of stage 0's bytes ship. This is simultaneously the answer to the secret-leak (build secrets live and die in a discarded stage) and the on-ramp to minimal base images (Chapter 3 — Base Image Strategy), which shrink the attack surface we turn to now.

The image attack surface

Everything in the image is your attack surface and your vulnerability surface: whatever bytes are in those layers can be exploited if reachable, and every package is a component that scanners (Chapter 4) and SBOMs (Book 3) must account for. Map it in four buckets, roughly by size:

- **The base image.** The OS userland — `libc`, `coreutils`, `OpenSSL`, package manager, shell — is usually the *bulk of the components and the bulk of the vulnerabilities* in an image, often by an order of magnitude over your own code. A `debian:12`-based image inherits every Debian CVE in its installed set; that is why base image choice (Chapter 3) is the single highest-leverage security decision for an image, and why SBOMs for services (Book 3) are dominated by base-image packages.
- **Your app and its dependencies.** Your compiled binary or interpreted code plus the language dependency tree (Book 2 — the dependency supply chain). A vulnerable transitive `npm/pip/Go` module is in the image exactly as much as if you had written it.

- **Added tools and binaries.** `curl`, `bash`, `netcat`, `python`, a package manager left in for "debugging," a static `busybox` — each is a capability handed to whoever gets code execution in the container.
- **Embedded secrets and config.** Tokens, keys, `.env` files, world-readable credential files — including the ones "deleted" in a later layer (above), which are still present.

Tampering vectors and what content-addressing defends

An image can be attacked at several seams between build and run:

- A **modified layer** — a backdoor injected into a layer blob.
- A **swapped manifest** — pointing a tag at different config/layers.
- A **poisoned base image** — the upstream `FROM` was compromised, so everything built on it inherits the malware (Chapter 3; and the concentration argument below).
- A **malicious layer injected in the registry** — a compromised or malicious registry serving different bytes (Chapter 2 — Registries).

Content-addressing is the structural defense against undetected tampering. Because every object is named by the SHA-256 of its own bytes and the manifest commits to config and layers by digest, *any* change to *any* byte changes the top-level digest. You cannot alter a layer, swap a config, or substitute a manifest without producing a different `sha256:...`. So a verifier that **knows the digest it expects** and recomputes it on pull detects all four tampering vectors — a modified layer won't match its descriptor, a swapped manifest won't match the pinned digest, a poisoned or malicious registry cannot serve different bytes than the digest names (Book 5, Chapter 1). This is precisely why you **deploy and verify by digest, not by tag** — and why the referrers model (below) lets signatures and provenance *reference* an image by that digest.

What content-addressing does **not** defend: it guarantees *integrity* (these are the bytes you named), not *goodness* (these bytes are safe). A digest for a backdoored base image is a perfectly valid, verifiable digest. Integrity plus a *trusted source* and *signed provenance* is the full story (Book 5); the digest is the anchor all of it attaches to.

The mutable-tag problem

A **tag** — `:latest`, `:v1.4`, `:prod` — is a **mutable pointer**, a name in the registry that maps to a manifest digest and **can be repointed at any time**. A **digest** — `@sha256:...` — is **immutable**: it is the content, so it cannot point at anything else. This distinction is a supply chain hole, not a nicety:

```
sequenceDiagram
    participant CI as CI (verify)
    participant Reg as Registry
    participant K8s as Kubernetes (run)
    Note over CI,K8s: Deploying by TAG – TOCTOU window
    CI->>Reg: pull acme/app:v1.4
    Reg-->>CI: manifest sha256:G00D
    CI->>CI: scan + verify sha256:G00D (passes)
    Note over Reg: attacker (or a rebuild) repoints
    tag v1.4 -> sha256:EVIL
    K8s->>Reg: pull acme/app:v1.4
    Reg-->>K8s: manifest sha256:EVIL
    Note over K8s: runs EVIL – not what CI verified
```

This is a **time-of-check-to-time-of-use (TOCTOU)** gap: you verified `:v1.4` and it was `sha256:G00D`; between verification and run, the tag was repointed — by an attacker with registry write access, by a well-meaning `docker push` that overwrote it, or by an upstream `:latest` moving — and the pod runs `sha256:EVIL`. **The artifact you verified is not guaranteed to be the one that runs.** The fix is to collapse the check and use onto the *same* immutable name: resolve the tag to a digest once, then verify, deploy, and admit **by digest** (Book 5, Chapter 8 — Provenance Verification; Chapter 5 — Image Signing in Kubernetes). Kubernetes runs `image: acme/app@sha256:G00D` happily; admission controllers can *require* digest pins and reject bare tags. Tags are fine as human-facing labels; they must not be the thing you trust.

Bloat is attack surface

Every unnecessary thing in an image is a tool for whoever compromises the container. A shell lets an attacker pivot interactively; `curl/wget` let them pull a second-stage payload; a package manager lets them install one; `setuid` binaries are privilege-escalation primitives; leftover dev tooling is a ready-made toolkit. Minimizing the image is therefore a security control, not only an image-size optimization: fewer packages

mean fewer CVEs, fewer exploitation primitives, and a smaller SBOM to reason about. This is the motivation for **minimal and distroless base images** (Chapter 3), where a production image may contain a single static binary and its CA certificates — nothing for an attacker to live off of.

Image configuration and runtime posture

A handful of config fields set the container's default security posture. Deep runtime security is beyond supply chain, but **image-config hygiene is squarely in scope** because it is baked into the artifact you ship:

- **User**. If unset, the container's default user is **root (UID 0)**. A process running as root in-container is root against any kernel path it can reach and against mounted host resources; a container breakout from root is far worse than from an unprivileged UID. Set a non-root `USER` (a numeric UID, e.g. `USER 65532`, so it works without an `/etc/passwd` entry) in the image. This is a property of the *image*, independently enforceable by admission (`runAsNonRoot`), but it should be correct at build time.
- **Entrypoint / Cmd**. The default process. A shell-form entrypoint (`ENTRYPOINT nginx`) runs via `/bin/sh -c`, which requires a shell in the image (attack surface) and complicates signal handling; prefer exec form (`ENTRYPOINT ["nginx"]`).
- **Env**. Environment variables are stored **in the config in plaintext** and are visible to anyone who can `crane config` the image or read it in the registry. **Never bake secrets into ENV**. Inject them at runtime (Kubernetes Secrets, a secrets manager), not into the image.
- **ExposedPorts**, **Volume**, **WorkingDir**. Documentation and defaults rather than enforcement, but they shape how the image is run and are worth getting right.

None of these is enforced by the image alone — the runtime and admission layer enforce them — but a well-configured image makes the secure default the *easy* default across the fleet.

Signatures, SBOMs, and provenance: the OCI referrers model

We have said repeatedly that signatures, SBOMs, and provenance **attach to an image by its digest**. Here is the mechanism — and it is the bridge from this chapter to the rest of Book 6 and to Book 5.

The problem: an image is content-addressed and immutable, so you *cannot* add metadata to it — any change would change its digest and thus its identity. You need to associate *external* artifacts (a signature, an SBOM, an SLSA provenance attestation) with an existing image **without altering it**, and store them **in the same registry** so they travel with the image. OCI solves this with the **Referrers API**, standardized in the **Distribution Spec v1.1** and supported by the **Image Spec v1.1** `subject` field.

The idea: an attestation is itself a normal OCI manifest — with its own config and layers (the signature bytes, the SBOM document, the provenance JSON) — that carries a `subject` field. The `subject` is a *descriptor* pointing at the digest of the image it refers to, and an `artifactType` naming what kind of thing it is:

```
{
  "schemaVersion": 2,
  "mediaType": "application/vnd.oci.image.manifest.v1+json",
  "artifactType": "application/vnd.dev.cosign.artifact.sig.v1+json",
  "config": { "mediaType": "application/vnd.oci.empty.v1+json", "digest": "sha256:44136f...", "size": 2 },
  "layers": [
    { "mediaType": "application/vnd.dev.cosign.simplesigning.v1+json", "digest": "sha256:sig...", "size": 250 }
  ],
  "subject": {
    "mediaType": "application/vnd.oci.image.manifest.v1+json",
    "digest": "sha256:4b2c9e6f1a3d...", // <-- the image being signed
    "size": 2841
  }
}
```

A verifier asks the registry: *what refers to this image?*

```
$ crane referrers acme/app@sha256:4b2c9e6f1a3d...
```

which the registry answers (GET /v2/acme/app/referrers/sha256:4b2c...) with an **image index** whose entries are exactly the manifests whose **subject** is that image digest, each tagged with its **artifactType**. You then fetch the signature, SBOM, or provenance you want and verify it.

```
flowchart BT
    IMG["Image manifest  
sha256:4b2c9e6f...  
(immutable identity)"]
    SIG["Signature manifest  
artifactType: cosign sig  
subject -> sha256:4b2c..."]
    SBOM["SBOM manifest  
artifactType: spdx+json  
subject -> sha256:4b2c..."]
    PROV["Provenance manifest  
artifactType: slsa provenance  
subject -> sha256:4b2c..."]
    SIG -->|subject| IMG
    SBOM -->|subject| IMG
    PROV -->|subject| IMG
    Q["crane referrers sha256:4b2c..."]
    GET["/v2/.../referrers/<digest>"] -->|returns index of| SIG
    Q --> SBOM
    Q --> PROV
```

Before the Referrers API existed, cosign (Book 5, Chapter 3 — Sigstore) solved the same problem with a **tag convention**: for an image with digest `sha256:4b2c...`, cosign stores the signature as an ordinary image **tagged** `sha256-4b2c...sig` in the same repository (note the `sha256-` prefix and the `.sig` suffix — a tag, because the digest character `:` is not tag-legal). Attestations went to `...att`, SBOMs to `...sbom`. It works on any Distribution-conformant registry — no v1.1 support needed — which is why it is still widespread, but it clutters the tag namespace and does not let a registry answer "what refers to this?" natively. The Referrers API is the successor: metadata is discoverable by a first-class API, GC understands the **subject** link, and (via a fallback where the registry doesn't yet support referrers) tooling can emulate it with a `sha256-<hex>` referrers tag that holds the index.

The load-bearing point for the rest of Book 6: **the image digest is the join key for the entire security ecosystem**. Signatures (Book 5), SBOMs (Book 3), VEX (Book 3, Chapter 6), and SLSA provenance (Book 4, Chapter 3) all reference the image by the same immutable `sha256:...`,

stored alongside it, and admission control (Chapter 6) resolves an image to its digest, fetches its referrers, and enforces policy on them. None of that works if you cannot say precisely what "the image" is — which is why the format had to come first.

Distributed-systems lens

At the scale you operate — thousands of services, hundreds of teams, images rebuilt many times a day — the format's properties stop being trivia and become fleet dynamics.

Shared base layers concentrate risk. Content-addressed layers mean your entire fleet's images are almost certainly built on a small number of shared base images, and the registry and every node store each base blob *once*. This is wonderful for cost and cache-hit rate, and dangerous for blast radius: a single CVE in a widely used base layer is instantly a vulnerability in *every* image built on it, and a *poisoned* shared base layer (Chapter 3) spreads to everything downstream with no further attacker effort. This is the concentration argument of Book 1, Chapter 9 — Distributed Systems Lens, in container form: the same sharing that gives you leverage gives an attacker leverage. Govern base images centrally (a small set of vetted, signed, continuously rebuilt bases), because a good base is a fleet-wide control and a bad one is a fleet-wide incident.

Dedup and caching cut both ways. The digest-keyed blob store that lets a 4 GB base image cost one pull per node also means that if a bad blob lands under a trusted digest — which content-addressing *prevents* for a fixed digest but not against a *newly pushed* malicious layer reused across images — it, too, propagates by the same efficient machinery. Integrity (the digest is what it says) is not the same as trust (the digest should be there); the fleet needs both.

Deploy-by-digest is the fleet's core integrity practice. Mutable tags create a TOCTOU window at *every* deployment across the fleet; the only immutable coordinate is the digest. Resolving to digests at build/release time, recording them, and admitting only digest-pinned images (Chapters 5–6) closes that window uniformly. It also makes the fleet *auditable*: a digest is a precise, global name you can correlate across the registry, the transparency log (Book 5, Chapter 5), your SBOM inventory (Book 3, Chapter 9), and your admission logs.

The registry is the shared substrate, and referrers are its metadata backbone. Every image, layer, signature, SBOM, and provenance for the whole fleet lives in the registry (Chapter 2), keyed by digest, with attestations hanging off images via `subject`. The referrers graph is the fleet's software supply chain metadata layer (Book 3, Chapter 5 — SBOMs at scale; Book 5). Understanding the OCI format is the prerequisite to securing any of it: signing signs a manifest digest, scanning enumerates a config's layers, admission checks a subject-linked signature, and all of them agree on identity only because the format defines it precisely.

Key takeaways

- An **OCI image is a content-addressed graph**: a **manifest** (JSON of descriptors) pointing to a **config** (JSON: `env/entrypoint/user, roots.diff_ids, history`) and an ordered list of **layer** tar blobs, optionally under an **index** for multi-platform. All named by **SHA-256**.
- The **manifest's own digest is the image's identity** — the thing you sign, pin, and deploy. It differs from a layer's `diff_id`: **manifest layer digests are of the compressed blob; config `diff_id`s are of the uncompressed tar**.
- Layers stack via an **overlay/union filesystem**; they are read-only and immutable. **Deletion is a whiteout, not an erasure** — a secret added in one layer and "removed" in a later `RUN` is **still in the earlier layer's blob and fully recoverable**. Use **build secrets** and **multi-stage builds** to keep secrets and build junk out of shipped layers.
- The **image attack surface** is base OS packages (usually the bulk of vulns and components), app dependencies, added tooling, and embedded secrets/config. **Bloat is attack surface**; minimal and distroless bases (Chapter 3) shrink it.
- **Content-addressing defends integrity** (any tampered byte changes the digest) but not goodness. A **tag is a mutable pointer; a digest is immutable**. Deploying by tag is a **TOCTOU hole** — verify and run the *same digest*. Deploy by digest across the fleet.
- **Image-config hygiene** (non-root `USER`, exec-form `entrypoint`, **no secrets in `ENV`**) is baked into the artifact and is in scope for supply chain.

- Signatures, SBOMs, and provenance attach to an image **by its digest** via the **OCI 1.1 Referrers API** (a manifest with a `subject` descriptor pointing at the image) or the older cosign `sha256-...sig` tag convention. **The image digest is the join key for the whole security ecosystem.**

Further reading

- **OCI Image Format Specification** — manifest, config, descriptor, index, and layer media types and the content-addressing model; the `subject` field and `artifactType` in v1.1. <https://github.com/opencontainers/image-spec>.
- **OCI Distribution Specification** — the `/v2/` pull/push API, pulling by tag vs digest, and the **Referrers API** and its tag-schema fallback (v1.1). <https://github.com/opencontainers/distribution-spec>. Registry architecture and threats are Chapter 2.
- **OCI Runtime Specification** — the on-disk bundle and `config.json` a runtime executes. <https://github.com/opencontainers/runtime-spec>.
- **go-containerregistry / crane** — `crane manifest`, `crane config`, `crane digest`, `crane referrers`, `crane blob`, `crane export`; the cleanest way to inspect the format directly. <https://github.com/google/go-containerregistry>.
- **Docker history and image spec** — `docker history --no-trunc` for reading the build record and spotting secret-leak layers. <https://docs.docker.com/reference/cli/docker/image/history/>.
- **BuildKit build secrets and multi-stage builds** — `RUN --mount=type=secret` and `COPY --from`; the correct way to keep secrets and toolchains out of shipped layers. <https://docs.docker.com/build/>. Reproducibility and hermeticity are Book 4, Chapter 2.
- **cosign and the OCI referrers/sig conventions** — how signatures, attestations, and SBOMs are stored against an image digest. <https://docs.sigstore.dev/> (Book 5, Chapter 3 — Sigstore Architecture).
- Cross-references: Book 5, Chapter 1 (content addressing, digests), Chapters 3 and 8 (signing and verifying by digest); Book 4, Chapters 2-3 (reproducible builds, SLSA provenance); Book 3 (SBOMs — Chapters 5, 6, 9); Book 2 (dependency supply chain); Book 1, Chapter 9 (distributed-systems lens, concentration); Book 6, Chapters 2-6

(registries, base images, scanning, signing in Kubernetes, admission and policy).

Chapter 2 — Container Registries: Architecture, Trust, and Threats

What this chapter covers. A container registry is where images live between the build that produces them and the node that runs them. Every `docker pull`, every `kubelet` image fetch, every GitOps deploy terminates at a registry. That makes the registry the single most load-bearing piece of distribution infrastructure in a containerized backend: it is the OCI analogue of the package registries you met in Book 2, Chapter 1 — Package Managers and Registries: Architecture and Trust Models — and it sits at exactly the same place in the threat model. If you understood *why* an internal package proxy is a control point and not a convenience (Book 2, Chapter 8 — Vendoring, Mirroring, and Internal Registries), most of this chapter's conclusions will feel familiar; the mechanics differ because the artifact is an OCI image, not an npm tarball, but the trust questions are identical. This chapter takes the registry apart at the protocol level (the OCI Distribution Specification), surveys the real registry landscape without marketing gloss, builds the trust model precisely — *what does a pull actually prove?* — and then enumerates the threats and the controls that answer them. It assumes the image internals of Chapter 1 (manifests, config, layers, digests, the mutable-tag-versus-immutable-digest distinction) and hands off enforcement to Chapters 5 and 6 (signing verification and admission control).

Learning goals — after this chapter you should be able to:

- Describe what an OCI registry stores and serves, and trace the **pull flow** and **push flow** of the OCI Distribution Specification at the level of individual HTTP requests, by digest and by tag.
- Explain **content-addressable storage**, blob deduplication, and the **referrers API**, and how signatures, SBOMs, and attestations attach to an image as referring artifacts.
- Contrast the real registry options — Docker Hub, GHCR, ECR, Google Artifact Registry, ACR, Harbor, Quay, Artifactory/Nexus, Zot,

and the reference `distribution` implementation — on hosting model and built-in scanning/signing.

- State precisely **what a pull trusts** — and what it does not — and enumerate the registry threat model: registry compromise, tag-mutability image replacement, malicious public images, push-credential theft, misconfigured/anonymous registries, blob and cache poisoning, and the image analogue of dependency confusion.
- Map each threat to a concrete control: scoped/short-lived credentials, sign-and-verify, registry-integrated scanning, allowed-registries and signing admission policy, tag immutability, and an internal pull-through registry as the fleet-wide chokepoint.
- Argue the distributed-systems case for treating the internal registry as **tier-0 fleet infrastructure**: HA, audited, deploy-by-digest, pull-only-from-approved-registries.

A note on scope. This chapter is about the registry as *architecture and trust point*. The image format it serves is Chapter 1. Base-image selection is Chapter 3. What scanning actually finds and how to triage it is Chapter 4. How you *verify* a signature at admission time — the control that makes signing worth doing — is Chapters 5 and 6. This chapter tells you what the registry is, what can go wrong at it, and how to wrap it in controls.

What a registry is, mechanically

Strip the branding and a container registry is a content-addressable blob store with a thin metadata layer and an access-control front door, speaking one HTTP protocol: the **OCI Distribution Specification** (current stable v1.1, which folded in the referrers API). Docker Hub, ECR, Harbor, GHCR, and the 150-line reference implementation all speak the same wire protocol; a client that can pull from one can pull from any of them. That interoperability is the whole point of standardizing distribution, and it is why "which registry" is mostly an operational and trust decision, not a compatibility one.

A registry stores exactly three kinds of object, all addressed under a **repository name**:

1. **Blobs** — opaque, immutable byte strings addressed by their content digest, almost always `sha256:<hex>`. Layer tarballs are blobs. The image config JSON is a blob. Any attached artifact's content is a blob.

A blob's name *is* the hash of its bytes; this is the property everything else rests on.

2. **Manifests** — small JSON documents that reference blobs by digest and give them meaning. An image manifest lists the config blob and the ordered layer blobs (Chapter 1). A manifest is itself addressable by its own digest.
3. **Tags** — mutable human-readable pointers (`latest` , `v1.2.3` , `sha-9f3a1c`) that resolve to a manifest digest within a repository. A tag is the *only* mutable thing in the system, and — as we will belabor — that mutability is the root of an entire threat class.

The full name of an image is `registry-host[:port]/repository[:tag|@digest]`. In `ghcr.io/acme/payments-api:v1.4.2`, `ghcr.io` is the registry host, `acme/payments-api` is the repository (namespaced by org), and `v1.4.2` is a tag. Replace `:v1.4.2` with `@sha256:9f3a1c...` and you have pinned the exact manifest by digest, bypassing the mutable tag entirely. Hold that distinction; it is the hinge of the security story.

The pull flow, request by request

A pull is a sequence of GETs that walks the manifest → config → layers graph. Suppose a kubelet pulls `registry.internal/payments-api:v1.4.2`.

1. **Resolve and fetch the manifest.** GET `/v2/payments-api/manifests/v1.4.2` with an `Accept` header listing the media types the client understands (image manifest, image index, and the legacy Docker schema). The registry returns the manifest JSON and, in the `Docker-Content-Digest` response header, the manifest's digest. If the reference had been a digest (`.../manifests/sha256:9f3a...`) the client would verify the returned bytes hash to exactly that digest and reject any mismatch. If the manifest is an **image index** (multi-arch, Chapter 1), the client picks the entry matching the node's platform and does a second manifest GET by that child digest.
2. **Fetch the config.** The manifest names a config blob by digest; GET `/v2/payments-api/blobs/sha256:<config>` retrieves it. The config carries the `rootfs.diff_ids`, `env`, `entrypoint`, and `history`.
3. **Fetch the layers.** For each layer digest in the manifest, GET `/v2/payments-api/blobs/sha256:<layer>`, typically served as a redirect to a CDN or object store. Layers the node already has (by digest) are

skipped — this is why a shared base layer downloads once per node, not once per image.

4. **Verify.** A correct client hashes every blob it downloads and checks it against the digest it was asked for. This is the integrity guarantee the registry gives you *for free*: if the bytes were corrupted or swapped in transit or at rest, the digest will not match. Note carefully what this does and does not prove — it proves the bytes match the digest you asked for; it proves nothing about whether that digest is the one you *should* have asked for. That gap is the entire signing story.

```
sequenceDiagram
    participant C as Client (kubelet/docker)
    participant R as Registry (OCI Distribution API)
    participant S as Blob store / CDN
    Note over C,R: Pull registry/payments-api:v1.4.2
    C->>R: GET /v2/payments-api/manifests/v1.4.2 (Accept: manifest types)
    R-->>C: 200 manifest JSON + Docker-Content-Digest: sha256:9f3a...
    Note over C: if image index, pick platform, GET child manifest by digest
    C->>R: GET /v2/payments-api/blobs/sha256:
    R-->>C: 200 config JSON
    loop each layer in manifest
        C->>R: GET /v2/payments-api/blobs/sha256:
        R-->>S: 307 redirect
        S-->>C: 200 layer blob (skip if node already has digest)
    end
    Note over C: hash every blob, verify == requested digest
```

The push flow, request by request

A push runs the graph in reverse: **blobs first, manifest last**, so the manifest never references a blob the registry does not yet have.

1. **Check what already exists.** For each blob, the client may `HEAD /v2/<repo>/blobs/sha256:<digest>`. A `200` means the registry already has that content and the upload is skipped — cross-image, cross-tag **deduplication** falls out of content-addressing for free. A shared base layer is uploaded by the first image that uses it and never again.
2. **Upload missing blobs.** `POST /v2/<repo>/blobs/uploads/` opens an upload session and returns a `Location`; the client `PUT`s the bytes (monolithically or in chunks) and finalizes with the `?digest=sha256:<digest>` query parameter. The registry recomputes

the digest and rejects the upload if the bytes do not match — you cannot push a blob under a digest that is not its actual hash.

3. **Push the manifest.** `PUT /v2/<repo>/manifests/<tag-or-digest>` uploads the manifest JSON. This is the step that makes an image *appear at a tag*. The registry validates that every blob the manifest references is present.

The ordering matters for a subtle reason: because the manifest is written last and by tag, the act of "publishing to `:latest`" is a single mutable pointer write over an immutable content graph. Everything the tag points at is content-addressed and tamper-evident; the *pointer* is not. Attacks live in that pointer.

```
sequenceDiagram
    participant B as Builder / CI
    participant R as Registry
    Note over B,R: Push registry/payments-api:v1.4.2
    loop each blob (layers + config)
        B->>R: HEAD /v2/payments-api/blobs/sha256:
        alt already present
            R-->>B: 200 (skip upload – dedup)
        else missing
            B->>R: POST /v2/payments-api/blobs/uploads/
            R-->>B: 202 Location
            B->>R: PUT ?digest=sha256: (bytes)
            R-->>B: 201 Created
        end
    end
    B->>R: PUT /v2/payments-api/manifests/v1.4.2 (manifest JSON)
    R-->>B: 201 Created (tag now points at manifest digest)
```

Referrers: signatures, SBOMs, and attestations as first-class objects

The interesting supply-chain metadata about an image — its cosign signature, its SBOM, its SLSA provenance attestation — is not *inside* the image. It is stored as separate OCI artifacts that *refer to* the image by digest, and OCI Distribution v1.1 standardized how to find them: the **referrers API**. Each referring artifact is itself a manifest with a `subject` field holding the digest of the image it describes and an `artifactType` naming what it is. A client asks `GET /v2/<repo>/referrers/sha256:<image-digest>`, optionally filtered by `artifactType`, and gets back an image index enumerating everything that refers to that image: "here are the two

signatures, the CycloneDX SBOM, and the provenance attestation attached to this digest."

This is what lets an admission controller, given only an image digest, discover and verify the signature that vouches for it (Chapters 5 and 6) without a side-channel database. Two mechanics matter for security. First, referrers hang off the *digest*, not the tag — a signature is bound to specific content, so re-tagging cannot silently move a signature onto different bytes. Second, on registries that predate v1.1 or disable the referrers endpoint, tooling falls back to a **tag schema**: the referring artifact is pushed to a tag derived from the subject digest (e.g. `sha256-9f3a1c....sig`). It is the same idea projected onto older registries; be aware that a registry lacking real referrers support degrades this discovery, which matters when you standardize on cosign or SBOM attach across a heterogeneous fleet.

Content-addressing and deduplication

Everything above rests on one property: **a blob's identity is the hash of its content**. Consequences worth stating explicitly, because the controls later depend on them:

- **Deduplication is automatic.** Ten images built `FROM debian:12` share the base layers; each unique layer is stored once per repository (or once per registry, depending on the backend). This is why registries scale and why a node caches by digest.
- **Integrity is intrinsic to the address.** You cannot alter a blob's bytes and keep its name. Any tampering changes the digest, so a client pinning a digest is tamper-evident by construction — the registry would have to break SHA-256 to serve different bytes under the same digest.
- **The tag is the only lie you can tell.** Since only the tag→manifest mapping is mutable, every integrity attack that does not break SHA-256 reduces to *changing what a tag points at* or *convincing you to pull a name you should not trust*. Keep this reduction in mind; it organizes the entire threat section.

The registry as content store *and* access-control point

A registry is two things wearing one uniform. It is a **content store** (the blobs and manifests above) and it is an **access-control point** — the gate

that decides who may pull and who may push which repository. The Distribution spec deliberately does not mandate an authN/authZ scheme; it defines a 401 challenge and a bearer-token dance (the "docker auth" token flow), and each registry plugs in its own identity model behind it. That separation is why the *same* image protocol fronts everything from an anonymous public mirror to an IAM-gated private ECR repository. For our purposes the crucial point is that push and pull are distinct, per-repository capabilities, and the security of your images depends as much on who holds the *push* capability as on the bytes themselves. We return to this under credential theft.

The registry landscape

The registries you will actually encounter differ on three axes that matter: **hosting model** (public SaaS, cloud-managed, or self-hosted), **built-in security features** (integrated scanning, signing, policy), and **operational posture** (rate limits, availability, cost). They do *not* differ on the wire protocol — all speak OCI Distribution — so switching is an operational migration, not a rewrite.

Registry	Hosting	Built-in scanning	Signing / policy	Notes
Docker Hub	Public SaaS	Optional (Docker Scout)	No native enforcement	Official-images program; library/ namespace; anonymous-pull rate limits
GitHub Container Registry (ghcr.io)	Public SaaS	Via GitHub code/dep tooling	Cosign-friendly; per-repo/org visibility	Tied to GitHub identity; convenient for GitHub Actions OIDC
Amazon ECR	Cloud-managed (AWS)	Basic (native) + enhanced (Inspector)	IAM-gated; tag immutability; pull-through cache	IAM is the authZ model; private by default

Registry	Hosting	Built-in scanning	Signing / policy	Notes
Google Artifact Registry (GAR)	Cloud-managed (GCP)	On-push scanning (Container Analysis)	IAM-gated; remote/ virtual repos	Successor to Container Registry (GCR); multi-format
Azure Container Registry (ACR)	Cloud-managed (Azure)	Defender for Cloud integration	Entra/RBAC; tokens; retention	Tasks for in-registry build; geo-replication
Harbor	Self-hosted (CNCF)	Built-in (Trivy)	Cosign/ Notation verify; project policy; quarantine	The reference "policy-enforcing private registry"
Quay / Project Quay	Self-hosted or SaaS	Built-in (Clair)	Robot accounts; per-repo notifications	Clair pioneered layer-level vuln scanning
JFrog Artifactory / Sonatype Nexus	Self-hosted or SaaS	Add-on (Xray / IQ)	Multi-format incl. OCI; RBAC	The multi-format proxy of Book 2, Ch 8 — hosts images too
Zot	Self-hosted	Optional (Trivy)	OCI-native; referrers-first	Lightweight, standards-focused; good air-gap fit
distribution/ distribution	Self-hosted	None	None	The reference implementation; a store, not a platform

A few landscape realities worth stating plainly, because they drive architecture decisions:

- **Docker Hub rate limits are an availability dependency, not a footnote.** Anonymous and free-tier pulls are rate-limited by source IP over a rolling window. In a Kubernetes cluster, *every node* NAT'd

behind one egress IP shares that budget, so a modest fleet can exhaust anonymous pulls and see `ImagePullBackOff` fleet-wide during a scale-up or a node-image refresh. This single fact has pushed more organizations toward an internal pull-through mirror than any security argument — and, conveniently, the mirror is also the security chokepoint. We come back to this in the distributed-systems lens.

- **The official-images program and the `library/` namespace.**

Docker Hub curates a set of "Docker Official Images" (`debian` , `postgres` , `nginx` , ...) that live in the special `library/` namespace and are referenceable by bare name (`postgres:16` is `docker.io/library/postgres:16`). These are reviewed and rebuilt for CVEs, which makes them *relatively* trustworthy — but "official" is a curation signal, not a cryptographic guarantee, and the bare-name convenience is exactly what namespace-confusion attacks abuse (below).

- **"Built-in scanning/signing" means integrated, not automatic.**

Harbor ships Trivy and can *enforce* on push; ECR/GAR/ACR can scan on push and expose findings; but none of this blocks a deploy unless you wire it to a policy that fails closed. The registry feature is the sensor; the enforcement is admission control (Chapter 6). Do not confuse "the registry can scan" with "vulnerable images cannot ship."

The recurring architectural conclusion — the mirror of Book 2, Chapter 8 for images — is that a serious backend org fronts all of these with an **internal registry** it controls, and points its builders and clusters at that one endpoint. Everything below explains why that is a security decision and not just an ops one.

The trust model: what a pull actually proves

Here is the load-bearing question, asked precisely. When a node runs `registry/payments-api:v1.4.2` and the container starts, what has actually been *proven* about those bytes?

- **The registry served bytes that hash to some digest, and TLS protected them in transit.** That is real: the digest check means the bytes were not corrupted or swapped between the registry's store and your node, and TLS means no on-path attacker rewrote them. Both are worth having.

- **If you pinned a digest, you got exactly that content.** Digest pinning is tamper-evident: the bytes provably match the digest you named. But it says nothing about whether that digest is *the right one* — only that you got what you asked for.
- **If you pulled by tag, you got whatever the tag pointed at at pull time.** The tag is mutable. You are trusting the registry to have resolved the tag honestly, *and* trusting whoever last pushed that tag, *and* trusting that no one has repushed the tag since you last looked. That is three separate trusts, none of them cryptographic.
- **Without signing, you have proven nothing about authenticity.** The digest ties bytes to a name; it does not tie them to a *publisher*. A registry (or anyone with push access) can put any bytes under any tag, sign nothing, and every puller's digest check will pass — because the check only verifies bytes-match-digest, and the attacker chose both.

State it as a slogan: **a plain pull trusts the registry to serve the right bytes and trusts the tag to point at the right image; it does not, by itself, authenticate the image at all.** The digest gives you integrity relative to a name you chose; only a signature you *verify* (Chapters 5–6) gives you authenticity relative to a publisher you chose. Everything in the threat model below is a way of exploiting the gap between "integrity relative to a name" and "authenticity relative to a publisher."

Threats

The registry sits at the same place in the container supply chain that a package registry sits in the dependency chain (Book 1, Chapter 2 — A Taxonomy of Supply Chain Attacks), and the threats rhyme. Six matter.

```

flowchart TB
    IMG["Image at registry/repo:tag"]
    subgraph threats["Registry threat map"]
        T1["Registry compromise  
attacker controls the store"]
        T2["Tag-mutability replacement  
repush malicious over :latest / :v1.2"]
        T3["Malicious public image  
typosquat / fake-official / backdoor"]
        T4["Push-credential theft  
CI token leaks → attacker pushes"]
        T5["Misconfigured registry  
anonymous pull/push, exposed"]
        T6["Blob / cache poisoning  
pull-through mirror poisoned"]
    end
    T1 --> IMG
    T2 --> IMG
    T3 --> IMG
    T4 --> IMG
    T5 --> IMG
    T6 --> IMG
    IMG --> FLEET["Every node that pulls → fleet-wide blast radius"]

```

1. Registry compromise

If an attacker controls the registry — through a platform compromise, a stolen admin credential, or a vulnerability in a self-hosted deployment — they can replace or add images at will and serve them to everyone who pulls. This is the container analogue of a package-registry compromise (Book 1), and its blast radius is the same shape: the registry is a single point of distribution, so a compromise is a fleet-wide malware delivery channel. Note the one thing a registry compromise *cannot* silently do: it cannot serve different bytes under a digest you pinned without breaking SHA-256, and it cannot forge a signature it does not have the key for. Those two facts — digest pinning and verified signing — are precisely the controls that survive a registry compromise, which is why they are the load-bearing defenses and the registry's own integrity check is not.

2. Tag-mutability image replacement

This is the most common and most underrated attack, and it requires no registry compromise at all — only push access to one repository. Because a tag is a mutable pointer, anyone who can push can repush a *different* image to an *existing* tag. Push a backdoored image to `:latest`, or to

:v1.2 after v1.2 has been "released," and every consumer who pulls by that tag from that moment on gets the swapped bytes. Their digest check passes — the attacker recomputed the digest for their own bytes. Nothing in the plain pull path detects it. CI that runs `docker pull app:latest && deploy` will faithfully deploy the poison; a Kubernetes Deployment with `imagePullPolicy: Always` and a mutable tag will pull the swap on the next reschedule.

The defense is the digest/tag distinction from Chapter 1, applied ruthlessly: **deploy by digest, not by tag**. A manifest reference of `@sha256:9f3a1c...` cannot be repointed; it names content, and content is immutable. A tag repush produces a *new* digest, and a deploy pinned to the old digest simply keeps running the old, known content. The tag becomes a human-friendly label used at *selection* time (in the build/release pipeline, under access control), while the thing that reaches the cluster is a digest.

```
flowchart TB
    subgraph attack["Attack: deploy by tag"]
        A1["Attacker pushes malicious image to :latest"]
        A2["Tag :latest now → sha256:BAD"]
        A3["Consumer: docker pull app:latest"]
        A4["Runs sha256:BAD – digest check passes on attacker's bytes"]
        A1 --> A2 --> A3 --> A4
    end
    subgraph defend["Defense: deploy by digest"]
        D1["Release pins app@sha256:GOOD"]
        D2["Attacker repushes :latest → sha256:BAD"]
        D3["Consumer: pull app@sha256:GOOD"]
        D4["Still runs sha256:GOOD – pin unaffected by tag repush"]
        D1 --> D3 --> D4
        D2 -. "no effect on pinned deploy" .-> D4
    end
```

Immutable tags (below) are the complementary control: some registries can *refuse* a repush over an existing tag, converting this attack from "silent swap" into "push rejected." Digest pinning defends the consumer; tag immutability defends the tag. Use both.

3. Malicious public images

Public registries host user-uploaded images, and some of those images are hostile by design. The recurring, well-documented pattern on Docker Hub over the past several years has been **cryptomining images**: images

that run a miner on pull-and-run, uploaded under plausible-looking names and, in some campaigns, pulled millions of times before takedown. Security vendors and Docker have removed such images repeatedly; this is an *ongoing* problem, not a single historical incident, and I will not attach invented counts or dates to it — the accurate statement is that malicious images, cryptominers prominent among them, are a persistent presence on open public registries and are periodically discovered and removed. The delivery tricks are familiar from Book 2, Chapter 3 — Dependency Confusion, Typosquatting, and Namespace Attacks:

- **Typosquatting.** An image named to resemble a popular one (a transposed character, an extra word, a plausible fork name) so a fat-fingered or copy-pasted reference lands on the attacker's image.
- **Fake "official" images.** Images named or described to imply first-party or vendor provenance they do not have — trading on the reputation of a real project. The `official-images/ library/` curation exists precisely to give the genuine ones a distinguishable home.
- **Namespace confusion.** The bare-name convenience of `library/` cuts both ways: `postgres` resolves to the curated `library/postgres`, but users frequently assume a bare-looking or vendor-looking name is official when it is an ordinary user repository. A reference to `someuser/postgres` is *not* the official image, and nothing in the pull enforces that you noticed.
- **Backdoored images.** Otherwise-functional images carrying an added payload — a reverse shell, credential exfiltration, a miner — so the image "works," passing casual inspection, while doing extra work.

The through-line: on a public registry, an image name plus a tag is a *claim*, not a credential. The defense is not to trust names from open registries at all — pull only from an internal registry that has vetted, scanned, and (ideally) re-signed the images it holds, and enforce that with allowed-registries policy at admission (below).

4. Push-credential theft

Push access is the crown jewel, and in a modern org it usually lives in CI (Book 4, Chapter 6 — Securing CI/CD Credentials and OIDC). A registry push credential leaked from a CI job — a long-lived Docker Hub token in a plaintext secret, a static ECR credential in an environment variable, a robot-account password in a build log — hands the attacker the ability to

push malicious images under your legitimate repository names. This is the container version of the Codecov and event-stream credential-abuse pattern (Book 1): the attacker does not need to compromise the registry platform if they can just log in as a publisher. Worse, a stolen push credential composes with the tag-mutability attack: the attacker pushes over your existing, trusted tags, and consumers deploying by tag ship the swap. The mitigations are the CI-credential mitigations — short-lived OIDC/workload-identity tokens instead of long-lived secrets (Book 4, Chapter 6; Book 5, Chapter 4 — Keyless Signing and Workload Identity), least-privilege scoping (push to one repository, not the whole registry), and, as the safety net that survives a leak, verified signing so that unsigned or wrongly-signed pushes fail at admission even when they carry valid push credentials.

5. Unauthenticated and misconfigured registries

A depressingly common real-world finding is a registry exposed to the network with **no authentication** — the reference `distribution` implementation deployed for "internal" use and then reachable, anonymous **pull** enabled where it should not be, or worst of all anonymous **push** enabled, letting anyone add images. Internet-wide scans routinely turn up open container registries; the reference implementation ships without authentication by design (it is a store, not a platform) and it is easy to stand one up and forget the front door. The consequences are direct: anonymous pull leaks proprietary images and their secrets (baked-in credentials, source, config — Chapter 1's attack surface), and anonymous push lets an attacker seed malicious images that your own systems then pull by name. The control is unglamorous: authentication on by default, network policy so the registry is not reachable from where it should not be, and periodic checks (including external attack-surface scanning) that no registry is answering `/v2/` anonymously.

6. Blob poisoning and pull-through cache poisoning

A **pull-through cache** (a mirror that fetches from an upstream public registry on first request and serves the cached copy thereafter) is a control point, but a poorly-secured one is also a poisoning target. If an attacker can influence what the mirror caches for a given reference — by racing a tag, by exploiting a mirror that keys its cache on a mutable tag rather than on the resolved digest, or by compromising the upstream —

the mirror can serve poisoned content to every downstream client from a single successful poisoning. The digest guarantee still holds *if consumers pin digests*: a cache cannot serve different bytes under a pinned digest. The risk concentrates on tag-based pulls through the mirror. The mitigation is to have the mirror resolve and pin digests (cache by digest, not by tag), verify signatures on ingest, and, again, have consumers deploy by digest so a poisoned tag cannot move a running workload.

The image analogue of dependency confusion

Finally, the container mirror of dependency confusion (Book 2, Chapter 3). If your internal image name — say `acme/payments-api` — is *also resolvable on a public registry*, and your clients or clusters can reach that public registry, then a client that fails to specify the registry host, or a mirror/registry-mirror configuration that falls back to a public registry, can be steered to an attacker's public image of the same repository name. The container ecosystem is somewhat more resistant than npm/PyPI here because image references are usually *fully qualified* with a registry host (`registry.internal/acme/payments-api`), and Kubernetes image references are explicit. But the failure modes exist: registry-mirror configs that add a public fallback, tooling that defaults an unqualified name to `docker.io`, and multi-registry search paths (the historical `search-registries` behavior in some runtimes). The defense is to **fully qualify every image reference**, disable unqualified-search and public fallback in runtime and mirror configuration, and enforce an allowed-registries admission policy so a reference to an unexpected registry host is rejected before it schedules.

Controls

The controls are not registry features in isolation; they are a layered system in which the registry is a sensor and a chokepoint, and *enforcement* happens at admission. Map them to the threats explicitly.

Threat	Primary control(s)
Registry compromise	Deploy by digest; verify signatures at admission (survives a compromised store)
Tag-mutability replacement	Deploy by digest; immutable tags ; sign-and-verify

Threat	Primary control(s)
Malicious public images	Pull only from internal registry; scan on ingest; allowed-registries admission policy
Push-credential theft	Short-lived OIDC/workload-identity; least-privilege per-repo scoping; sign-and-verify as safety net
Misconfigured / anonymous registry	Auth on by default; network policy; external attack-surface monitoring
Blob / cache poisoning	Mirror caches by digest; verify signatures on ingest; consumers deploy by digest
Image dependency confusion	Fully-qualified references; allowed-registries policy; disable public fallback/search

Authentication and authorization

Treat push and pull as distinct, least-privilege capabilities. Use **robot / service accounts** and **scoped tokens** rather than human or shared credentials: a CI job that publishes `payments-api` should hold a credential that can push *only* `payments-api`, not the whole registry, and consumers should hold pull-only credentials. Every managed registry supports this — ECR through IAM policies scoped to repository ARNs, GAR/ACR through cloud RBAC, Harbor and Quay through per-project robot accounts. Prefer **OIDC / workload identity** for pushes so the build platform presents a short-lived, verifiable token instead of a long-lived secret that can be stolen and replayed (Book 4, Chapter 6; Book 5, Chapter 4). The security property you want is that a leaked credential is both narrow (one repo) and short-lived (minutes), so the blast radius and the window are both small.

Content trust: sign, and verify

The registry's own integrity check proves bytes-match-digest, which — as we established — does not authenticate the publisher. The control that does is **signing the image and verifying the signature on the consuming side**. In the Sigstore world (Book 5, Chapter 3 — Signing Software Artifacts with Cosign), the pipeline signs the image *by digest*:

```
# Sign the image by digest (keyless / OIDC-backed), attaching a
signature
# as a referring artifact discoverable via the referrers API.
cosign sign --yes \
  registry.internal/payments-api@sha256:9f3a1c...
```

and the *consumer* — a Kubernetes admission controller, not a human — verifies it before the image is allowed to run:

```
cosign verify \
  --certificate-identity-regex '^https://github.com/acme/.+/.github/
workflows/.+$.+' \
  --certificate-oidc-issuer https://
token.actions.githubusercontent.com \
  registry.internal/payments-api@sha256:9f3a1c...
```

The signature is stored as a referrer alongside the image, discoverable by the referrers API without a side database. The critical, easily-missed point: **signing is worthless without verification**. A signature nobody checks defends nothing; the value materializes only when an admission policy *fails closed* on a missing or wrong signature (Chapters 5 and 6). This is the control that survives a full registry compromise and a stolen push credential — the attacker can serve any bytes and hold any push token, but cannot produce a valid signature from an identity your policy trusts. Deploy by digest is its precondition: you sign a digest and verify a digest, so the reference that reaches the cluster must be a digest.

Registry-integrated scanning

Registries that embed a scanner — Harbor with Trivy, Quay with Clair, ECR basic/enhanced, GAR Container Analysis, ACR with Defender — let you **scan on push** and surface (or block on) vulnerable or malicious images (Chapter 4 — Image Scanning and Vulnerability Management). The architectural value is placement: the registry sees every image once, at ingest, so it is the natural chokepoint to scan *all* images with *one* integration. Two cautions. First, scanning on push is a sensor; blocking a *deploy* is admission control — wire the finding to a policy that fails closed, or the scan is merely informative. Second, a scan reflects the vulnerability databases *at scan time*; images must be re-scanned as new CVEs land, which is why registries support scheduled rescans and why the scanning lives at the registry rather than only in the build.

Policy: allowed-registries, signing, and quarantine

Policy is where the sensors become enforcement, and it lives at **admission** (Book 6, Chapters 5-6 — Image Signing and Verification in Kubernetes; Admission Control and Policy Engines). Three policies matter here:

- **Allowed-registries.** Reject any workload whose images do not come from an approved registry (in practice: your internal registry only). This single policy neutralizes malicious public images and image dependency confusion at the cluster boundary — a reference to `docker.io/someuser/...` never schedules.
- **Signing policy.** Require a valid signature from a trusted identity, by digest, before an image runs — the enforcement half of the previous section.
- **Quarantine on push.** Some registries (Harbor's project policies, for instance) can hold a newly-pushed image in a state where it cannot be pulled until it has been scanned and/or signed — failing closed at *ingest* rather than at *deploy*, so a bad image never becomes pullable in the first place.

A minimal Kyverno policy expressing "only from our registry, and signed" looks like:

```

apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: registry-and-signing
spec:
  validationFailureAction: Enforce # fail closed
  rules:
    - name: only-internal-registry
      match:
        any:
          - resources:
              kinds: [Pod]
      validate:
        message: "images must come from registry.internal"
        pattern:
          spec:
            containers:
              - image: "registry.internal/*"
    - name: require-signature
      match:
        any:
          - resources:
              kinds: [Pod]
      verifyImages:
        - imageReferences:
            - "registry.internal/*"
          attestors:
            - entries:
                - keyless:
                    subject: "https://github.com/acme/*"
                    issuer: "https://
token.actions.githubusercontent.com"

```

The details are Chapter 6; the point here is that the registry supplies the artifacts (image, signature via referrers) and the admission engine supplies the *no*.

Immutability, retention, and garbage collection

Immutable tags — supported by ECR (per-repository tag immutability), Harbor (project-level), and others — make the registry *refuse* a push that would overwrite an existing tag. This directly defeats the silent-repush half of the tag-mutability attack: `:v1.2` can be written once and never moved, so "the image at `:v1.2`" means one fixed thing forever. Combine with consumer-side digest pinning: immutability protects the tag, digest pinning protects the consumer, and together they close the tag-swap vector from both ends. **Retention and garbage collection** policies keep

the store bounded (delete untagged manifests, expire old tags), but note the interaction with digests — GC that reclaims a blob no longer referenced by any manifest can break a deploy that pinned a now-orphaned digest, so retention policy and long-lived digest pins must be reconciled (keep-N-versions, protect signed/attested digests).

The internal pull-through registry as chokepoint

The single highest-leverage control — the one that ties the others together — is an **internal registry that proxies the public ones** and that your builders and clusters are configured to use *exclusively*. This is Book 2, Chapter 8's internal-proxy argument, instantiated for images. It serves four purposes at once:

1. **Availability.** It caches upstream images, so a Docker Hub rate-limit event or outage does not break your deploys (the motivating pain point from the landscape section). Your fleet pulls from infrastructure you control.
2. **A scan/policy chokepoint.** Every image entering your world passes through one place, so you scan and apply policy *once* and protect *all* consumers — the same "one control, whole fleet" property that makes the internal proxy worth running.
3. **Provenance re-anchoring.** The mirror can verify upstream signatures on ingest and re-sign vetted images under *your* trust root, so downstream verification checks *your* identity, not a public one.
4. **Egress control.** Configure builds and clusters to reach *only* the internal registry, and block egress to public registries at the network layer (Book 4, Chapter 8 — Network Egress Control for Build and Runtime). Now "pull from a random public image" is not merely discouraged by policy, it is *impossible* by network — the strongest form of allowed-registries enforcement.

```

flowchart LR
    subgraph public["Public registries (untrusted)"]
        DH["Docker Hub"]
        GH["ghcr.io"]
        OTH["quay.io / others"]
    end
    end
    subgraph internal["Internal registry (tier-0, controlled)"]
        PROXY["Pull-through proxy"]
    end
    cache + scan + verify + policy + re-sign
    end
    subgraph fleet["Fleet"]
        B["Builders / CI"]
        K["Clusters / kubelets"]
    end
    end
    DH --> PROXY
    GH --> PROXY
    OTH --> PROXY
    PROXY --> B
    PROXY --> K
    B -. egress blocked to public .-> DH
    K -. egress blocked to public .-> DH

```

Air-gapped registries

The limit case of the internal registry is the **air-gapped** one: an environment with no egress to public registries at all, where images are imported deliberately (often as OCI layouts on removable media or across a one-way transfer) and served from a fully self-contained internal registry. This is standard in regulated and high-assurance settings, and it turns the allowed-registries policy into a physical property. The operational cost is real — you own the entire ingest, vetting, scanning, signing, and update lifecycle for every image and every base layer — but the trust story is the cleanest possible: nothing runs that you did not deliberately import and vet. Zot and Harbor both target this use case (OCI-layout import/export, offline scanner database updates), and the reference implementation can serve a curated store.

Distributed-systems lens

Zoom out to the fleet, and the registry's true nature is clear: it is **tier-0 infrastructure**, in exactly the sense that a service whose failure or compromise takes down or poisons everything else is tier-0. Three properties follow, and they should shape how you run it.

Every deploy depends on it, so its availability is your availability. A registry that is down does not degrade gracefully — new pods cannot start, scaled-up nodes cannot pull, a node reboot cannot recover its workloads. The Docker Hub rate-limit episodes that broke deploys across many organizations were not exotic; they were the predictable consequence of coupling a fleet's availability to a shared, throttled, third-party endpoint. The mitigation is the internal pull-through registry, run **highly available** (replicated, backed by durable object storage, multi-AZ), so that the thing every deploy depends on is infrastructure you operate to your own SLO rather than someone else's free tier.

Its compromise is fleet-wide, so its integrity controls must be fleet-wide. A poisoned registry, or a single stolen push credential, is a delivery channel to *every service that pulls*. You cannot make that blast radius small by hardening individual services; you make it small by making the two controls that survive registry compromise *universal*: **deploy by digest** everywhere (no service deploys a mutable tag), and **verify signatures at admission** for every cluster. One admission policy, applied fleet-wide, is worth more than any amount of per-service diligence — it is the same leverage as the internal proxy, on the enforcement side.

The internal registry is *the* image chokepoint — treat it like the package proxy and the build platform. This is the unifying idea of the cloud-native supply chain and the mirror of Book 2, Chapter 8. Because every image passes through one controlled endpoint on ingest and on serve, that endpoint is where you concentrate your controls: **scan** on ingest, **verify** (or re-anchor) signatures, apply **policy**, and **audit** every push and pull. One control point, correctly operated, protects all services — and the audit log of who pushed what and who pulled what is your forensic record when something goes wrong. Correspondingly, the registry's **push credentials and access control are among the highest-value secrets in the organization**: a push-credential leak is not a single-service incident, it is a fleet-wide image-poisoning capability, so those credentials deserve the short-lived, scoped, workload-identity treatment the rest of this book argues for, and the registry's admin plane deserves the same scrutiny as the CI control plane (Book 4) and the build platform (Book 4, Chapter 5).

Put the three together and the operating posture writes itself: run the internal registry as **tier-0, HA, audited** infrastructure; **deploy by digest** across the whole fleet; **verify signatures and enforce allowed-**

registries at admission on every cluster; and pull **only** from the internal chokepoint, with egress to public registries blocked. The registry stops being a place images happen to sit and becomes what it should be: the controlled front door through which every image the fleet runs must pass, once, under policy.

Key takeaways

- A registry is a **content-addressable blob store** with a mutable **tag** layer and an access-control front door, speaking one protocol — the **OCI Distribution Specification**. Pull walks manifest → config → layers by digest; push uploads blobs first, manifest last; content-addressing gives free dedup and intrinsic integrity.
- The **referrers API** (OCI Distribution v1.1) makes signatures, SBOMs, and attestations discoverable as artifacts that refer to an image *by digest* — the basis for verifying an image at admission from its digest alone.
- A plain pull proves only **bytes-match-digest and TLS-in-transit**. It authenticates *nothing* about the publisher. Integrity relative to a name you chose is not authenticity relative to a publisher you chose; only a **verified signature** closes that gap.
- The threat model reduces to abusing that gap: **registry compromise, tag-mutability replacement** (repush over `:latest / :v1.2` — the most common and most underrated), **malicious public images** (typosquat/fake-official/backdoor — cryptomining images are a persistent Docker Hub problem), **push-credential theft, misconfigured/anonymous registries, cache/blob poisoning**, and the **image analogue of dependency confusion**.
- The controls that **survive a registry compromise or a stolen push credential** are **deploy-by-digest** and **verify-signatures-at-admission**; the registry's own integrity check is not one of them. Sign by digest and *verify* by digest, or the signature defends nothing.
- Wrap all public registries in an **internal pull-through registry** you control: availability against rate limits/outages, a single scan/verify/policy/audit chokepoint, provenance re-anchoring, and — with egress control — the strongest form of allowed-registries enforcement.

- Treat the registry as **tier-0 fleet infrastructure**: HA, audited, deploy-by-digest fleet-wide, allowed-registries + signing policy at admission, push credentials short-lived and scoped. A compromised or unavailable registry is a whole-fleet event.

Further reading

- **OCI Distribution Specification v1.1** — the HTTP API this chapter describes (pull, push, referrers). <https://github.com/opencontainers/distribution-spec>
- **OCI Image Specification** — manifests, image index, config, and the digest/media-type model the distribution spec references (see also Book 6, Chapter 1). <https://github.com/opencontainers/image-spec>
- **distribution/distribution** — the reference registry implementation (formerly "Docker Registry"). <https://github.com/distribution/distribution>
- **Harbor** — the CNCF registry with built-in scanning (Trivy), signature verification, and project policy/quarantine. <https://goharbor.io/docs/>
- **Amazon ECR** — private registry, IAM authZ, tag immutability, pull-through cache, and image scanning. <https://docs.aws.amazon.com/AmazonECR/latest/userguide/>
- **Google Artifact Registry** and **Azure Container Registry** — cloud-managed registries with on-push scanning and cloud RBAC. <https://cloud.google.com/artifact-registry/docs> · <https://learn.microsoft.com/azure/container-registry/>
- **Docker Official Images** and Docker Hub rate limits — the curation program, the `library/` namespace, and the pull-limit policy that motivates internal mirrors. https://docs.docker.com/docker-hub/official_images/
- **Sigstore / cosign** — signing and verifying OCI images and storing signatures as referrers (Book 5, Chapter 3). <https://docs.sigstore.dev/>
- **Kyverno** and **OPA/Gatekeeper** — admission policy engines for allowed-registries and image-signature verification (Book 6, Chapters 5–6). <https://kyverno.io/policies/> · <https://open-policy-agent.github.io/gatekeeper/>
- Cross-references within this suite: Book 2, Chapter 1 (package registry trust models), Book 2, Chapter 3 (dependency confusion and

typosquatting), Book 2, Chapter 8 (internal registries and mirrors), Book 4, Chapter 6 (CI credentials and OIDC), Book 5, Chapters 3–4 (cosign and keyless signing), and Book 6, Chapters 1, 4, 5, and 6.

Chapter 3 — Base Image Strategy: Minimal, Distroless, Hardened

What this chapter covers. Every image you build starts with a `FROM` line, and that one line decides most of what ends up in the running container. Chapter 1 mapped the image attack surface and observed, almost in passing, that the base OS layers dominate an image by size, by component count, and by CVE count. This chapter takes that observation seriously and makes it the subject. The base image is the largest *inherited* dependency in a container: you did not write it, you may not read it, and yet it ships in every process on every node — the container analogue of a transitive dependency you never chose (Book 2 — the dependency supply chain). Choosing and managing base images well is, after signing, the single highest-leverage supply-chain decision you make for a containerized fleet, because one base choice multiplies across every service that builds on it. We build the argument from the ground up: why the base dominates risk; the concrete spectrum of base images from full distros through slim, Alpine, distroless, `scratch`/static, and hardened curated images like Chainguard's Wolfi; the CVE-reduction argument and its honest limits; the lifecycle discipline — freshness, digest-pinning, provenance — that keeps a minimal base from silently rotting; and the fleet-scale answer, a **golden base image program** that lets a platform team patch once and rebuild everything.

Learning goals — after this chapter you should be able to:

- Explain **why the base image dominates** an app image's size, component count, and CVE count, and why that makes it a shared, load-bearing dependency across a fleet.
- Place the common base choices on a **spectrum** from full distro to `scratch`, and articulate the size / attack-surface / CVE-count / debuggability trade-off each one makes.
- Describe precisely **what distroless is** (it is not "no OS"), what Alpine's **musl** trade-offs actually are, and what **Chainguard**

Images / Wolfi do differently (glibc, continuous rebuild, signed, SBOM'd).

- Make the **CVE-reduction argument correctly** — the operational win is real and large, but fewer findings is not automatically "more secure" (Book 2, Chapter 7 — reachability).
- Run the **base-image lifecycle**: pin by **digest**, then **automate digest bumps** (Renovate / Dependabot for Docker) so pinning does not mean staleness, and verify base **provenance**.
- Design a **golden base image program** — curated, signed, SBOM'd, continuously-rebuilt internal bases enforced by admission policy — as the fleet-scale control.

Why the base image dominates risk

Open a shell in a typical Ubuntu-based application image and count what is there. You will find the distribution's package database and its several hundred installed packages: `glibc`, `coreutils`, `bash`, `dash`, `apt` and `dpkg`, `perl` as a dependency of something, `libssl`, `zlib`, a CA certificate bundle, `login/passwd` and the shadow suite, `tar`, `gzip`, `sed`, `grep`, maybe `python3` pulled in transitively. Then, near the top, a thin sliver: your application binary and its runtime dependencies. On a Java or Node image the runtime (a full JDK, a Node distribution) is itself large, but it too sits on the distro. The proportions are lopsided. Of a few hundred megabytes, your code is often single-digit megabytes; the rest is inherited.

This has three consequences that run through the whole chapter.

Most reported CVEs are not in your code. When a scanner (Chapter 4) reports on that image, the overwhelming majority of findings are in base OS packages — `openssl`, `glibc`, `bash`, the distro's `libxml2`, a `zlib` that has not been rebuilt since the base was cut. These are packages you did not choose individually and mostly cannot patch individually; they arrive as a set with the base. A scan report on a full-distro image is, first and foremost, a report card on the base image. Change the base and the report changes wholesale, before you touch a line of application code.

Attack surface is what is in the image. Every shell, every package manager, every interpreter, every `setuid` binary, every library in the image is two things at once: a potential vulnerability to patch, and a potential

tool for an attacker who lands inside the container. This second point is the one operations teams underweight. A container compromise — an RCE in your app, a deserialization bug, an SSRF that becomes command execution — does not end at the process boundary. The attacker now wants to **live off the land**: read secrets from the environment and mounted files, resolve and reach internal services, pull a second-stage payload, escalate. A `bash`, a `curl`, an `apt-get`, a `python3`, a writable package manager, an `nc` from busybox — these are the attacker's toolkit, pre-installed, at their disposal. Remove them and the same code-execution bug lands the attacker in a near-empty room. This is the deepest argument for minimalism, and it is independent of the CVE count: even a shell with zero known CVEs is attacker infrastructure.

The base is a shared, inherited dependency. Chapter 1 described layer sharing — many images built `FROM` the same base share the identical base layers by digest, stored once and pulled once. That efficiency is also a coupling. One base choice is not a per-service decision; it propagates to every service that builds on it. A vulnerability in that base is simultaneously present in every one of those services. This is the concentration property that Book 1, Chapter 9 (Distributed Systems Lens) frames as both leverage and risk, and it is why the back half of this chapter is about managing the base as *fleet* infrastructure rather than a per-repo `FROM` line.

```
flowchart TD
    subgraph IMG["A typical app image, by content"]
        direction TB
        APP["Your app code + direct deps  
~single-digit MB  
few, known, patchable CVEs"]
        RT["Language runtime / JDK / libs  
tens of MB"]
        BASE["Base OS distro  
shell, package manager, coreutils,  
libc, openssl, CA bundle, perl, ...  
hundreds of MB, hundreds of packages"]
    end
    BASE --> SURF["Attack surface:  
shells, package managers,  
interpreters, setuid binaries"]
    BASE --> CVES["Most scanner findings  
live here"]
    APP --> OWN["The part you actually wrote  
and can fix directly"]
```

The strategic conclusion writes itself: if the base dominates size, surface, and CVEs, then *shrinking and controlling the base* is where the leverage is. The rest of the chapter is how.

The spectrum of base images

There is no single "secure base image." There is a spectrum, and every point on it trades a different mix of size, attack surface, CVE count, developer convenience, and debuggability. The engineering skill is placing each workload on the spectrum deliberately rather than defaulting to whatever the language's `Dockerfile` tutorial used. We walk it from heaviest to lightest, then treat the hardened/curated category, which cuts across the whole range.

```
flowchart LR
    A["Full distro  
ubuntu, debian,  
rockylinux, fedora"] --> B["Slim  
debian-slim,  
-slim tags"]
    B --> C["Alpine  
musl + busybox"]
    C --> D["Distroless  
libc + certs,  
no shell / no pkg mgr"]
    D --> E["scratch + static binary  
your binary + maybe certs"]
    A -. "curated / rebuilt  
alternative at any point" .-> F["Hardened  
Chainguard/Wolfi,  
UBI-minimal"]
    F --> D

    A --- A1["biggest size · biggest surface · most CVEs · easiest to  
debug"]
    E --- E1["smallest size · smallest surface · fewest CVEs · hardest to  
debug"]
```

The axes move together but not identically. Size and attack surface track each other closely. CVE count tracks package count, which tracks size. Debuggability moves the *opposite* way — the things that make an image small (no shell, no package manager) are exactly the things that make it hard to poke at when something breaks in production. Keep that inversion in mind; it is the tax you pay for minimalism, and later sections show how ephemeral debug containers pay it down.

Full distribution images

`ubuntu`, `debian`, `rockylinux` and `centos` derivatives, `fedora`: a complete general-purpose Linux userland in a container. Everything works, every tutorial assumes them, every tool you might `apt-get install` at debug time is one command away. That convenience is the whole value proposition, and it is a real one — for **development images**, for **CI build stages**, and for runtimes with genuinely complex or dynamically-discovered native dependencies, a full distro is the pragmatic choice.

The cost is everything the previous section listed, at maximum. A `debian:bookworm` or `ubuntu:24.04` base carries the full package set, the full shell-and-package-manager toolkit for an attacker, and the largest CVE surface of any option. It is also the base most likely to be *stale*: because it "just works," teams pin it to a tag and forget it, and it accretes unpatched CVEs between rebuilds. A full distro is a defensible **build-stage** base and a poor **runtime** base for most services. The multi-stage pattern later in the chapter lets you have the first without paying for the second.

Slim variants

`debian:bookworm-slim`, the `-slim` tags generally: the same distribution with the documentation, locales, and optional tooling stripped — a smaller footprint, fewer packages, meaningfully fewer CVEs than the full tag, but still a distro with a shell (`dash` or `bash`), still `dpkg` / `apt`, still `glibc` and the C toolchain's runtime. Slim is the sensible default when you have decided you need a distro at runtime — you get most of the convenience at a fraction of the surface. It is a strict improvement over the full tag for production runtimes and rarely the wrong first step. It is not, however, the end of the road: it still hands an attacker a shell and a package manager.

Alpine: small, musl, and honest trade-offs

Alpine Linux is the reflexive answer to "make the image smaller." A base `alpine` image is a few megabytes, because it replaces the GNU userland with **BusyBox** (a single multi-call binary that provides `sh`, `ls`, `wget`, and dozens of other applets) and the GNU C library with **musl libc**, a small, cleanly-implemented alternative. It uses `apk` as its package

manager. For many services Alpine is a genuinely good choice and a large improvement over a full distro.

But "just use Alpine" is folk wisdom that hides real trade-offs, and a senior engineer should know them before defaulting to it:

- **musl is not glibc, and the difference is observable.** The two libraries implement the same C standard but differ in behavior at the edges. The historically painful one is **DNS resolution**: musl's resolver has, at various points, diverged from glibc's in how it handles multiple `nameserver` entries, `search` domains, `ndots`, and large responses — behavior that matters enormously in Kubernetes, where service discovery leans hard on the resolver and `ndots:5` is the default. Teams have chased intermittent DNS failures for days that vanished on a glibc base. musl has closed much of the gap over time, but the class of problem is real.
- **glibc-only binaries do not run.** Anything shipped as a dynamically-linked glibc binary — many vendor tools, some proprietary agents, certain database drivers, Nvidia userland — will not run on musl without a compatibility shim, and shims are their own source of subtle breakage.
- **Python and other compiled wheels.** The Python ecosystem's binary wheels (`manylinux`) are built against glibc. On Alpine, `pip install` frequently cannot use a prebuilt wheel and falls back to **compiling from source**, which means dragging a full build toolchain into your image (bloating it, ironically) or fighting long, fragile builds. `musllinux` wheels exist but coverage is thinner than `manylinux`.
- **Performance and defaults.** musl's default per-thread stack size is smaller than glibc's, which has surprised threaded applications into stack-overflow crashes; its `malloc` and some string routines have different performance characteristics under specific workloads. These are edge cases, not everyday problems, but they are the kind of edge case that surfaces at scale and is miserable to diagnose.

None of this makes Alpine wrong. It makes Alpine a *deliberate* choice: excellent for Go and Rust static-ish builds, for simple services, for anything where you have validated musl behavior; risky as a thoughtless default for glibc-dependent stacks, especially Python-with-native-deps and DNS-sensitive services. And note the deeper point: Alpine is still a

distro with a shell and a package manager. It is small, but it is not distroless. It shrinks the CVE surface; it does not remove the attacker's toolkit.

Distroless: your app, libc, and almost nothing else

Distroless is the category most misunderstood, so be precise. Google's `distroless` images (published under `gcr.io/distroless`) are **not** "no operating system." They contain exactly the runtime dependencies an application needs and nothing else: **glibc**, the CA certificate bundle, `tzdata` for timezone handling, `/etc/passwd` and `/etc/nsswitch.conf` so a non-root user and NSS resolution work, and — for the language variants — the language runtime. What they deliberately *omit* is the entire general-purpose userland: **no shell** (`sh`, `bash`), **no package manager** (`apt`, `apk`), **no coreutils**, no `curl`, no `wget`, no interpreters you did not ask for.

That omission is the point. The distroless attack-surface reduction is not primarily about CVE counts (though those drop sharply); it is that **an attacker who achieves code execution lands in a container with no tools to pivot with**. There is no shell to spawn, no package manager to fetch a second stage, no `curl` to exfiltrate. They must bring everything themselves, over whatever channel your app's own code gives them, which is a dramatically harder position than "you now have a root shell with `apt`."

Distroless comes in language-specific flavors, and choosing the right one matters:

- `gcr.io/distroless/static` — for **statically-linked** binaries (typically Go with `CGO_ENABLED=0`, or static Rust). Contains CA certs, `tzdata`, `/etc/passwd`, and a nonroot user; no `libc`, because a static binary carries its own.
- `gcr.io/distroless/base` — adds **glibc** and `libssl`, for dynamically-linked binaries (Go with `cgo`, C/C++ that links against system `libc`).
- `gcr.io/distroless/cc` — adds the C/C++ runtime (`libgcc`, `libstdc++`) for compiled languages that need it.
- `gcr.io/distroless/java`, `.../python3`, `.../nodejs` — bundle the respective runtime. (Note that Google's Python and Node distroless images have historically carried caveats about version pinning and

support; verify current status before standardizing on them, and consider Chainguard or Wolfi's language images as alternatives.)

The cost of distroless is **debuggability**, and it is the honest downside. With no shell, you cannot `kubectl exec -it pod -- /bin/sh` your way into a running container to look around — there is nothing to `exec`. The modern answer is the **ephemeral debug container**: `kubectl debug` attaches a *separate*, temporary container with a full toolset into the target pod's namespaces, so you get your shell and `curl` and `ps` sharing the process and network namespace of the distroless container without those tools ever living in the production image.

```
# Attach a throwaway debug container that shares the target's
namespaces.
$ kubectl debug -it payments-7c9f-abcde \
  --image=cgr.dev/chainguard/busybox:latest \
  --target=payments --profile=general
# You now have a shell that can see the distroless container's
processes,
# filesystem (/proc/1/root), and network — without a shell in the image
itself.
```

Google also publishes `:debug` tags (for example `gcr.io/distroless/static:debug`) that include a BusyBox shell, intended for local troubleshooting — but the discipline is to run the **non-debug** image in production and reach for `kubectl debug` when you need to look inside, so the shell is never resident in the deployed artifact.

scratch and static binaries: the empty base

The floor of the spectrum is `FROM scratch` — the reserved, **completely empty** base image: zero bytes, no files at all. On top of it you place a **statically-linked binary** and, if the app makes outbound TLS calls, a CA certificate bundle, plus perhaps `/etc/passwd` for a non-root UID and `tzdata` if it does timezone math. The result is the smallest possible image: your binary and a handful of data files, and *nothing else* — no `libc`, therefore no `libc` CVEs; no shell; no userland; nothing for an attacker to use and nearly nothing for a scanner to find in the base.

This is where **Go and Rust shine**, because both can produce fully static binaries. A Go build with `CGO_ENABLED=0` links no `libc` and needs no dynamic loader. Rust can target `x86_64-unknown-linux-musl` for a static

binary. A multi-stage `Dockerfile` compiles in a full builder stage, then copies the single binary into `scratch`:

```
# ---- build stage: full toolchain, thrown away ----
FROM golang:1.23 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
# Static build: no cgo, no dynamic linking.
RUN CGO_ENABLED=0 GOOS=linux go build -trimpath -ldflags="-s -w" -o /
app ./cmd/server

# ---- runtime stage: an empty base ----
FROM scratch
# CA roots for outbound TLS, copied from a trusted stage.
COPY --from=build /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/
# A non-root identity (UID 65532) so the process need not run as root.
COPY --from=build /etc/passwd /etc/passwd
COPY --from=build /app /app
USER 65532
ENTRYPOINT ["/app"]
```

The trade-offs are real and worth stating. Static linking means your binary **bundles everything**, so a vulnerability in a statically-linked library is not "the base's `openssl` CVE" that a base rebuild fixes — it is *your* dependency, which you fix by rebuilding *your* app (this is where SBOMs of the binary's own dependencies, Book 3, matter, because the scanner can no longer read packages off a distro database). You give up NSS-based name resolution niceties, you must remember the CA bundle or every HTTPS call fails with an opaque error, and debugging is `kubectldbg` or nothing. For a Go or Rust network service, though, `scratch` (or the near-identical `distroless/static`, which throws in the certs and nonroot user for you) is frequently the correct production base and the cleanest supply-chain story you can tell: the image is your code plus data.

Hardened and curated images

Cutting across the whole spectrum is a category defined not by *how much* it contains but by *how it is produced and attested*: continuously-rebuilt, minimal, signed, SBOM-carrying base images from a curator who treats freshness and provenance as the product.

Chainguard Images, built on the **Wolfi** undistribution, are the standard-bearer of this modern trend, and precision matters because they are easy

to conflate with Alpine. Wolfi is a minimal, **glibc-based** Linux distribution — this is the key difference from Alpine, and it means Wolfi avoids the musl compatibility class of problems entirely while keeping images tiny. It uses the `apk` package format and tooling (borrowed from Alpine's ecosystem) but ships its own glibc-based packages. It is a **rolling** distribution with **no traditional release**, built to be **continuously rebuilt from source** so that images are always fresh — the freshness problem the next section describes is attacked at the source rather than left to downstream rebuild schedules. Chainguard's runtime images are **distroless by default** (no shell, no package manager), with `-dev` variants that add a shell, `apk`, and BusyBox for building and debugging. Critically, every image ships with a **cosign signature** and a build-time **SBOM** (Book 3) as standard artifacts, and provenance is part of the product. The pitch is a base that is minimal like distroless, glibc-compatible unlike Alpine, continuously patched unlike a pinned distro, and signed-and-SBOM'd unlike almost everything else — a "secure base" as a managed dependency.

Red Hat Universal Base Image (UBI) is the curated option from the RHEL world: a RHEL-derived base that Red Hat licenses for **free redistribution** (you can build on and ship UBI without a RHEL subscription), so it brings RHEL's package hygiene and long support lifecycle to anyone. `ubi9` is the full base; `ubi9-minimal` is the small variant that swaps `dnf` for the lightweight `microdnf` and strips the footprint substantially; `ubi9-micro` goes further still. UBI is the natural base for RHEL-aligned shops and for anything that wants an enterprise support lifecycle, and it is a common requirement in regulated environments.

Beyond these, **Docker Official Images** (the curated `library/` namespace on Docker Hub) and **Docker Verified Publisher** images are a *baseline* trust signal — maintained, documented, reasonably fresh — but they are still ordinary distro-based images with the surface that implies; "official" is a curation claim, not a minimalism or hardening claim. **Bitnami** publishes a large catalog of hardened, regularly-updated application and language images that many teams standardize on. The unifying theme of this category is that *someone else runs the freshness-and-provenance discipline for you* — which is exactly the discipline the next two sections argue you cannot skip.

The spectrum as a decision table

Base type	Typical size	Attack surface	Typical CVE count	Debuggability	When to use
Full distro (ubuntu , debian)	70-300 MB+	Highest: shell, pkg mgr, full userland	Highest	Easiest: full toolset resident	Dev images, CI build stages, complex/dynamic native runtime deps
Slim (debian-slim)	30-80 MB	High: still shell + pkg mgr	High, but well below full	Easy	Default when a distro is required at runtime
Alpine	~5-15 MB	Medium: BusyBox shell + apk	Low-medium	Easy: BusyBox shell present	Go/Rust, simple services, validated-musl stacks; avoid for glibc/DNS-sensitive or native-Python workloads
Distroless (gcr.io/distroless)	~2-25 MB (varies by runtime)	Very low: no shell, no pkg mgr	Very low	Hard: kubectl debug / :debug tag	Production runtime for most services; language-matched variant

Base type	Typical size	Attack surface	Typical CVE count	Debuggability	When to use
scratch + static	~app size (single-digit MB)	Minimal: only your binary	Minimal (no base pkgs)	Hardest: <code>kubectl debug</code> only	Static Go/ Rust network services
Hardened/ curated (Chainguard/ Wolfi, <code>ubi-minimal</code>)	~2-40 MB	Very low (distroless-style) to low	Very low / near-zero, continuously rebuilt	Hard (<code>-dev</code> / <code>-minimal</code> variants ease it)	Fleet-wide secure base; signed + SBOM'd + fresh out of the box; UBI for RHEL/ regulated

Read the table as guidance, not law. The size and CVE figures move constantly and depend on the language runtime you add; treat them as orders of magnitude. The stable truth is the *ordering* and the *shape* of the trade-off: as you move down, size / surface / CVE count fall and debugging difficulty rises, and the hardened category lets you buy a low point on the surface axis while someone else carries the freshness-and-provenance cost.

The CVE-reduction argument, made honestly

Switching a fleet from full-distro bases to distroless or Chainguard bases collapses scanner findings — often by an order of magnitude, sometimes to near zero on the base layers. That number is seductive and it is worth having a precise, honest theory of *what it buys*, because it is easy to oversell and the overselling invites a justified backlash.

What is genuinely won. Two things, both large.

First, the **operational win**, which is the biggest and least disputable. Fewer packages means fewer CVEs to triage, and CVE triage at fleet scale is a dominant, demoralizing cost — the alert fatigue that Book 2, Chapters 6-7 diagnose. Every base CVE a scanner reports is a ticket someone must assess: is it reachable, is it exploitable in our

configuration, is there a fix, do we rebuild now or wait. Cut the base from three hundred packages to twenty and you have cut that triage load proportionally. Fewer findings also means **lower MTTR to patch** what remains: when a real, reachable base CVE lands, an image with twenty packages is faster to rebuild, re-test, and roll than one with three hundred, and the signal is not buried under noise. This alone justifies minimal bases in most shops.

Second, the **real attack-surface reduction**, which is independent of any CVE count. Removing the shell and the package manager removes *attacker capability*, as argued above. That win is not measured by the scanner at all — a `bash` with zero CVEs is still the thing that turns an RCE into a foothold — and it is arguably the more important of the two. Less in the image is less to exploit, less to patch, and less to go wrong, full stop.

Where the naïve version is wrong. The seductive but sloppy claim is "fewer CVEs equals more secure," as if the security of an image were a decreasing function of its scanner finding count. It is not, for a reason Book 2, Chapter 7 (reachability) develops in full: **most CVEs in a base image are in packages your application never calls**. A CVE in `libxml2` in a base whose only job is to host a Go binary that never parses XML is not exploitable *in that image*, regardless of the package's CVSS score. Removing that package removes a scanner finding but changes your actual exposure by roughly nothing, because the finding was never reachable. If you judged the switch to distroless purely by the drop in finding count, you would be measuring partly noise — you would take credit for "fixing" vulnerabilities that were never a threat.

So hold both truths at once. The finding-count drop **overstates** the pure vulnerability-exposure improvement, because much of what disappears was unreachable anyway. But the switch is still a large real win, because (a) the operational relief — less triage, faster MTTR, less noise hiding real signal — is genuine and enormous, and (b) the attack-surface reduction — no shell to pivot with — is genuine and does not show up in the CVE count at all. The correct executive summary is not "we eliminated 95% of our vulnerabilities." It is: "we eliminated 95% of the *triage load* and removed the attacker's on-box toolkit; the fraction of those findings that were ever exploitable was small, but not paying to chase the rest is exactly the point." That is a defensible claim. The inflated one invites a security team to notice, correctly, that you did not

actually change your exploitable surface as much as the dashboard suggests — and to distrust the whole program.

Base image lifecycle: freshness, pinning, provenance

A minimal base is not a set-and-forget artifact. Three lifecycle disciplines separate a base strategy that works from one that quietly decays, and they interact in a way that is easy to get half-right.

Freshness: bases go stale

An image is a **point-in-time snapshot** of packages. The moment it is built, it begins to age: upstream projects publish patches, distributions push updated packages, and the frozen snapshot in your base accumulates known-but-unpatched CVEs. A base image built six months ago and never rebuilt is, by definition, missing six months of security updates, no matter how minimal it was on day one. Minimalism reduces the *rate* of staleness (fewer packages, fewer things to go stale) but does not stop the clock.

The only cure is **rebuilding**: producing a fresh image that pulls current upstream packages, on a schedule fast enough that no service runs on a base older than your tolerance. There are three common models:

- **Automated periodic rebuilds** of your own images — a scheduled CI job that rebuilds nightly or weekly against the latest base, the image analogue of the dependency-update automation in Book 2, Chapter 9.
- **Curator continuous rebuild** — this is precisely Chainguard/Wolfi's model: the *base provider* rebuilds continuously from source so freshness is upstream of you; you inherit it by re-pulling.
- **Pin plus scheduled bump** — pin the base for reproducibility (next), and run automation that proposes a bump on a schedule. This is the pattern most teams should run, and it resolves the tension the next subsection sets up.

The distributed-systems framing is to make freshness a **fleet SLO**: *no service runs on a base image older than N days*. That single objective, measured against inventory, converts "we should rebuild sometime" into an operable, alertable target — and it is only achievable if base usage is inventoried, which the golden-image section makes possible.

Pinning: by digest, not by tag

Chapter 1 established that **tags are mutable pointers and digests are immutable content addresses**. `FROM node:20` resolves to *whatever* `node:20` points at when the build runs — and that moves. Two builds of the identical `Dockerfile` a week apart can produce different images because the base tag advanced underneath them. That is fatal to **reproducibility** (Book 4, Chapter 2 — reproducible builds): you cannot reason about, reproduce, or trust a build whose inputs silently change. Worse, it is a supply-chain exposure: if an attacker (or a compromised registry account, Chapter 2) republishes the base tag, your next build ingests it with no signal.

The fix is to pin the base by **digest**:

```
# Mutable – the base can change under you, build to build.  
FROM node:20-slim  
  
# Pinned – this exact content, or the build fails. Reproducible and  
# tamper-evident.  
FROM node:20-slim@sha256:9d0e0b1f6f3a...c2a1
```

With a digest pin, `FROM` names a specific set of bytes. The build is reproducible with respect to its base, and a republished tag cannot substitute different content — the digest would not match. Pin your base by digest, and (Chapter 1) deploy your *own* images by digest too, so immutability runs end to end.

But digest pinning creates the very problem freshness warns against: **a pinned digest never advances**, so a naïvely pinned base is a *frozen* base that rots. Pinning without automation is how teams end up running eighteen-month-old bases full of patched-upstream-but-not-here CVEs, having mistaken reproducibility for safety.

Pin-and-automate: the pattern that resolves the tension

The resolution is not to choose between reproducibility and freshness but to **pin by digest and automate the digest bumps** — the pin-and-automate pattern (Book 2, Chapter 9). You keep the digest pin, so every individual build is reproducible and tamper-evident; and you run a bot that watches for new base versions, opens a pull request updating the pinned digest, and lets CI test the bump before it merges. The pin gives

you determinism; the automation gives you freshness; the PR gives you a review-and-test gate.

Renovate and **Dependabot** both support Docker FROM lines as first-class dependencies. Renovate's `docker` and `docker-compose` managers will even maintain the *tag-plus-digest* form — keeping a human-readable tag as a comment while bumping the pinned digest — so you get readable Dockerfile s and immutable builds together:

```
{
  "$schema": "https://docs.renovatebot.com/renovate-schema.json",
  "extends": ["config:recommended"],
  "docker": { "enabled": true },
  "packageRules": [
    {
      "matchDatasources": ["docker"],
      "pinDigests": true,
      "schedule": ["before 6am on monday"]
    }
  ]
}
```

```
# Renovate maintains the digest; the tag comment stays human-readable
and gets bumped too.
FROM cgr.dev/chainguard/static:latest@sha256:3f1c...ab90
```

```
sequenceDiagram
    autonumber
    participant Up as Upstream base
    (registry)
    participant Bot as Renovate / Dependabot
    participant Repo as Your repo (pinned FROM)
    participant CI as CI (build + scan + test)
    participant Reg as Your registry

    Up->>Bot: new base digest published
    Bot->>Repo: open PR: bump FROM ...@sha256 old to new
    Repo->>CI: PR triggers build against new pinned digest
    CI->>CI: build · scan (Ch 4) · run tests
    alt green
        CI-->>Repo: checks pass, merge
        Repo->>Reg: publish rebuilt, freshly-based image (signed, Book 5)
    else regression / new CVE
        CI-->>Bot: fail; hold the bump for a human
    end
end
```

This loop is the mechanical heart of a working base strategy: **reproducible because pinned, fresh because automated, safe because gated**. Skipping either half breaks it — no pin means irreproducible and tamper-prone builds; no automation means frozen, rotting bases.

Provenance: trust the base you inherit

A base image is code you did not write, running in every one of your containers. If it is compromised, everything built on it is compromised — the base is a **fleet-wide blast radius** and a prime target for exactly the registry threats Chapter 2 catalogs (tag replacement, poisoned public images, typosquatting the base name). You verify your *own* images (Book 5); you must extend the same skepticism upstream.

Ask of any base you adopt: **Is it signed?** Can you verify a cosign signature from a publisher you trust before you build on it (Book 5)? Chainguard images ship signatures as standard; you can and should verify them:

```
# Verify a Chainguard image's signature before trusting it as a base.
$ cosign verify \
  --certificate-oidc-issuer=https://
token.actions.githubusercontent.com \
  --certificate-identity-regexp='.*chainguard.*' \
  cgr.dev/chainguard/static@sha256:3f1c...ab90
```

Does it carry an SBOM? (Book 3.) Can you enumerate what is actually in the base, so that when the next Log4Shell-class disclosure lands you can answer "is that component in our base?" from data rather than archaeology? **Is the source trustworthy** — an official curated namespace, a verified publisher, your own internal build — rather than a random Docker Hub account whose `python-fast` tag you pulled because it was small? The strongest posture pulls bases only from sources that are signed, SBOM'd, and pinned by digest, and re-anchors that provenance in an internal registry (Chapter 2) so the whole fleet inherits verified bytes. A base you cannot verify is a base you are trusting on faith, multiplied across every service that builds on it.

Minimizing what you ship: multi-stage and non-root

Two build-time practices convert base strategy into shipped reality. Both were introduced in Chapter 1 and Book 4, Chapter 2; here they are the

mechanism that lets you *use* a heavy base for building while *shipping* a minimal one.

Multi-stage builds let you compile in a full-toolchain stage and copy only the finished artifact into a minimal final stage, so build-time bulk, secrets, and tooling never reach the runtime image:

```
# Build stage: full JDK + Maven, everything the build needs.
FROM eclipse-temurin:21-jdk AS build
WORKDIR /src
COPY . .
RUN ./mvnw -q -DskipTests package

# Runtime stage: distroless Java – JRE, glibc, certs, tzdata; no shell,
no pkg mgr.
FROM gcr.io/distroless/java21-debian12@sha256:...
COPY --from=build /src/target/app.jar /app/app.jar
USER 65532
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

The build stage's `jdk`, its Maven cache, any credentials in the build context — none of it exists in the final image. You paid for a heavy base at build time and shipped a minimal one.

Not running as root (`USER`, Chapter 1) is the other half. Even a minimal image should not run its process as UID 0: a container escape or a mounted-secret read is worse from root. Distroless and Chainguard images ship a `nonroot` user (commonly UID 65532) precisely so you can `USER nonroot` without inventing one. Combined with a minimal base, the running container is a non-root process in an image with no shell — a genuinely hard target.

A golden base image program

Everything so far is a per-image discipline. At fleet scale — many services, many teams, many repos (the distributed-systems reality this suite is written for) — leaving each team to independently pick a base, pin it, automate bumps, and verify provenance produces exactly the sprawl you would expect: dozens of different bases, most unpinned, few verified, patched on no schedule, with no one able to answer "which services are on a vulnerable base?" The fleet-scale answer is a **golden base image program**: the platform team curates a small set of blessed bases and everyone builds on them.

A golden base image is **hardened** (minimal, non-root, distroless-style), **signed** (Book 5), **SBOM'd** (Book 3), **continuously rebuilt** (fresh), and **published from an internal registry** (Chapter 2) that the whole fleet pulls from. The platform team owns its lifecycle; application teams consume it with a single `FROM`. This is the **paved road** pattern (Book 1, Chapter 10; Book 4, Chapter 10) applied to base images: the secure default is the *easy* default, so teams follow it not by mandate but because it is less work than rolling their own.

```
flowchart TD
    subgraph Platform["Platform / security team"]
        UP["Upstream hardened base  
Chainguard/Wolfi, distroless, UBI-minimal"]
        HARD["Harden + configure:  
non-root, CA policy, org certs,  
required agents"]
        ATT["Sign + SBOM + provenance  
(cosign, Book 5; SBOM, Book 3)"]
        REB["Continuous / scheduled rebuild  
freshness SLO"]
        UP --> HARD --> ATT --> REB
    end
    REB --> GOLD["Golden base images  
internal registry (Ch 2)"]
    GOLD --> registry["e.g. registry.corp/base/go, /java, /python"]
    GOLD --> T1["Team A service"]
    GOLD --> T2["Team B service"]
    GOLD --> T3["Team C service"]
    GOLD --> Tn["... rest of fleet"]

    CVE["New base CVE disclosed"] --> REB
    REB -. "patch once" .-> GOLD
    GOLD -. "rebuild all consumers" .-> T1
    GOLD -. " " .-> T2
    GOLD -. " " .-> T3
```

The payoff is **centralized patching**, and it is the whole reason the program exists. When a base CVE lands — the next `openssl` or `glibc` or Log4Shell-class event — the platform team patches the golden base *once*, and every service is remediated by rebuilding against the new golden digest (driven by the same Renovate/Dependabot loop from the previous section, now pointed at the internal base). Contrast the alternative: a hundred teams each discovering the CVE, each finding their own base, each patching independently on their own schedule, some never. Concentration turns a hundred-team scramble into one platform action

plus an automated fleet rebuild. That is Book 1, Chapter 9's "concentration as leverage" made concrete.

Concentration is also **risk**, and the program must respect it. A compromised golden base is a fleet-wide compromise — the very blast radius the provenance section warned about, now centralized by design. This is *why* the golden base must be the most rigorously signed, SBOM'd, and access- controlled artifact in the organization: you have deliberately concentrated trust, so you must concentrate assurance to match. The golden base build pipeline deserves the tier-0 treatment the registry gets in Chapter 2 and the CI control plane gets in Book 4.

Enforcing base policy

A golden base program that is merely *available* becomes a golden base program that half the fleet ignores. Enforcement makes it real, and it rests on three mechanisms this book develops elsewhere:

- **Admission / policy control** (Chapter 6): a cluster policy (Kyverno, OPA/Gatekeeper) that **requires images to derive from an approved base** — implemented as an allowed-registry/repository rule (only `registry.corp/base/*` and images built from it may run), often combined with the signature-verification policy of Book 5, Chapter 5. An image on an unapproved base is refused admission. This is the hard backstop that turns "please use the golden base" into "the cluster will not run anything else."
- **Scan bases at ingest** (Chapter 4): scan the golden bases themselves as they are built and before they are published, so a base never enters the fleet's supply with a known-and-unassessed critical CVE. The base is the highest-leverage thing to scan because its findings are inherited by everything downstream.
- **Inventory base usage** (Book 3, Chapter 5 — SBOMs at scale): maintain a fleet-wide record of *which service runs which base at which digest*, derived from image SBOMs and deploy metadata. This is what makes the freshness SLO measurable ("show every service on a base older than N days") and what makes incident response fast ("show every service whose base contains the vulnerable package"). Without inventory, a golden base program is patching in the dark; with it, the Log4Shell question — *which of our services are affected?* — is a query, not an archaeology project.

Put the three together and the program is self-enforcing and self-observing: policy keeps everyone on approved bases, scanning keeps the approved bases clean at ingest, and inventory tells you at any moment where every base is deployed and how old it is. That is the difference between owning your base supply and hoping about it.

Distributed-systems lens

The base image is the clearest small example of a theme that runs through this entire suite: **a shared, load-bearing dependency, concentrated across the fleet, is simultaneously your greatest point of leverage and your greatest point of risk.**

- **Concentration as leverage.** A golden-image program concentrates base security into one owned artifact: patch the base once, rebuild everything (Book 1, Chapter 9). The same Renovate/Dependabot loop that bumps one repo's `FROM` becomes, pointed at an internal golden base, a fleet-wide remediation mechanism. Nothing else in container supply chain gives you this much reach from a single action.
- **Concentration as risk.** A base compromise is not a service incident; it is a fleet incident, because every image inherits the base's bytes (Chapter 2's registry threats aimed at the base). The mitigation is to match concentrated trust with concentrated assurance: sign the base, SBOM it, pull it only through a verified internal chokepoint, and treat its build pipeline as tier-0.
- **Fleet-wide triage load.** Minimal bases cut CVE-triage volume across every service at once — the operational win of the CVE section, multiplied by the fleet. At one service, distroless saves a few tickets a month; at a thousand services, it is the difference between a security team that drowns in base-package noise (Book 2, Chapters 6–7) and one that can attend to reachable, application-level findings.
- **MTTR as a fleet property.** When a real base CVE lands, mean-time-to-remediate across the fleet is dominated by how fast you can rebuild-and-roll every affected service — which is fastest when bases are minimal, pinned-and-automated, and centralized. Base strategy is, in effect, a lever on fleet-wide patch latency.
- **Freshness as an SLO.** "No service on a base older than N days" is an operable objective only when base usage is inventoried; then it is

measurable and alertable like any other SLO, and the golden-image program is the thing that makes it achievable.

- **Inventory as incident response.** The Log4Shell lesson (Book 3) is that the expensive question in a supply-chain incident is *which of our things are affected?* A base-usage inventory answers the base-layer form of that question as a query. That single capability repays the whole golden-image program the first time a base-package disclosure lands.

The synthesis: the **golden base image + automated rebuild + digest pin** pattern is the highest-ROI container supply-chain control after signing itself. Signing (Book 5) tells you an image is what its publisher says it is; a governed base tells you that what the publisher built on is minimal, fresh, verified, and centrally patchable. Together they convert the base image from the largest, most inherited, least-examined part of your fleet into a controlled, observable, patch-once dependency.

Key takeaways

- The **base image dominates** an app image's size, component count, and CVE count. Most scanner findings are in base OS packages you inherited, not code you wrote — so shrinking and governing the base is the highest-leverage container decision after signing.
- **Attack surface is what is in the image.** Every shell, package manager, and interpreter is both a CVE to patch and a tool for an attacker living off the land inside a compromised container. Removing them is a security win independent of any CVE count.
- The base-image **spectrum** runs full distro → slim → Alpine → distroless → `scratch` / `static`, with hardened/curated images (Chainguard/Wolfi, UBI-minimal) cutting across it. Down the spectrum, size / surface / CVE-count fall and debuggability worsens.
- **Distroless is not "no OS":** it is `libc` + CA certs + `tzdata` + your runtime, with **no shell and no package manager**. Alpine trades `glibc` for **musl** and `BusyBox` — real DNS, native-wheel, and compatibility gotchas make "just use Alpine" a deliberate choice, not a default. **scratch + static** (Go/Rust) is the minimal floor. **Chainguard/Wolfi** is `glibc`-based, continuously rebuilt, distroless-by-default, and signed + SBOM'd out of the box.

- **Fewer CVEs is not automatically more secure** (Book 2, Chapter 7 — reachability): much of what a minimal base removes was never reachable. The honest wins are the **operational** one (far less triage, faster MTTR, less noise) and the **real attack-surface** one (no on-box toolkit) — both large, neither identical to the raw finding-count drop.
- Run the base **lifecycle**: pin by **digest** for reproducibility and tamper-evidence, then **automate digest bumps** (Renovate/Dependabot for Docker) so pinning never means staleness; keep bases **fresh** (rebuild on a schedule or inherit a continuously-rebuilt base); and **verify base provenance** — signed, SBOM'd, from a trusted source — because a poisoned base is a fleet-wide compromise.
- A **golden base image program** — a small set of hardened, signed, SBOM'd, continuously-rebuilt internal bases, enforced by admission policy, scanned at ingest, and tracked in a base-usage inventory — concentrates patching (fix once, rebuild the fleet) and provenance. It is the paved-road, highest-ROI base control at scale, provided you match the concentrated trust with concentrated assurance.

Further reading

- **Google Distroless** — the images, their exact contents, language variants, and the `:debug` tags. <https://github.com/GoogleContainerTools/distroless>
- **Chainguard Images and the Wolfi undistribution** — continuously-rebuilt, glibc-based, minimal, signed + SBOM'd base images; read on both the images and the Wolfi build model. <https://www.chainguard.dev/chainguard-images> · <https://github.com/wolfi-dev>
- **Alpine Linux and musl libc** — the distribution, BusyBox, and the musl functional-differences page that documents glibc divergences. <https://alpinelinux.org/> · <https://wiki.musl-libc.org/functional-differences-from-glibc.html>
- **Red Hat Universal Base Image (UBI)** — the freely-redistributable RHEL base and its `-minimal` / `-micro` variants. <https://catalog.redhat.com/software/base-images>
- **FROM scratch and Go static binaries** — the empty base and building fully static images. <https://docs.docker.com/build/building/base-images/>

- **Docker multi-stage builds** — the mechanism for shipping a minimal final stage (see also Book 4, Chapter 2 and Book 6, Chapter 1). <https://docs.docker.com/build/building/multi-stage/>
- **Renovate — Docker digest pinning and updates** — the pin-and-automate pattern for `FROM` lines (Book 2, Chapter 9). <https://docs.renovatebot.com/docker/>
- **kubect1 debug — ephemeral debug containers** — how to debug shell-less images without shipping a shell. <https://kubernetes.io/docs/tasks/debug/debug-application/debug-running-pod/>
- **Sigstore / cosign** — verifying base-image signatures before you build on them (Book 5, Chapter 3). <https://docs.sigstore.dev/>
- Cross-references within this suite: Book 1, Chapters 9-10 (concentration and paved roads); Book 2, Chapters 6-7 and 9 (alert fatigue, reachability, update automation); Book 3, Chapter 5 (SBOMs at scale and inventory); Book 4, Chapters 2 and 10 (reproducible builds, platform paved roads); Book 5 (signing and verification); and Book 6, Chapters 1, 2, 4, and 6. ``

Chapter 4 — Image Scanning and Vulnerability Management

What this chapter covers. You already know what Software Composition Analysis is (Book 2, Chapter 6 — Software Composition Analysis in Depth): enumerate the components an artifact contains, match them against vulnerability data (Book 2, Chapter 5 — Vulnerability Databases and Identifiers), and produce a list of findings. **Container image scanning is exactly that, applied to an OCI image** — but the container substrate changes the mechanics, the tooling, the error modes, and, above all, the *scale* at which you must operate. An image is not one dependency manifest; it is a stack of layers (Book 6, Chapter 1 — Container Images) containing an entire OS distribution's worth of packages *plus* your application's language dependencies *plus* whatever binaries and secrets leaked in during the build. And you do not have one image; you have hundreds of services, each with many versions, each rebuilt continuously, all sitting in a registry (Book 6, Chapter 2 — Registries) waiting for the next Log4Shell.

This chapter is deliberately *not* a re-teaching of SCA internals — version-range matching, CVE vs. GHSA vs. OSV identifiers, reachability analysis, EPSS and KEV prioritization. Those live in Book 2 and Book 3 and we reference them rather than repeat them. What this chapter adds is everything that is *specific to containers*: how a scanner reconstructs a component inventory across layers, the **distro-backporting nuance** that is the single largest source of false positives in image scanning, where in the pipeline scanning happens (and why it must happen *continuously*, not once), and the operational reality that the dominant remediation for image CVEs is not patching packages one by one — it is **rebasing onto a fresher base image and rebuilding the fleet**.

Learning goals — after this chapter you should be able to:

- Explain the **mechanics** of image scanning: layer walk → OS package DB + language-manifest + binary inventory → SBOM → match against OS-vendor advisories *and* ecosystem advisories → findings.
- Explain **distro backporting** precisely — why a "vulnerable" upstream version string can be a *patched* distro package, why version-matching against upstream data produces false positives, and why scanners must use Debian/Alpine/Red Hat security-tracker data.
- Describe what modern scanners find **beyond CVEs**: embedded secrets, Dockerfile misconfigurations, license issues, and (for some) malware.
- Compare the real tooling honestly: **Trivy, Grype + Syft, Clair, Docker Scout**, cloud-built-in scanning (ECR/GAR/ACR), Harbor's integrated scanning, and commercial platforms — by *what* they scan and *how* they are deployed.
- Explain **SBOM-based scanning** (scan the inventory once, re-match forever) and why decoupling inventory from matching is the key move at fleet scale.
- Place scanning at the four **scan points** — build/CI, registry, admission, runtime — and argue why **continuous re-scan** is non-negotiable.
- Run an **operational vulnerability-management program for images**: manage volume with minimal bases, backport-aware matching, VEX, and prioritization; remediate primarily by **rebase**; gate with warn-then-enforce policy; and aggregate centrally so you can answer "which running images are affected by CVE-X" fleet-wide.

- State the honest **limits**: scanning finds *known* vulns in *known* components — not zero-days, not your logic bugs, not runtime misconfiguration. Necessary, not sufficient.

What image scanning actually does

Scanning an image is two steps that Book 2 already named: **build a component inventory**, then **match that inventory against vulnerability data**. The container-specific difficulty is entirely in the first step and in the *data source* you match against in the second. Everything else is SCA.

Step 1 — inventory the image across all layers

An OCI image is an ordered set of layers, each a tar of filesystem changes, that overlay into a single root filesystem (Book 6, Chapter 1). A scanner does not run the container. It reads the image's blobs from the registry or local store, applies the layers in order to reconstruct the final filesystem (or walks each layer's tar directly), and then does what a package manager and a language toolchain would recognize: it finds the **evidence of installed software** and reads it.

There are three broad classes of evidence, and a good scanner harvests all three:

- **OS packages.** Every mainstream distro records what it installed in an on-disk database that lives *inside the image's layers*:
- Debian/Ubuntu: `/var/lib/dpkg/status` — a flat text file listing every `.deb`, its exact version, and its architecture.
- Alpine: `/lib/apk/db/installed` — the apk database, one stanza per package.
- RHEL/Fedora/CentOS/Amazon Linux: the RPM database (`/var/lib/rpm/`, an SQLite or BerkeleyDB file depending on version) queried the way `rpm -qa` would. The scanner parses these directly. It does **not** shell out to `dpkg / apk / rpm` inside the image — it reads the database format itself, which is why a scanner can inventory an Alpine image on an amd64 CI runner with no Alpine present.
- **Language dependencies.** The same ecosystem-specific evidence SCA uses (Book 2, Chapter 6), now found by walking the reconstructed filesystem: Java `.jar / .war` files (read the embedded

`pom.properties / MANIFEST.MF` for group/artifact/version), Python wheels and `*.dist-info/METADATA`, Node `package.json` and `node_modules`, Ruby gems, Rust and Go. Go is a special and pleasant case: a statically linked Go binary embeds its **module build info** (`go version -m binary`), so the scanner can read the exact module versions out of the compiled ELF with no manifest present at all. Modern scanners extract this from binaries directly.

- **Binaries and "the rest."** Some scanners fingerprint standalone binaries and language runtimes that were dropped in without a package manager (a hand-copied `node`, a `busybox`, a vendored `openssl`), and detect the language interpreter versions themselves.

The output of step 1 *is an SBOM* (Book 3, Chapter 4 — SBOM Generation). This is not an analogy; it is literally the same operation. Syft generates an SBOM; Trivy generates the same inventory internally and can emit it as SPDX or CycloneDX. **Scanning = SBOM generation + vulnerability matching.** Holding that equation in your head is what unlocks the scale strategy later in this chapter.

```

flowchart TD
    subgraph IMG["OCI image (all layers)"]
        L0["Layer 0: base OS  
/var/lib/dpkg/status  
/lib/apk/db/installed  
/var/lib/rpm"]
        L1["Layer 1: runtime  
python, node, jvm"]
        L2["Layer 2: app  
jars, wheels, node_modules,  
go binaries"]
    end
    INV["Component inventory / SBOM  
OS pkgs + lang deps + binaries  
name @ version @ ecosystem"]
    OSADV["OS-vendor advisories  
Debian / Alpine / RHEL  
security trackers  
(backport-aware)"]
    ECOADV["Ecosystem advisories  
OSV / GHSA / language DBs  
(Book 2, Ch 5)"]
    FIND["Findings  
component -> CVE -> severity  
fixed-in version / no-fix"]

    L0 --> INV
    L1 --> INV
    L2 --> INV
    INV -->|"OS packages"| OSADV
    INV -->|"language packages"| ECOADV
    OSADV --> FIND
    ECOADV --> FIND

```

Step 2 — match against the *right* vulnerability data

Matching is the SCA engine from Book 2, Chapter 6: for each component (name, version, ecosystem), find advisories whose affected-range covers that version. The identifiers (CVE, GHSA, OSV), the NVD/OSV data model, and the version-comparison logic are all as described there. The **container-specific twist is which database you match against**, and it is the difference between a useful scanner and a false-positive generator.

Language dependencies match against **ecosystem advisories** — OSV, GHSA, PyPA, RustSec, the npm advisory DB — keyed by the ecosystem's own version scheme (Book 2, Chapter 5). A `log4j-core 2.14.1` jar is CVE-2021-44228 (Log4Shell) regardless of which image it sits in,

because the JAR's identity and the advisory's affected-range are both in Maven-coordinate space. This half behaves like ordinary SCA.

OS packages are where containers diverge, and the reason is **distro backporting**.

The distro-backporting nuance

This is the most important container-scanning concept in the chapter, because getting it wrong turns your scanner into a false-positive firehose that teams learn to ignore — and an ignored scanner is worse than no scanner.

Distributions do not ship upstream version numbers, and they do not upgrade to fix vulnerabilities. They backport. When a CVE is found in, say, OpenSSL 1.1.1, Debian's security team does not bump the Debian package to the next upstream OpenSSL release — that would risk API breakage across everything linked against it in a stable release. Instead they take the *specific security patch*, apply it on top of the OpenSSL version already in the Debian release, and ship a package whose version string still says `1.1.1n-0+deb11u5`. The upstream marketing version is unchanged; the vulnerability is *fixed*; the fix is encoded in the **distro's own revision suffix** (`-0+deb11u5`, `~deb11u1`, the Alpine `-r5` revision, the RPM `.el9_3` dist tag).

The consequence for scanning is stark. If you take the OpenSSL package's *upstream* version (`1.1.1n`) and match it against NVD's CPE range for OpenSSL, NVD may well say "1.1.1 is vulnerable to CVE-XXXX" — and you report a finding for a package the distro **already patched weeks ago**. That is a false positive, and if you scan a few hundred images this way you generate thousands of them.

The correct behavior is to match OS packages against the **distribution's own security tracker**, which speaks in distro-package version space and knows the exact revision at which each CVE was fixed *for that release*:

- **Debian Security Tracker** — per-CVE, per-suite (bullseye, bookworm) status: fixed in version X, or "no-dsa" (won't fix), or open. <https://security-tracker.debian.org>
- **Ubuntu Security (USN)** and the OVAL data behind it — per-CVE, per-release, with the exact `.deb` version that carries the fix.

- **Alpine secdb** — per-branch (v3.19 , edge) mapping of package → fixed-in -rN revision.
- **Red Hat Security Data / OVAL**, and the equivalents from SUSE, Amazon Linux (ALAS), Oracle, Wolfi/Chainguard. Red Hat additionally publishes per-CVE **impact ratings** that often differ from NVD's CVSS, because Red Hat rates the vulnerability *as shipped and configured in RHEL*.

A scanner that consumes these sources compares your installed package version against the *fixed version for your specific release* and only reports a finding when your version is genuinely below it. Trivy, Grype, and Clair all do this — it is a defining feature of a container-aware scanner, and it is precisely what a naive "grep the version and hit NVD" approach gets wrong.

```

flowchart TD
    PKG["Installed package  
openssl 1.1.1n-0+deb11u5  
(Debian bullseye)"]

    subgraph NAIVE["Naive: upstream version vs NVD/CPE"]
        N1["Strip distro suffix -> 1.1.1n"]
        N2["NVD CPE: OpenSSL 1.1.1 < 1.1.1t vulnerable"]
        N3["FALSE POSITIVE:  
report CVE, but distro already patched"]
        N1 --> N2 --> N3
    end

    subgraph AWARE["Backport-aware: distro version vs distro tracker"]
        A1["Keep full version  
1.1.1n-0+deb11u5"]
        A2["Debian tracker: CVE fixed in  
1.1.1n-0+deb11u4 for bullseye"]
        A3["Installed u5 >= fixed u4  
-> NOT vulnerable, no finding"]
        A1 --> A2 --> A3
    end

    PKG --> NAIVE
    PKG --> AWARE

```

The mirror-image failure also exists: a distro can mark a CVE **won't-fix** ("no-dsa", "end-of-life", "affected but low priority") for a package that upstream *has* fixed. A backport-aware scanner will still surface those as open findings with a "no fix available" status, which is correct — the

package in your image really is vulnerable and the distro has chosen not to patch it. This is exactly the kind of finding that should drive you toward a **different base image** (Book 6, Chapter 3 — Base Image Strategy) rather than waiting for a fix that is never coming.

Beyond CVEs: what modern scanners also find

Early image scanners did CVE matching and nothing else. Modern scanners — Trivy is the clearest example — treat the image as a target for *several* analyses that happen to share the same layer-walk:

- **Secret detection.** Layers never delete (Book 6, Chapter 1): a `RUN` that copies in an AWS key, a `.npmrc` with a token, an SSH private key, or a `.env`, followed by a later `rm`, leaves the secret in the earlier layer's tar forever. Scanners run regex/entropy rules (the same family of detectors as Book 7, Chapter 4 — Secrets in Source, and the CI-side scanning in Book 4, Chapter 6) across the *layer contents*, catching credentials that a filesystem-only view would miss because the file no longer exists in the merged root. This is one of the highest-value, lowest-noise things an image scanner does.
- **Misconfiguration.** Dockerfile and image-config checks: running as `root` (no `USER` directive), `ADD` of a remote URL, use of `:latest` base tags, unnecessary `setuid` binaries, sensitive ports. Trivy folds this together with its IaC scanning (Book 6, Chapter 8 — Infrastructure as Code) so the same tool checks the Dockerfile, the Kubernetes manifests, and the Terraform.
- **License detection.** The SBOM already carries declared and detected licenses; the scanner can flag policy-violating licenses (GPL in a proprietary image, unknown/unlicensed components) — license risk management rather than security, but it rides the same inventory.
- **Malware.** A minority of scanners (mostly commercial runtime platforms — Prisma Cloud, Aqua, Sysdig) add signature/heuristic malware detection over image contents. Note the honest caveat carried forward from Book 2, Chapter 4 (Malicious Packages): CVE scanning does **not** detect malware or backdoors, because a backdoor is not a *known vulnerable version of a known component* — it is malicious code with no CVE. Malware detection is a separate capability bolted alongside, not a property of vulnerability matching.

The unifying point: the layer walk is expensive and you only want to do it once, so scanners amortize it across every analysis they can. But keep the categories distinct in your head — a clean CVE scan says nothing about secrets, misconfig, or malware unless the tool ran those passes too.

The tooling

The landscape sorts into open-source scanners, cloud-provider built-ins (usually one of the OSS engines under the hood), registry-integrated scanning, and commercial platforms. Treat vendor capability claims skeptically and evaluate by *what the tool actually inventories and what data it matches against* (Book 3, Chapter 7 — SBOM Quality and Limitations: different tools, different results).

Tool	Kind	Vuln	Secret	Misconfig	License	Malware	Notes
Trivy (Aqua)	OSS	✓	✓	✓	✓	–	Ubiquitous scanner for files, repos, IaC; <i>and</i> CycloneDX Backstage for m. distr.
Grype (Anchore)	OSS	✓	–	–	–	–	Purely matches with SBO Cons SBO (scan) matc
Syft (Anchore)	OSS	– (SBOM only)	–	–	✓ (detect)	–	SBOM generator, matches inventory, the e

Tool	Kind	Vuln	Secret	Misconfig	License	Malware	Notes
Clair (Quay/RH)	OSS	✓	-	-	-	-	Serv mod engi Quay (hist Harb inde back
Docker Scout	OSS-ish / SaaS	✓	-	✓	✓	-	Dock in; c scou SBO base reco ("up fix N
osv-scanner (Google)	OSS	✓	-	-	-	-	OSV stron lang light distr data
ECR / GAR / ACR scanning	Cloud built-in	✓	some	some	-	-	On-p cont "enh Ama Insp ACR Trivy engi integ imag in-cl

Tool	Kind	Vuln	Secret	Misconfig	License	Malware	Notes
Harbor integrated	Registry	✓	–	–	–	–	Plug Clair push qua gate re-sc Ch 2
Prisma Cloud / Aqua / Sysdig / Snyk Container / Wiz	Commercial	✓	✓	✓	✓	✓ (most)	Add corr reach regist inve work cent dash the p not t

A few honest distinctions worth internalizing:

- **Matcher vs. generator.** Syft generates; Grype matches. Trivy and Clair do both internally. The separation matters because it is the seam along which the scale strategy is built (next section).
- **The engine under the cloud button.** "ECR image scanning" and "GAR vulnerability scanning" are convenient because the images are already there and the scan runs with no pipeline changes — but they are OSS engines (Inspector, Clair/Trivy lineages) wearing a cloud badge, with the cloud's advisory feeds. Do not assume the cloud button finds more than Trivy; often it finds the same things with less configurability, traded for zero integration cost and native continuous re-scan.
- **Commercial value is operational, not detection.** For the *core* CVE-matching task, a well-fed Trivy is competitive with anything. What the commercial platforms sell is the layer *around* matching: fleet inventory, runtime reachability ("this vulnerable package is actually loaded in a running container"), central VEX/exception management, and admission integration. Whether that is worth the price is a program decision, not a detection-quality one.

SBOM-based scanning: scan once, match forever

Here is the move that makes scanning tractable at fleet scale, and it falls straight out of "scanning = SBOM + matching." The two halves have very different cost and cadence:

- **Inventory** is expensive (pull and unpack every layer of a large image) but only changes when the *image* changes — i.e., at build time.
- **Matching** is cheap (compare versions against a database) but its *result* changes every time the vulnerability data changes — i.e., every few hours, as new CVEs land.

Coupling them — re-pulling and re-unpacking every image every time you want fresh results — is what makes naive "scan the whole registry nightly" jobs so slow. Decouple them instead (Book 3, Chapter 5 — SBOM Distribution and Querying at Scale):

1. Generate the SBOM **once** per image build (Syft, or Trivy's SBOM output), store it as an OCI referrer to the image or in a central store keyed by image digest.
2. **Re-match** the stored SBOMs against the latest vulnerability database as often as you like — Grype and Trivy both accept an SBOM as input (`grype sbom:./sbom.json` , `trivy sbom sbom.spdx.json`). Re-matching a hundred thousand stored SBOMs is a database join, not a hundred thousand image pulls.

```
# Build time: expensive inventory, done once, attached to the image by
digest.
syft registry.internal/payments@sha256:abcd... -o spdx-json > payments.
sbom.json
cosign attach sbom --sbom payments.sbom.json registry.internal/
payments@sha256:abcd...

# Anytime later, on every DB refresh: cheap re-match, no image pull.
grype sbom:payments.sbom.json           # new CVEs surface without
touching the registry
```

This is the technical foundation of **continuous scanning** and of the central-aggregation program at the end of the chapter. An image whose bytes never change still develops new CVEs as the world learns about them (Log4Shell is the canonical case — Book 1, Chapter 5); the SBOM

re-match pattern is how you learn *today* which of yesterday's clean images are affected, without re-scanning the world.

Where scanning happens: the scan points

Scanning is not a single gate; it is the same analysis wired in at four points in the image lifecycle, each answering a different question and taking a different action. "Shift left" is real and valuable — catch it in CI where the developer who caused it can fix it — but shift-left alone is a trap, because it is a **point-in-time** verdict and vulnerabilities are discovered *after* the build. You need the left-hand gates *and* the continuous right-hand re-scan.

```
flowchart LR
    DEV["Developer / PR"] --> CI
    subgraph CI["1. Build / CI"]
        B["Scan built image  
fail on policy"]
    end
    CI -->|push| REG
    subgraph REG["2. Registry"]
        R["Scan on push  
quarantine / gate"]
        RC["Continuous re-scan  
SBOM re-match on new CVEs"]
        R --- RC
    end
    REG -->|deploy| ADM
    subgraph ADM["3. Admission"]
        A["Block vulnerable / unsigned  
images at kube-apiserver"]
    end
    ADM -->|runs| RUN
    subgraph RUN["4. Runtime"]
        RU["Re-scan running images  
correlate reachability  
new CVE -> what is affected NOW"]
    end
    RC -->|"new CVE in an already-deployed image"| RU
```

Scan point	Question it answers	Typical action
Build / CI	Is the image I just built clean <i>right now</i> , by policy?	Fail the pipeline; block the push. Fast feedback to the author (Book 4).
Registry (on push)	Did anything vulnerable get into our store?	Scan on ingest at the chokepoint (Book 6, Ch 2); quarantine / mark not-deployable (Harbor gate, ECR findings).
Registry (continuous)	Which stored images became vulnerable <i>since</i> we scanned them?	Periodic SBOM re-match; open tickets / notify owners; feed admission and runtime.
Admission	Should this specific image be allowed to run in <i>this</i> cluster?	Deny the Pod at the kube-apiserver on failing policy or missing scan verdict (Book 6, Ch 5-6).
Runtime	Of the images <i>actually running</i> , which are affected by the CVE that dropped this morning, and is the vulnerable code reachable?	Re-scan running inventory; prioritize by exposure; drive emergency rebase/redeploy.

The four are complementary, not redundant:

- **Build/CI** gives the fastest, cheapest feedback and stops obviously-bad images at the source — but its verdict is stale the moment a new CVE is published, and it cannot see images built before the gate existed. This is where you *fail the build* on policy (Book 4, Chapter 3 — SLSA, and the gating patterns of Book 4).
- **Registry-on-push** catches what bypassed CI (hand-pushed images, third-party images pulled through your mirror) at the one chokepoint every image transits (Book 6, Chapter 2). Harbor can **quarantine** an image — refuse to serve it — on a failing scan.
- **Registry-continuous** is the part people forget and the part that matters most. Because it re-matches stored SBOMs against fresh vuln data, it is your early-warning system: it turns "CVE-2021-44228 published" into "these 47 images in our registry contain vulnerable log4j" within a scan cycle, *without* anyone rebuilding anything.

- **Admission** is the *enforcement* point — the only place you can actually prevent a vulnerable image from running (Book 6, Chapters 5–6). It typically checks a *verdict* produced elsewhere (an attestation, a registry annotation, a policy-engine query) rather than scanning inline, because you do not want to block a Pod for the seconds-to-minutes a fresh scan takes. Deploy-by- digest (Book 6, Chapters 1–2) is what makes the admission verdict trustworthy: the digest the scanner rated is the digest that runs.
- **Runtime** closes the loop for images that were already deployed when a CVE dropped. It also adds information the other points cannot: *reachability in context* — a runtime platform can see that the vulnerable library is loaded into a running process, or that the exposed port is actually served, sharpening prioritization (Book 2, Chapter 7 — Reachability and Prioritization).

Continuous re-scan is the load-bearing idea. An image scanned clean at build and at push is not clean forever; it is clean *as of the vuln data on that date*. The Log4Shell reality (Book 1, Chapter 5) is that a component sitting untouched in thousands of images for years became critically exploitable overnight. A program without continuous re-scan learns about that from the news; a program with it learns from its own dashboard, scoped to its own fleet, before the news.

Operational vulnerability management for images

This is the hard part, and it is where Book 2, Chapters 6–7 (SCA and prioritization) meet the brutal arithmetic of containers.

The volume problem, intensified

A single dependency manifest might have a handful of CVEs. A container image starts with an entire base OS. A stock `ubuntu` or `debian` base carries dozens to low-hundreds of OS-package CVEs at any given moment — many low-severity, many won't-fix, many in packages you never invoke. Now multiply: hundreds of services × several live versions each × that base-image CVE count, and you have tens of thousands of raw findings. Dumped into a dashboard undifferentiated, this number is not just useless — it is *harmful*, because it trains every engineer who sees it to treat the scanner as noise. The entire discipline of image vuln

management is **turning that raw number into a small, ranked, actionable list**, and there are six levers, roughly in order of impact:

1. Minimal / distroless bases — cut findings at the source. The most effective noise reduction is to *have fewer components*. A distroless or minimal base (Book 6, Chapter 3) with no shell, no package manager, and a handful of libraries has a single-digit CVE count where a full distro base has hundreds. You cannot have a CVE in a package that is not in the image. This is the highest-leverage move because it reduces findings *and* attack surface *and* image size simultaneously, and it does so for every service that adopts the base. Chainguard/Wolfi "zero-CVE" images push this to its logical end.

2. Backport-aware matching — do not manufacture false positives. Covered above. This is table stakes: use a scanner that consumes distro security-tracker data, or a large fraction of your OS findings are fictional and your team is right to ignore them.

3. VEX — suppress the non-exploitable, with a paper trail. A Vulnerability Exploitability eXchange document (Book 3, Chapter 6 — VEX) is the machine-readable assertion that a given CVE in a given product is `not affected` (and why: "vulnerable code not present," "not in execution path," "inline mitigation exists") or `affected / fixed`. VEX is what lets the *producer* of an image (who knows the code) tell the *consumer's scanner* to stop reporting a CVE that is present-but-not-exploitable — turning a recurring false-positive into a one-time, auditable decision. Trivy and Gripe both consume VEX (OpenVEX, CSAF). At fleet scale VEX is how you keep a suppression from being a silent per-scanner mute: it travels with the image and is reviewable.

4. Reachability — is the vulnerable code actually invoked? For *application* dependencies, reachability analysis (Book 2, Chapter 7) can downgrade a CVE in a library whose vulnerable function your code never calls. For OS packages this is genuinely hard — a shared library may be linked by many processes and "reachability" is murky — which is one more reason OS-package findings are better addressed by *removing or rebasing* than by per-CVE reachability triage. Runtime platforms approximate reachability differently: is the package loaded in a running process, is the port exposed. Use it to *rank*, not to dismiss without a VEX record.

5. Prioritization — EPSS and KEV. Severity (CVSS) alone is a poor sort key; a fleet has too many "high" CVEs to fix them all at once. Rank by *likelihood and reality of exploitation* (Book 2, Chapters 5 and 7): **EPSS** (the FIRST exploit-prediction score, a 0-1 probability of exploitation in the wild) and **CISA KEV** (the Known Exploited Vulnerabilities catalog — CVEs *observed* being exploited). A KEV-listed CVE with a high EPSS in a reachable, exposed component is a drop-everything; a high-CVSS CVE with EPSS near zero, no KEV listing, in a non-reachable package, is a backlog item. This is the difference between a program that fixes what matters and one that drowns.

6. Fix by rebasing — the dominant remediation. The single most important operational fact about image CVEs: **most of them are fixed by bumping the base image, not by patching individual packages.** The OS-package CVEs that dominate the raw count were fixed *by the distro*, and the way you get those fixes is to rebuild on a newer base that already contains them. You do not `apt upgrade openssl` in a running container; you rebuild the image on `debian:bookworm-<newer>` (or a freshly-rebuilt golden base) and the whole cluster of OS CVEs clears at once. This reframes remediation from "triage 40 CVEs" to "advance one base-image pin," which is a far more tractable and automatable operation — and it is the direct link to the base-image freshness/auto-rebuild program of Book 6, Chapter 3.

Remediation for images, specifically

Image remediation has three moves, and their relative frequency is the opposite of what naive per-CVE triage assumes:

- **Rebuild on an updated base (the common case).** Bump the base-image reference to a version that incorporates the distro's fixes and rebuild. Most OS-package findings vanish in one action. When a **golden base image** (Book 6, Chapter 3) is auto-rebuilt centrally and downstream services auto-rebuild on top of it, this is a fleet-wide operation triggered once.
- **Update application dependencies (the language half).** For findings in your jars/wheels/modules, the fix is the ordinary dependency bump of Book 2, Chapter 9 (Dependency Update Strategy) — Dependabot/Renovate raising the version, re-resolving the lockfile, rebuilding. Rebasing does *not* fix these; they are your code's dependencies, not the OS's.

- **Remove the unneeded package.** If the vulnerable component is not needed at runtime, delete it — drop the build-only toolchain via multi-stage builds (Book 6, Chapter 1), switch to a base without it, or explicitly remove it. A removed package has no CVEs. This overlaps with the minimal-base strategy: much "remediation" is really "stop shipping things you do not run."

```
flowchart TD
    CVE["CVE["Continuous re-scan flags  
base-image CVE across the fleet  
e.g. CVE in glibc / openssl"]"]
    DISTRO["DISTRO["Distro ships fix  
in a newer package revision"]"]
    BUMP["BUMP["Bump golden base image pin  
(Book 6, Ch 3)"]"]
    REBUILD["REBUILD["Auto-rebuild base  
-> auto-rebuild dependent services  
(fleet-wide, one trigger)"]"]
    NEWIMG["NEWIMG["New image digests  
with patched base"]"]
    RESCAN["RESCAN["Re-scan / SBOM re-match"]"]
    CLEAR["CLEAR["Base CVEs cleared  
across every rebuilt service"]"]
    DEPLOY["DEPLOY["Roll out by digest  
through admission gate"]"]

    CVE --> DISTRO --> BUMP --> REBUILD --> NEWIMG --> RESCAN --> CLEAR
    CLEAR --> DEPLOY
```

The metric that captures this whole program is **MTTR-to-rebase across the fleet**: from "a critical CVE with a distro fix is published" to "every running image has been rebuilt on a base that contains the fix and redeployed." A mature program measures and drives that number down; it is a far better health signal than "count of open findings," which minimal bases and VEX can move without any real risk reduction.

Policy and gating

Gating is where scan verdicts become enforcement, and the cardinal rule is **warn before you enforce** (the rollout discipline of Book 1, Chapter 10 — Building a Program). A gate that blocks builds on day one, before anyone has cleaned the backlog, gets an emergency exception carved through it within a week and never recovers its authority. Turn it on in *warn* mode, publish the findings, give teams a remediation window, then flip to *enforce*.

A workable policy has these dimensions:

- **Thresholds combining severity + exploitability + fixability.** Not "block on any CVSS ≥ 7 ." Something like: *block on Critical, or on KEV-listed, or on EPSS > 0.5, and a fix is available* — the last clause matters because blocking on a *won't-fix* CVE just wedges teams with no action to take. Reachability and KEV/EPSS keep the gate focused on what is both dangerous and actionable.
- **Grace periods.** A newly-published CVE gets a window (e.g., 7/14/30 days by severity) before it becomes blocking, so a Friday-night disclosure does not brick Monday's deploys. The clock and the policy should be explicit and visible.
- **Exceptions with expiry.** Every suppression is time-boxed and owned. A permanent exception is a policy hole; an exception that auto-expires forces re-justification. Back exceptions with **VEX** where the claim is "not exploitable," so the reasoning is machine-readable and portable rather than a dashboard mute.
- **Base-image-age policy.** Independent of specific CVEs, gate on *freshness*: refuse an image whose base is older than N days, or not the current golden base (Book 6, Chapter 3). Because rebase is the dominant remediation, enforcing base freshness pre-empts most CVE gate failures — you fix the class instead of the instances.

```
# Sketch of a Trivy gating config used in CI (trivy.yaml) – fail the
build
# only on actionable, exploitable, fixable findings.
severity:
  - CRITICAL
  - HIGH
scan:
  security-checks: [vuln, secret, misconfig]
vulnerability:
  ignore-unfixed: true          # do not block on won't-fix / no-patch
findings
# .trivyignore or VEX carries time-boxed, justified exceptions.
```

Central aggregation — the fleet query

The final operational necessity: **do not scan 500 images into 500 dashboards**. A per-image scan result answers "is *this* image vulnerable?" The question a security team actually gets paged with is the inverse and fleet-wide: **"which of our running images are affected by**

CVE-2021-44228, right now?" You can only answer that from a *central* store that holds every image's SBOM plus current vuln data plus VEX (Book 2, Chapter 6; Book 3, Chapter 5).

Concretely, that is **Dependency-Track** (ingests CycloneDX SBOMs, continuously re-matches them against OSV/NVD, tracks findings and VEX per project), **Harbor's** aggregated scan view for images in the registry, or the fleet inventory in a commercial platform. The pattern is the SBOM-re-match one from earlier, operated centrally: SBOMs flow in at build time, the matcher re-runs on every DB update, and the store supports the pivot query — *component* → *images* → *running deployments*. That pivot is your Log4Shell capability. When the next Log4Shell lands, the difference between "we queried the aggregator and had the list of 47 affected services in minutes" and "we spent three days grepping build logs across 500 repos" is entirely whether you built the central store *before* you needed it.

The limits — honest, and important

Scanning is necessary and it is not sufficient. State the boundaries plainly so no one mistakes a green scan for a safe image (Book 2, Chapter 6; Book 3, Chapter 7):

- **It finds *known* vulns in *known* components.** The whole mechanism is "match my inventory against a database of disclosed vulnerabilities." A vulnerability nobody has disclosed (**zero-day**) is invisible. A component the scanner failed to identify (an unpackaged binary, a renamed jar, a vendored library with stripped metadata) contributes *no* findings — absence of findings is not absence of vulnerability, it can be absence of *inventory* (Book 3, Chapter 7).
- **It is not malware detection.** CVE matching cannot find a backdoor, because a backdoor is not a known-vulnerable version of a known component (Book 2, Chapter 4; Book 1's SolarWinds and xz cases in Chapters 3 and 5). The xz backdoor shipped *inside a legitimately-versioned package*; no version-matching scanner would flag it. Some scanners add a separate malware pass — that is a different capability, not a property of the CVE scan.
- **It says nothing about your application's logic.** SQL injection, broken authz, SSRF in *your* code (Book 9, Volume-1 appsec territory) are not components with CVEs. Image scanning does not see them.

- **It says nothing about runtime configuration.** An image can scan perfectly and still be deployed with a mounted host path, a privileged securityContext, or a leaked service-account token. That is admission and runtime posture (Book 6, Chapters 5–6), not image contents.
- **Scanner results disagree.** Different tools, different advisory feeds, different matching heuristics → different results on the same image (Book 3, Chapter 7). One scanner's `Critical` is another's non-finding, often because of backport-data quality or identifier mismatches. Pick a scanner deliberately, understand its data sources, and do not treat any single tool's number as ground truth.

None of this argues against scanning. It argues against *complacency* about scanning: it is one control in a stack that also includes provenance (Book 5), signing and admission (Book 6, Chapters 5–6), minimal bases (Book 6, Chapter 3), and secrets and source integrity (Book 4, Book 7). A green scan means "no *known* vulnerabilities in *identified* components as of today's data" — a precise and useful statement, and a much narrower one than "safe."

Distributed-systems lens

Every theme in this chapter is really the same theme: **image scanning is fleet-scale vulnerability management, and the only way it works is by placing the work at chokepoints and doing matching continuously.** You cannot secure hundreds of services by scanning each in isolation; you secure them by exploiting the structure of the container supply chain.

- **Scan at the registry chokepoint** (Book 6, Chapter 2), and **re-match SBOMs continuously** (Book 3, Chapter 5). Because every image transits the internal registry once, and because the SBOM decouples expensive inventory from cheap matching, *one* pipeline covers every service and keeps covering it as new CVEs arrive. The registry is to images what the internal proxy is to packages: the single place worth instrumenting.
- **The dominant remediation is rebase, and rebase is a fleet operation.** Pair scanning with the golden-base-image + auto-rebuild program (Book 6, Chapter 3): patch the base *once*, rebuild the fleet, and most image CVEs vanish across every service simultaneously. This is the property that makes container vuln management tractable

at all — the fix is centralized even though the findings are distributed.

- **Central aggregation answers the fleet question.** "Which running images are affected by CVE-X" is unanswerable per-image and trivial from a central SBOM+vuln store (Dependency-Track, Harbor, commercial). Build it before the incident; it is your Log4Shell response capability.
- **Minimal bases cut the fleet's noise at the source** (Book 6, Chapter 3). Fewer components, fleet-wide, means fewer findings, fleet-wide — the cheapest scale lever there is.
- **Gate at admission** (Book 6, Chapters 5–6) so a vulnerable image *cannot deploy*, and **deploy by digest** (Book 6, Chapters 1–2) so the verdict binds to the bytes that run.
- **The metric is MTTR-to-rebase across the fleet**, not raw finding count. A program is healthy when a critical, exploited, fixable CVE goes from disclosure to fully-redeployed-fleet fast — and when it can *prove* that state from its central store, image by image, on demand.

Key takeaways

- **Image scanning is SCA on an image:** build a component inventory across all layers (OS package DBs — dpkg/apk/rpm — plus language deps plus binaries), then match against vulnerability data. The inventory step is SBOM generation (Book 3, Chapter 4).
- **Distro backporting is the defining nuance.** Distros patch by backporting fixes without changing upstream version strings, so OS packages must be matched against **distro security trackers** (Debian/Alpine/Red Hat), not upstream/NVD version ranges — else you generate massive false positives. This is the single biggest quality difference between scanners.
- **Modern scanners do more than CVEs:** secret detection across layers (catching secrets a later `rm` could not remove), Dockerfile/misconfig checks, license flags, and — for some — malware. But CVE matching does *not* find malware, backdoors, zero-days, or your logic bugs.
- **Trivy, Grype+Syft, Clair, Docker Scout** are the OSS core; cloud built-ins (ECR/GAR/ACR) and Harbor wrap OSS engines with zero-integration deployment; commercial platforms sell the operational

layer (fleet inventory, runtime reachability, VEX workflows), not better matching.

- **SBOM-based scanning — generate the inventory once, re-match forever** — decouples expensive inventory from cheap matching and is the technical foundation of continuous scanning at scale.
- **Scan at four points:** build/CI (fail fast), registry-on-push (quarantine at the chokepoint), registry-continuous (early warning as new CVEs drop), admission (the actual enforcement gate), and runtime (what's affected *now*, with reachability). **Continuous re-scan is non-negotiable** — yesterday's clean image has today's CVEs (Log4Shell).
- **Manage volume with six levers:** minimal/distroless bases (cut at source), backport-aware matching (no false positives), VEX (suppress non-exploitable with a paper trail), reachability (rank, don't dismiss), EPSS/KEV prioritization (fix what's exploited), and **rebase-to-fix** (the dominant remediation — bump the base, rebuild the fleet, most OS CVEs clear at once).
- **Gate with warn-then-enforce**, thresholds that combine severity + KEV/EPSS + *fixability*, grace periods, time-boxed VEX-backed exceptions, and a **base-image-age** policy that pre-empts most CVE failures by keeping the base fresh.
- **Aggregate centrally** (Dependency-Track / Harbor / commercial) so you can answer the fleet-wide "which running images are affected by CVE-X" — the pivot query is your incident-response capability.
- Scanning is **necessary, not sufficient**. A green scan means "no known vulns in identified components as of today's data" — precise, useful, and much narrower than "safe."

Further reading

- **Trivy** — the scanner, its target types, and its backport-aware distro data sources. <https://trivy.dev/latest/docs/>
- **Grype** and **Syft** (Anchore) — the matcher and the SBOM generator, and the SBOM-in / scan-once model. <https://github.com/anchore/grype> · <https://github.com/anchore/syft>
- **Clair** — the layer-indexed scanning engine behind Quay and Harbor. <https://quay.github.io/clair/>

- **Docker Scout** — SBOM-based scanning and base-image update recommendations. <https://docs.docker.com/scout/>
- **osv-scanner** and the **OSV** database — ecosystem-native vulnerability matching (Book 2, Chapter 5). <https://google.github.io/osv-scanner/> · <https://osv.dev>
- **Debian Security Tracker**, **Ubuntu Security Notices**, **Alpine secdb**, and **Red Hat Security Data** — the distro advisory sources that make backport-aware matching possible. <https://security-tracker.debian.org> · <https://ubuntu.com/security/notices> · <https://secdb.alpinelinux.org> · <https://access.redhat.com/security/data>
- **CISA KEV** (Known Exploited Vulnerabilities) and **EPSS** (FIRST) — exploitation-based prioritization (Book 2, Chapters 5 and 7). <https://www.cisa.gov/known-exploited-vulnerabilities-catalog> · <https://www.first.org/epss/>
- **OpenVEX** and the **CSAF/VEX** profile — machine-readable exploitability assertions to suppress non-exploitable findings (Book 3, Chapter 6). <https://github.com/openvex> · <https://docs.oasis-open.org/csaf/csaf/v2.0/csaf-v2.0.html>
- **Dependency-Track** — central SBOM ingestion, continuous re-analysis, and fleet-wide component queries (Book 3, Chapter 5). <https://dependencytrack.org>
- **Amazon Inspector / ECR enhanced scanning**, **Google Artifact Registry scanning**, and **Azure Container Registry / Defender** — cloud-built-in continuous image scanning. <https://docs.aws.amazon.com/inspector/> · <https://cloud.google.com/artifact-analysis/docs> · <https://learn.microsoft.com/azure/defender-for-cloud/>
- **Harbor** — registry-integrated scanning, project quarantine gates, and periodic re-scan (Book 6, Chapter 2). <https://goharbor.io/docs/latest/administration/vulnerability-scanning/>
- Cross-references within this suite: Book 2, Chapters 5–7 (vuln data, SCA, reachability), Book 2, Chapter 9 (dependency updates); Book 3, Chapters 4–7 (SBOM generation, scale, VEX, limitations); Book 1,

Chapter 5 — Image Signing and Verification in Kubernetes

What this chapter covers. Book 5 built the general machinery of signing and attestation: content-addressed digests (Chapter 1), Sigstore's cosign/Fulcio/Rekor architecture (Chapter 3), keyless signing from workload identity (Chapter 4), transparency logs (Chapter 5), in-toto attestations (Chapter 6), verification in practice (Chapter 8), and the design of attestation-based deployment gates (Chapter 10). This chapter takes that machinery and points it at one specific artifact — the **container image** — and one specific enforcement point — the **Kubernetes cluster**. The signing half is a short recap: cosign signs an image *by digest* and stores the signature next to the image in the registry (Book 6, Chapter 1's referrers model). The verification half is the substance, because it is where the whole edifice earns its keep. A signature nobody checks is theatre (Book 5, Chapter 8); the check that matters for containerized backends happens at **admission**, the moment the API server decides whether a pod may run. This chapter explains the Kubernetes admission flow, where image-verification controllers plug into it, what they actually verify, the real controllers (Sigstore policy-controller, Kyverno, Connaisseur, Ratify+Gatekeeper, Binary Authorization) with working policy, the **verify-and-run-the-same-digest** imperative that closes the TOCTOU gap from Book 6, Chapter 1, and the brutal operational reality that this webhook sits in the critical path of *every pod in the fleet*. The policy language itself — Rego, CEL, Kyverno's engine — is Chapter 6; here we stay on the image-verification aspect.

Learning goals — after this chapter you should be able to:

- Recap how **cosign signs an image by digest** and stores the signature and attestations (SBOM, SLSA provenance, vuln scan, VEX) as OCI artifacts referencing that digest.
- Explain the **Kubernetes admission flow** (API server → mutating webhooks → schema validation → validating webhooks → etcd) and where image-verification controllers register.

- State exactly **what an admission-time image check verifies**: resolve tag→digest, verify signature and expected signer identity, verify required attestations, check Rekor inclusion.
- Explain the **verify-and-run-the-same-digest** rule and why a controller that verifies a tag but lets the kubelet re-resolve it later has closed nothing.
- Compare **policy-controller**, **Kyverno verifyImages**, **Connaisseur**, **Ratify+Gatekeeper**, and **Binary Authorization**, and write a working `ClusterImagePolicy` or Kyverno `verifyImages` rule.
- Reason about **fail-open vs fail-closed**, `failurePolicy`, namespace exclusions, HA, and why this webhook is **tier-0** — a down webhook can stop the whole fleet from scheduling.
- Design a **tiered rollout**: audit → enforce, strict policy for internal images, relaxed policy for third-party images, fleet-wide.

Signing the image: a Book 5 recap, image-specific

You have seen this in Book 5, Chapter 3; the image-specific mechanics are small and worth stating precisely because the verification side depends on them being exactly right.

An image's identity is its **manifest digest** — `sha256:...` over the manifest bytes (Book 6, Chapter 1). A tag like `registry.example.com/app:1.4.2` is a mutable pointer; the digest is the immutable thing. **Cosign signs the digest, never the tag**. When you run:

```
cosign sign registry.example.com/app@sha256:9f2b...c4
```

cosign does not modify the image. It computes a signature over a small **payload** that binds the image digest (a `simplesigning` claim referencing `sha256:9f2b...c4`), and it *pushes the signature back into the registry* as a separate OCI artifact that references the image by digest. Two storage conventions exist, both covered in Book 6, Chapter 1:

- **The `.sig` tag convention (legacy)**. The signature manifest is pushed to a derived tag `sha256-9f2b...c4.sig` in the same repository. Attestations go to `...c4.att`, SBOMs attached with `cosign attach sbom` to `...c4.sbom`. This works on any OCI registry, including old ones.

- **OCI 1.1 Referrers (current).** The signature manifest carries a `subject` field pointing at the image digest, and the registry's **Referrers API** lets a verifier list "everything that refers to `sha256:9f2b...c4`." This is cleaner — no tag-name games, garbage-collected with the image — and cosign uses it when the registry supports it (`--registry-referrers-mode oci-1-1`), falling back to the tag scheme otherwise (Book 3, Chapter 5).

The important invariant for verification: **the signature is addressed to a digest**. A verifier that has the digest can find and check the signature; a verifier holding only a tag must first resolve the tag to a digest, and *that resolution is the security-critical step* we return to below.

Keyless is the norm for CI-built images

For images built in CI, **keyless signing (Book 5, Chapter 4)** is the default, not an optimization. The build job has a workload identity — a GitHub Actions OIDC token, a GitLab CI JWT, a Google/AWS service-account token — and cosign exchanges that token with **Fulcio** for a short-lived (≈ 10 -minute) X.509 certificate whose Subject Alternative Name *is the build identity*: the workflow ref, the repository, the issuer. cosign signs, records the certificate and signature in **Rekor** (the transparency log, Book 5, Chapter 5), and throws the key away. No long-lived signing key exists to steal or rotate (Book 5, Chapter 9). The command in a GitHub Actions job is just:

```
cosign sign --yes \  
  registry.example.com/app@sha256:9f2b...c4  
# OIDC identity is picked up from the ambient GITHUB_TOKEN / id-token;  
Fulcio  
# issues a cert whose SAN is:  
#   https://github.com/example-org/app/.github/workflows/  
release.yml@refs/tags/v1.4.2  
# and the entry lands in Rekor.
```

That SAN — the workflow file, the ref, the issuer — is precisely what the *cluster* will later demand. The build platform's identity is the anchor of trust for the whole scheme (Book 4, Chapter 10; Book 5, Chapter 4). Verification does not ask "is this signed?"; it asks "is this signed **by the identity we expect to build this image?**"

Signing attestations too

A signature says *this exact image was blessed by this identity*. An **attestation** (Book 5, Chapter 6) says *this exact image has this property*, signed by an identity. The same build platform that signs the image attaches, as signed attestations referencing the same digest:

- an **SBOM** (SPDX or CycloneDX — Book 3), so the cluster and downstream tooling know what is inside;
- **SLSA provenance** (Book 4, Chapter 3), the signed record of *how* and *where* the image was built — which builder, which source commit, which parameters;
- a **vulnerability-scan** attestation (Book 6, Chapter 4), the scan result at build time;
- optionally a **VEX** statement (Book 3, Chapter 6), asserting which known vulnerabilities are not exploitable in this image.

```
# SLSA provenance produced by the build, attached as a signed
attestation:
cosign attest --yes --type slsaprovenance \
  --predicate provenance.json \
  registry.example.com/app@sha256:9f2b...c4

# SPDX SBOM as a signed attestation:
cosign attest --yes --type spdxjson \
  --predicate sbom.spdx.json \
  registry.example.com/app@sha256:9f2b...c4
```

The result is a **metadata bundle in the registry**, all hanging off the one digest: the image, its signature, and one signed attestation per predicate type. This is the produce side of the produce-and-verify loop (Book 4, Chapter 10). The cluster is where the loop closes.

Who signs, and when

The answer is a design principle, not a preference: **the build platform signs at build time with its own workload identity, automatically, for every image**. Signing is not a step an engineer remembers to run; it is a stage of the paved-road pipeline (Book 4, Chapter 10) that no team opts out of and none can forge, because the identity in the signature is the CI system's, minted fresh per run and unavailable to humans. If signing is a manual afterthought, coverage is partial, coverage gaps are exactly where malware lands, and the admission policy below cannot be

set to "require a signature" without breaking half the fleet. Automatic, paved-road signing is the precondition for enforceable verification.

```
flowchart LR
    subgraph Build["Build platform (CI, workload identity)"]
        B1["Build image → sha256:9f2b...c4"]
        B2["cosign sign (keyless)"]
        B3["cosign attest: SBOM, SLSA prov, scan, VEX"]
    end
    subgraph Reg["Registry (OCI)"]
        R1["image manifest\nsha256:9f2b...c4"]
        R2[".sig / referrer\nsignature"]
        R3[".att / referrers\nattestations"]
    end
    subgraph Trust["Sigstore"]
        F["Fulcio\n(short-lived cert, SAN=build identity)"]
        K["Rekor\n(transparency log)"]
    end
    subgraph Cluster["Kubernetes"]
        A["Admission: verify signature+identity\n+attestations+Rekor, run the verified digest"]
    end
    B1 --> B2 --> B3
    B2 -. OIDC .-> F
    B2 -. record .-> K
    B2 --> R2
    B3 --> R3
    B1 --> R1
    A -- pull manifest+refs --> Reg
    A -. verify cert/log .-> Trust
```

Verifying in Kubernetes: admission control is the enforcement point

Here is the goal, stated as a negative because negatives are what policies enforce: **Kubernetes must refuse to run a pod whose image is not signed by an expected identity, or lacks the attestations we require.** Everything upstream — the signing, the attestations, the Rekor entries — is inert until something *checks* it at the moment of deployment. In Book 5, Chapter 10 we called this the deployment gate. In a Kubernetes fleet the gate is **admission control**, and it is the one place where enforcement is both **fleet-wide** and **non-bypassable**: it is not a CI step a `git push --force` can skip, not a scanner a developer can silence,

but a check the API server itself performs before the object is ever persisted.

The Kubernetes admission flow

When any client — `kubectl`, a Deployment controller creating a pod, an autoscaler — sends a create/update request to the API server, the request passes through a fixed pipeline before the object reaches etcd:

1. **Authentication** — who is the caller.
2. **Authorization** (RBAC) — may the caller do this.
3. **Mutating admission webhooks** — external HTTP callbacks that may *modify* the object (inject sidecars, set defaults, and — for our purposes — **rewrite an image tag to a digest**).
4. **Object schema validation** — the API server validates the (possibly mutated) object against the resource schema.
5. **Validating admission webhooks** — external HTTP callbacks that may *only accept or reject* the object; they cannot change it.
6. **Persist to etcd** — the object is stored, and controllers/kubelet act on it.

The two webhook phases are configured by `MutatingWebhookConfiguration` and `ValidatingWebhookConfiguration` objects. Each names an endpoint (a Service in the cluster, or an external URL), the resources/operations it intercepts (`rules`), which namespaces/objects (`namespaceSelector`, `objectSelector`), a `timeoutSeconds` (max 30), and — the field that decides your fleet's fate — a **failurePolicy** of `Ignore` (fail-open) or `Fail` (fail-closed).

Image-verification controllers plug in at both phases. They register a **validating** webhook to do the actual verification (accept the pod only if its images verify) and, crucially, a **mutating** webhook to *resolve the tags they verified to digests and pin them into the pod spec*. The mutating phase runs first, so by the time the validating webhook and later the kubelet see the pod, the image references are digests. We will see why that ordering is not incidental but load-bearing.

```

flowchart TD
    C["kubectl / controller:\ncreate Pod (image app:1.4.2)"] --> API["API server"]
    API --> AUTH["AuthN + AuthZ (RBAC)"]
    AUTH --> MUT["Mutating admission webhooks"]
    MUT --> IVC_M["Image-verification controller (mutating):\nresolve tag=digest, pin sha256:9f2b...c4 into pod spec"]
    IVC_M --> SCH["Schema validation"]
    SCH --> VAL["Validating admission webhooks"]
    VAL --> IVC_V["Image-verification controller (validating):\nverify signature + identity + attestations + Rekor\nnon sha256:9f2b...c4"]
    IVC_V -->|admit| ETCD["Persist to etcd → scheduled → kubelet pulls sha256:9f2b...c4"]
    IVC_V -->|deny| REJ["422 rejected: image not signed by expected identity"]

```

What the controller actually verifies

Verification at admission is Book 5, Chapter 8 applied to an image. In order:

1. **Resolve the image reference to a digest.** If the pod spec says `app:1.4.2`, the controller asks the registry "what digest does `1.4.2` point to *right now*?" and gets `sha256:9f2b...c4`. Everything after this operates on the digest. **The controller must run this resolution once and then verify and pin that same digest** — the TOCTOU point of the next section.
2. **Verify the signature and the expected signer identity.** The controller fetches the signature (via referers or the `.sig` tag), verifies the cryptographic signature over the digest, and — for keyless — verifies the Fulcio certificate: that it chains to the Fulcio root the cluster trusts, and that its SAN matches the **expected identity**. In cosign terms this is `--certificate-identity` and `--certificate-oidc-issuer` (Book 5, Chapter 4). "Signed" is not enough; "signed by `https://github.com/example-org/app/.github/workflows/release.yml@refs/tags/*` via `https://token.actions.githubusercontent.com`" is the check. An attacker who signs a malicious image with *their own* keyless identity produces a perfectly valid signature under the *wrong* identity, which the policy rejects.
3. **Verify required attestations.** For each required predicate type (SLSA provenance, SBOM, scan), fetch the attestation, verify *its* signature and signer identity the same way, and optionally evaluate a

predicate policy over its contents (e.g., provenance's `builder.id` is our builder; SLSA build level ≥ 3). Missing or wrongly-signed attestation $\rightarrow$ deny.

4. **Check Rekor inclusion.** For keyless, the short-lived certificate has long since expired by the time the pod is admitted. The proof that the signature was created *while the certificate was valid* is the **Rekor** entry and its signed timestamp (Book 5, Chapter 5). The controller verifies the Rekor inclusion proof / signed entry timestamp against the log's public key. This is what makes keyless verification sound offline-of-the-signer: the log, not a live key, is the witness. A private Sigstore deployment supplies its own Fulcio/Rekor roots — in policy-controller via a `TrustRoot` custom resource, in cosign via a TUF root (Book 5, Chapter 7).

Only if all four hold does the controller admit — and it admits the pod **running the digest it just verified**.

The digest-pinning imperative: verify-and-run the same bytes

This is the single most important idea in the chapter, and it is where naïve implementations silently fail. Recall the mutable-tag / immutable-digest TOCTOU problem from Book 6, Chapter 1:

A **time-of-check-to-time-of-use** gap opens whenever you *verify* one thing and *use* another.

A pod spec that references `app:1.4.2` is a reference to *whatever* `1.4.2` points to at pull time. Suppose the controller does this: sees `app:1.4.2`, resolves it to `sha256:9f2b...c4`, verifies that digest, admits the pod — and leaves the pod spec saying `app:1.4.2`. The pod is persisted with a **tag**. Later, the kubelet on the node resolves `app:1.4.2` **again** to pull it. Between the controller's resolution and the kubelet's pull, an attacker with push access (or a compromised CI, or a registry-account takeover — Book 6, Chapter 2) re-points the tag `1.4.2` to `sha256:BADBAD...`. The kubelet pulls and runs `BADBAD`, which was **never verified**. The check happened; the use happened on different bytes. Signing accomplished nothing.

The defense is a rule with two halves, and **both** are mandatory:

1. **Verify by digest.** Resolve tag→digest once, verify *that digest*.
2. **Run the verified digest.** Mutate the pod spec so the image reference is `app@sha256:9f2b...c4` — the exact digest verified. Now there is no second resolution: the kubelet pulls by digest, the registry is content-addressed, and the bytes it gets are the bytes the controller checked, or the pull fails.

This is why image-verification controllers register a **mutating** webhook and not only a validating one, and why the mutating phase running *before* validation matters: the controller pins the digest, then verifies the pinned digest, then the pod is stored with the digest. **policy-controller, Kyverno (mutateDigest: true), and Connaisseur all do this tag→digest mutation.** A validation-only integration that admits pods still bearing tags — the default shape of a plain Gatekeeper-plus-external-data setup that lacks a mutation step — leaves the TOCTOU gap wide open. When you evaluate a tool, "does it mutate the pod spec to the verified digest?" is a gating question, not a nice-to-have.

```
sequenceDiagram
    participant API as API server
    participant IVC as Image-verification controller
    participant Reg as Registry
    participant Kubelet as Node kubelet
    Note over API,Kubelet: WITHOUT digest pinning (vulnerable)
    API->>IVC: admit Pod(app:1.4.2)?
    IVC->>Reg: resolve app:1.4.2
    Reg-->>IVC: sha256:9f2b...c4
    IVC->>Reg: verify sig of sha256:9f2b...c4 ✓
    IVC-->>API: admit (spec still says app:1.4.2)
    Note over Reg: attacker re-points tag 1.4.2 → sha256:BADBAD
    Kubelet->>Reg: pull app:1.4.2
    Reg-->>Kubelet: sha256:BADBAD (NEVER VERIFIED)
    Note over API,Kubelet: WITH digest pinning (safe)
    API->>IVC: admit Pod(app:1.4.2)?
    IVC->>Reg: resolve → sha256:9f2b...c4, verify ✓
    IVC-->>API: mutate spec → app@sha256:9f2b...c4, admit
    Kubelet->>Reg: pull app@sha256:9f2b...c4
    Reg-->>Kubelet: exactly the verified bytes (or pull fails)
```

Deploy manifests should already pin digests — a paved-road GitOps pipeline (Book 6, Chapter 7) resolves tags to digests at render time, so the manifest that lands in git says `app@sha256:...`. The admission controller is the *enforcement backstop*: it guarantees the running digest

is a verified one **even when** a manifest slipped through with a bare tag, and it guarantees the tag was not moved between render and run. Belt and braces: pin in the pipeline, enforce at admission.

The tools

Several controllers implement admission-time image verification. They differ in policy model, whether they mutate to digest, and how they source verification results. The *policy-engine* comparison (OPA vs Kyverno vs CEL) is Book 6, Chapter 6; here we compare them **as image-verification controllers**.

Tool	Kind	Policy object	Keyless	Attestations	Mutating tag-
Sigstore policy-controller	Validating + mutating webhook	ClusterImagePolicy (CRD)	Yes (native)	Yes	Yes
Kyverno	Validating + mutating webhook	ClusterPolicy verifyImages	Yes	Yes	Yes (mutating)
Connaisseur	Mutating webhook	Helm values / CRD config	Yes	Notary v1/v2, cosign	Yes
Ratify + Gatekeeper	Gatekeeper (OPA) + external data	Rego Constraint + Ratify config	Yes	Yes	No (only)

Tool	Kind	Policy object	Keyless	Attestations	Mut- tag-
Google Binary Authorization	GKE control-plane admission	Cloud policy + attestors	Yes (Sigstore attestors)	Attestation-based	Man

Sigstore policy-controller and `ClusterImagePolicy`

policy-controller is the Sigstore project's admission controller. It enforces only in namespaces labeled `policy.sigstore.dev/include: "true"` (an explicit opt-in so you can roll it out namespace by namespace), and it resolves image references to digests via its mutating webhook. Policy lives in the `ClusterImagePolicy` CRD: match images by glob, require signatures from keyless identities (or keys), require attestations, and set a mode of `enforce` or `warn`.

Within one policy, `authorities` are **OR'd** — any one satisfied admits the image. Across multiple policies that match the same image, **all** must be satisfied (AND). That AND-across-policies rule is what makes tiering work: scope globs so that internal images match the strict policy and third-party images match a separate, relaxed one.

Here is a complete, strict policy for internal images: it requires a **keyless signature from a specific GitHub Actions build workflow**, an **SLSA provenance** attestation and an **SPDX SBOM** attestation from the same identity, with Rekor checked:

```

apiVersion: policy.sigstore.dev/v1beta1
kind: ClusterImagePolicy
metadata:
  name: internal-images-require-signed-provenance
spec:
  mode: enforce
  images:
    - glob: "registry.example.com/apps/**"
  authorities:
    - name: build-platform-keyless
      keyless:
        url: https://fulcio.sigstore.dev
      identities:
        - issuer: https://token.actions.githubusercontent.com
          subjectRegExp: "^https://github.com/example-org/[^/]+/\\
\\.github/workflows/release\\.yaml@refs/tags/v.*$"
      ctlog:
        url: https://rekor.sigstore.dev
      attestations:
        - name: must-have-slsa-provenance
          predicateType: slsaprovenance
          policy:
            type: cue
            data: |
              // provenance must name our builder and SLSA build level
              >= 3
              predicate: {
                builder: id: "https://github.com/example-org/build-
platform/.github/workflows/release.yaml@refs/heads/main"
              }
        - name: must-have-sbom
          predicateType: spdxjson

```

Notes on accuracy: `subjectRegExp/subject` and `issuer/issuerRegExp` are the real fields under `keyless.identities`; `predicateType` accepts the short names (`slsaprovenance`, `spdxjson`, `vuln`, `custom`) or a full predicate-type URI; the per-attestation `policy` block evaluates **CUE** or **Rego** over the predicate. `ctlog.url` is where Rekor inclusion is checked. For a private Sigstore stack, a `TrustRoot` resource supplies your own Fulcio/Rekor/CT roots and you drop the public `url` s.

For **third-party images** you cannot demand your build identity — nobody at Docker Hub signs with your workflow. A separate, relaxed policy handles them, either verifying the *vendor's* signature or (pragmatically, during rollout) allowing them via a `static` authority while a scan-only gate covers them elsewhere:

```
apiVersion: policy.sigstore.dev/v1beta1
kind: ClusterImagePolicy
metadata:
  name: third-party-images-allow
spec:
  mode: enforce
  images:
    - glob: "index.docker.io/**"
    - glob: "registry.k8s.io/**"
  authorities:
    - name: allow-external
      static:
        action: pass # do not require OUR identity on upstream
images; gate them via scanning instead
```

Kyverno `verifyImages`

Kyverno is a general Kubernetes policy engine (broad coverage in Book 6, Chapter 6); its image-verification feature is the `verifyImages` rule. It mutates the image to its digest by default (`mutateDigest: true`) and verifies the digest (`verifyDigest: true`). Attestors describe who must have signed; `attestations` describe required predicates with optional CEL/JMESPath conditions over their contents.

```

apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: verify-internal-images
spec:
  validationFailureAction: Enforce          # Audit during rollout, then
  Enforce
  webhookTimeoutSeconds: 30
  failurePolicy: Fail                      # fail-closed – see
operational section
  background: false
  rules:
    - name: require-keyless-signature-and-provenance
      match:
        any:
          - resources:
              kinds: [Pod]
            verifyImages:
              - imageReferences:
                  - "registry.example.com/apps/*"
                mutateDigest: true
# pin the verified digest into the pod spec
                verifyDigest: true
                required: true
                attestors:
                  - count: 1
                    entries:
                      - keyless:
                          subject: "https://github.com/example-org/
app/.github/workflows/release.yml@refs/tags/*"
                          issuer: "https://
token.actions.githubusercontent.com"
                        rekor:
                          url: https://rekor.sigstore.dev
                    attestations:
                      - type: https://slsa.dev/provenance/v1
                        attestors:
                          - entries:
                              - keyless:
                                  subject: "https://github.com/example-org/
app/.github/workflows/release.yml@refs/tags/*"
                                  issuer: "https://
token.actions.githubusercontent.com"
                                rekor:
                                  url: https://rekor.sigstore.dev
                              conditions:
                                - all:
                                    - key: "{{ buildDefinition.externalParameters.workf
low.repository || '' }}"

```

```
operator: Equals
value: "https://github.com/example-org/app"
```

The `attestations[].conditions` block is where you assert facts *about* the provenance — the source repo, the builder, the SLSA level — using Kyverno's expression engine over the predicate JSON. `attestors[].count` lets you require *m-of-n* signers. As with policy-controller, the subject pattern is the load-bearing part: it pins the *identity*, so an image signed by anyone else fails.

The corresponding verify command a developer would run locally to reproduce what the cluster does:

```
cosign verify \
  --certificate-identity-regex "^https://github.com/example-org/app/
  \.github/workflows/release\.yml@refs/tags/v.*$" \
  --certificate-oidc-issuer https://
  token.actions.githubusercontent.com \
  registry.example.com/apps/app@sha256:9f2b...c4

cosign verify-attestation --type slsaprovenance \
  --certificate-identity-regex "^https://github.com/example-org/
  app/.*$" \
  --certificate-oidc-issuer https://
  token.actions.githubusercontent.com \
  registry.example.com/apps/app@sha256:9f2b...c4
```

Connaisseur, Ratify + Gatekeeper, Binary Authorization

Connaisseur is a mutating admission controller focused squarely on image verification. It supports multiple validators (cosign/Sigstore, Notary v1 and v2) and **mutates image references to digests**, so it closes the TOCTOU gap by design. It is a good fit when you want a single-purpose, opinionated verifier rather than a general policy engine.

Ratify + Gatekeeper is a two-part design worth understanding because it exposes an architectural seam. **Gatekeeper** is the OPA-based validating admission controller; **Ratify** is a *verification engine* that plugs into Gatekeeper as an **external data provider**. When a pod is admitted, Gatekeeper's Rego constraint calls out to Ratify with the image references; Ratify pulls signatures/SBOMs/attestations, verifies them (cosign, Notary, license/vuln reports via verifier plugins), and returns structured results; the Rego constraint decides admit/deny on those results. The strength is composability — Ratify is a general artifact-

verification engine and OPA is a general policy engine, both reusable. The **weakness for our purposes**: Gatekeeper is a *validating* controller and does not mutate the pod spec, so a plain Ratify+Gatekeeper deployment **verifies a digest but can admit a pod still bearing a tag** unless you add a separate mutation mechanism. If you choose this stack, pair it with a digest-pinning mutator (in the pipeline and/or a mutating webhook); do not assume validation alone closed the gap.

Google Binary Authorization (GKE, with analogues on other clouds) moves the gate into the **managed control plane** rather than a webhook you operate. You define attestors — each backed by a KMS key or, increasingly, by Sigstore/cosign identities — and a policy that says "images admitted to this cluster must carry attestations from these attestors." The GKE admission controller enforces it, and **continuous validation** re-checks already-running pods and flags ones that fall out of policy (e.g., an attestation revoked after deploy). The trade-off is the usual managed one: less operational burden and no webhook of your own in the critical path, at the cost of cloud lock-in and a policy model narrower than Kyverno/OPA. For a GKE-only fleet it is often the pragmatic choice; for multi-cloud, a portable webhook (policy-controller/Kyverno) applied identically everywhere wins on consistency (Book 8, Chapter 4).

Operational realities: this webhook is tier-0

An image-verification webhook is not an ordinary integration. It sits in the admission path of **every pod in the cluster**. That fact drives every operational decision below.

Fail-open vs fail-closed

The `failurePolicy` decides what happens when the webhook cannot be reached or times out — Fulcio/Rekor roots unreachable, the controller pods down, a network partition, a 31-second registry call against a 30-second timeout:

- **failurePolicy: Ignore (fail-open)**. Admission proceeds *as if the check passed*. Availability is preserved; **security is not** — during any outage, unsigned and unverified images sail through, which is

precisely when a clever attacker would arrange for the webhook to be "busy" (Book 5, Chapters 8 and 10).

- **failurePolicy: Fail (fail-closed).** Admission is *denied* when the webhook is unreachable. Security is preserved; **availability is at risk** — and the risk is not subtle.

The fail-closed failure mode is genuinely dangerous and specific to this being tier-0: **if the verification webhook is down and set to fail-closed, no pod that matches its selector can be scheduled — including, potentially, the controller's own pods and the platform components needed to recover.** You can deadlock a cluster: the webhook is unhealthy, so it can't admit the replacement webhook pod, so it stays unhealthy. A crash loop in the controller becomes a fleet-wide inability to schedule anything.

```
flowchart TD
    DOWN["Verification webhook unavailable\n(crash, partition, Rekor/  
Fulcio unreachable, timeout)"]
    DOWN --> FP{"failurePolicy?"}
    FP -->|Ignore fail-open| OPEN["Pods admitted unchecked\n⚠ unsigned  
images can run"]
    FP -->|Fail fail-closed| CLOSED["Pod admission DENIED for matching  
namespaces"]
    CLOSED --> SCHED["No new pods schedule"]
    SCHED --> SELF{"Is the controller / kube-system\nexcluded from the  
webhook?"}
    SELF -->|No| DEAD["Deadlock: controller can't restart itself\n→ whole  
fleet frozen"]
    SELF -->|Yes, HA + exclusions| RECOV["Control plane & controller keep  
running\n→ self-heal, workloads gate correctly"]
```

The resolution is not to pick a side but to **engineer the fail-closed posture so its blast radius is bounded:**

- **Exclude the control plane and the controller itself.** `kube-system`, the controller's own namespace, and other bootstrap namespaces must be exempted via `namespaceSelector` / `objectSelector`, so a broken webhook can never block its own recovery or core components. `policy-controller`'s opt-in-by-label model gives this for free (only labeled namespaces are gated); with `Kyverno/Gatekeeper` you set the selectors explicitly.

- **Run the controller HA.** Multiple replicas across nodes/zones with a `PodDisruptionBudget`, so a single node or rollout can't take the webhook to zero.
- **Bound the timeout and the dependencies.** Keep `timeoutSeconds` modest, cache verification results (Kyverno's `useCache`), and consider the availability of Rekor/Fulcio — a public-Sigstore dependency on the hot path is an external SLA you don't control; a private/mirror deployment or aggressive caching de-risks it.

The honest recommendation: **fail-closed, with rigorous exclusions and HA**, is the target for production-tier namespaces, because fail-open verification is only slightly better than no verification (it's off exactly when you need it). But you reach fail-closed *after* the rollout below proves the policy is correct and the controller is reliable — flipping straight to fail-closed on day one is how you learn about a missing exclusion the hard way.

Bootstrapping and exclusions

You cannot require *your* build identity on images you didn't build. The cluster runs plenty:

- **System / infrastructure images** — `registry.k8s.io/*` (kube-proxy, CoreDNS, CSI drivers), the CNI, monitoring agents. Signed by *their* projects, not you.
- **The verification controller itself**, and the rest of the platform's bootstrap set.
- **Third-party application images** — databases, message brokers, vendor sidecars — signed (if at all) by *their* vendors.

The answer is **tiered policy** (Book 5, Chapter 10): different rules for different image classes, scoped by registry glob. Internal images (`registry.example.com/apps/**`) get the strict keyless-plus-provenance policy. Upstream images get a policy that verifies the *vendor's* signature where one exists (many CNCF projects now sign keyless; you can pin *their* GitHub identity the same way), or, where no signature exists, are gated by **scanning** instead (Book 6, Chapter 4) and exempted from signature requirements. System namespaces are excluded outright. The taxonomy is "trust boundary": *we built it* → demand our identity; *a vendor built it* → demand theirs or scan it; *the platform needs it to boot* → exempt.

Rollout: audit before enforce

Never flip an image-verification policy straight to enforce across a live fleet. The failure mode is not subtle — it is *every unsigned image in the cluster is suddenly unschedulable* — and the first thing you'll discover is how much of your fleet was never actually signed. The rollout mirrors Book 5, Chapter 10 and Book 1, Chapter 10:

1. **Audit / warn mode.** Deploy the policy in `mode: warn` (policy-controller) or `validationFailureAction: Audit` (Kyverno). It **logs would-be-denials and emits events but admits everything**. Run it long enough to see the true state: which images fail, which teams are unsigned, which upstream images lack policy. This is your coverage gap, made visible without an outage.
2. **Fix coverage.** Get the build platform signing every internal image (Book 4, Chapter 10); write the tiered upstream policy; add exclusions. Watch the would-be-denials drop toward zero.
3. **Enforce per namespace / per tier.** Move low-risk, well-covered namespaces to enforce first, then production. policy-controller's opt-in label makes this literally namespace-by-namespace. Keep `failurePolicy: Ignore` until the controller has proven reliable under enforce, *then* move the gated namespaces to `Fail`.
4. **Only then, fleet-wide fail-closed** for tier-0/tier-1 namespaces, with the exclusions and HA above.

Audit mode is not a formality; it is the difference between "we turned on verification" and "we took down every deploy in the company at 2pm."

Performance and availability at scale

Every pod creation — every deploy, every scale-up, every node drain that reschedules pods, every CronJob firing — calls this webhook. At fleet scale that is a high, bursty request rate against a service that does registry round-trips and cryptographic verification. Consequences to design for:

- **Cache aggressively.** The same digest is admitted thousands of times; verify it once and cache the result (with a bounded TTL and cache-key that includes the *digest*, so a moved tag can't serve a stale allow). Kyverno's image-verification cache and policy-controller's in-memory caching exist for exactly this.

- **Keep verification off slow external dependencies on the hot path** where possible — mirror or cache Fulcio/Rekor roots; the roots change rarely and don't need a live fetch per admission.
 - **Budget the timeout against the failurePolicy.** A 30-second timeout with fail-closed means a slow registry can start denying pods; a too-short timeout with a genuinely slow verify means spurious denials. Tune with the cache in mind.
 - **Scale the controller with the fleet.** It is a tier-0 service; give it the SLO, the alerting, and the capacity of one. A p99 admission-latency alert on this webhook is not optional.
-

Distributed-systems lens

Admission-based image verification is the **fleet-wide, non-bypassable enforcement point** the whole supply chain has been building toward (Book 5, Chapter 10), realized at the Kubernetes boundary. A few properties make it the right place for the gate:

- **One policy, all clusters, no bypass.** Unlike a CI check (skippable, per-repo, per-team), the API server *always* runs admission. A single `ClusterImagePolicy` or Kyverno policy, distributed identically to every cluster, guarantees that no service anywhere in the fleet runs an unsigned or unverified image — regardless of which team shipped it, which pipeline built it, or whether they meant to comply. Enforcement lives below the layer where humans make mistakes.
- **It is tier-0, and you must treat it as such.** Being in every pod's critical path is exactly what gives it its power and exactly what makes it dangerous: HA, careful `failurePolicy`, and ruthless exclusions of the control plane are the price of admission (literally). A gate this strong that is engineered like an ordinary microservice will one day freeze your fleet.
- **The trust anchor is the build platform's identity.** The policy doesn't trust images; it trusts a *signer* — the CI workflow's keyless identity (Book 4, Chapter 10; Book 5, Chapter 4). This is why paved-road, automatic, per-build signing is a precondition, not a nicety: the cluster can only demand an identity that the build reliably produces.
- **Verify-and-run-by-digest closes the TOCTOU gap fleet-wide.** The mutation-to-digest step turns every admitted pod's mutable tag

into the immutable bytes that were verified (Book 6, Chapter 1), across the whole fleet at once. A tool that verifies but doesn't pin leaves the gap open on every cluster.

- **Tiered policy models the real world.** Fleets run a mix of first-party (strict: our identity + provenance) and third-party (relaxed: vendor signature or scan-only) images; the tiering (Book 5, Chapter 10) is what lets a single non-bypassable gate coexist with images you didn't build.
- **The loop closes here.** The controller *consumes* exactly the signatures and attestations the paved-road build *produced* (Book 4, Chapter 10) — produce-and-verify, closed at the cluster boundary. Everything upstream is inert until this check runs, and this check is inert unless the upstream produced what it demands. The two halves only have value together.
- **Multi-cluster policy distribution is its own problem.** Getting the *same* policy to hundreds of clusters, versioned and drift-free, is a GitOps/governance concern (Book 6, Chapter 7; Book 8, Chapter 4). A gate is only fleet-wide if its policy is; a cluster running last quarter's policy is a hole in the fleet.

The one-line synthesis: **signing is a claim, admission is the enforcement, and the digest is what ties the claim to the bytes that run.** Book 5 taught you to make the claim; this chapter is how a Kubernetes fleet refuses to run anything that can't back it up.

Key takeaways

- **Cosign signs an image by its manifest digest**, not its tag, and stores the signature (and SBOM/SLSA-provenance/scan/VEX attestations) as OCI artifacts referencing that digest — via OCI 1.1 referrers or the legacy `.sig / .att` tag scheme (Book 6, Chapter 1). For CI images, **keyless signing from the build's workload identity** (Book 5, Chapter 4) is the norm; the certificate's SAN is the build identity the cluster will later demand.
- **Signing without verification is theatre.** The enforcement point in Kubernetes is **admission control**: an image-verification controller registers validating and mutating webhooks; on pod create it resolves the image to a digest, verifies signature + expected signer

identity + required attestations + Rekor inclusion, and admits or denies.

- **Verify-and-run the same digest.** The controller must resolve tag→digest *once*, verify that digest, and **mutate the pod spec to that digest** so the kubelet runs the exact verified bytes. A controller that verifies a tag but doesn't pin it leaves the Book 6, Chapter 1 **TOCTOU** gap open — an attacker moves the tag between check and pull. policy-controller, Kyverno (`mutateDigest`), and Connaisseur pin; **Ratify+Gatekeeper (validation-only) does not** — pair it with a mutator.
- **The webhook is tier-0.** It is in every pod's admission path. `failurePolicy: Ignore` is fail-open (unsigned images run during any outage); `failurePolicy: Fail` is fail-closed (a down webhook can freeze all scheduling and even deadlock its own recovery). Target **fail-closed with rigorous control-plane/self exclusions, HA, PDBs, caching, and bounded timeouts.**
- **Tier the policy.** Strict keyless-plus-provenance for internal images (`registry.example.com/**` matched to your build workflow's identity); relaxed policy for third-party images (vendor signature or scan-only — you can't demand your identity on upstream); exempt system namespaces.
- **Roll out audit-first.** Deploy in `warn / Audit` to surface the real coverage gap without an outage, fix signing coverage, then enforce per namespace/tier, then flip gated namespaces to fail-closed. Never flip straight to enforce fleet-wide.
- **Distributed-systems payoff:** one policy across all clusters gives non-bypassable, fleet-wide guarantees that nothing unsigned/unverified runs; the trust anchor is the build platform's identity; verify-and-run-by-digest closes TOCTOU everywhere at once; and the controller consuming what the paved-road build produced closes the produce-and-verify loop at the cluster boundary.

Further reading

- **Sigstore policy-controller** — the `ClusterImagePolicy` CRD, keyless/key authorities, attestation policies (`CUE/Rego`), `TrustRoot`, and

tag→digest resolution. <https://docs.sigstore.dev/policy-controller/overview/>

- **Kyverno — Verify Images** — `verifyImages` rules, `mutateDigest` / `verifyDigest`, attestors and attestations, keyless and key-based verification. <https://kyverno.io/docs/writing-policies/verify-images/>
- **cosign** — `sign`, `attest`, `verify`, `verify-attestation`, `--certificate-identity` / `--certificate-oidc-issuer`, referrers vs tag storage. <https://docs.sigstore.dev/cosign/>
- **Kubernetes — Dynamic Admission Control** — mutating/validating webhooks, `failurePolicy`, selectors, timeouts, ordering. <https://kubernetes.io/docs/reference/access-authn-authz/extensible-admission-controllers/>
- **Ratify** — artifact verification as a Gatekeeper external-data provider; verifiers for cosign, Notary, SBOM/scan. <https://ratify.dev/docs/>
- **OPA Gatekeeper** — the OPA-based validating admission controller and external data. <https://open-policy-agent.github.io/gatekeeper/website/docs/>
- **Connaisseur** — admission controller for image-signature verification (cosign, Notary), with digest pinning. <https://sse-secure-systems.github.io/connaisseur/>
- **Google Binary Authorization** — attestation-based admission and continuous validation on GKE, including Sigstore attestors. <https://cloud.google.com/binary-authorization/docs>
- **OCI Distribution Specification v1.1 — Referrers API** — how signatures/attestations attach to an image by digest (Book 6, Chapter 1). <https://github.com/opencontainers/distribution-spec>
- **SLSA v1.0 — Provenance** — the provenance predicate an admission policy inspects (Book 4, Chapter 3). <https://slsa.dev/spec/v1.0/provenance>

Chapter 6 — Admission Control and Policy Engines: OPA and Kyverno

What this chapter covers. The previous chapter pointed one specific mechanism — Kubernetes admission control — at one specific problem —

image signature verification (Book 6, Chapter 5 — Image Signing and Verification in Kubernetes). This chapter zooms back out and treats admission control *as a general enforcement substrate*: the last-chance-before-persistence hook where the API server hands every proposed object to your code and asks "should this be allowed, and does it need changing first?" It is the machinery behind image verification, but also behind approved-registry enforcement (Book 6, Chapter 2 — Registries), base-image policy (Book 6, Chapter 3 — Base Images), digest pinning (Book 6, Chapter 1 — Container Images), workload hardening, required ownership metadata, and the deployment gates of Book 5, Chapter 10 — Attestation-Based Deployment Gates. We cover the admission request lifecycle in precise detail, then the three policy engines that matter in 2026 — **OPA/Gatekeeper** (Rego), **Kyverno** (YAML), and in-tree **ValidatingAdmissionPolicy** (CEL) — with real, working policy for each; the catalogue of supply-chain policies worth enforcing; and the hard operational reality that a webhook-based policy engine sits in the critical path of *every write to the cluster* and can, if you get it wrong, wedge the whole control plane.

Learning goals — after this chapter you should be able to:

- Trace a write request through the API server — **authentication** → **authorization** → **mutating admission** → **schema validation** → **validating admission** → **etcd** — and say exactly why admission is the policy enforcement point.
- Explain **mutating vs validating admission webhooks**: what each may do, why mutating runs first, and the meaning of `failurePolicy`, `timeoutSeconds`, `namespaceSelector/objectSelector`, `matchPolicy`, and `reinvocationPolicy`.
- Write and read policy in **all three engines**: a Gatekeeper `ConstraintTemplate` + `Constraint` in Rego, a Kyverno `ClusterPolicy` in YAML, and a `ValidatingAdmissionPolicy` in CEL — and choose the right one for a given rule.
- Enumerate the **supply-chain policies** worth enforcing at admission and map each to the book that produced the evidence it consumes.
- Run a **safe rollout**: audit/warn before enforce, `failurePolicy` and HA tuning, control-plane exclusions, policy exceptions with expiry, CI testing of policy, and GitOps distribution across a fleet.

- Reason about the engine as a **tier-0, in-critical-path dependency** and why in-tree VAP changes that risk calculus.

Where admission sits: the API request lifecycle

Every mutation to cluster state — `kubectl apply`, a controller reconciling, a CI deploy — is an HTTP request to the kube-apiserver. Before those bytes become a row in etcd, the request passes through a fixed pipeline. Admission is the last stage, and its position is the whole point.

```
flowchart LR
    C["Client  
kubectl / controller / CI"] --> A["Authentication  
who are you?"]
    A --> Z["Authorization / RBAC  
may you do this verb  
on this resource?"]
    Z --> M["Mutating admission  
webhooks + policies  
may change the object"]
    M --> S["Schema validation  
+ defaulting  
OpenAPI / structural"]
    S --> V["Validating admission  
webhooks + policies  
accept or reject"]
    V --> E["Persist to etcd"]
    M -. reject .-> X1["4xx to client"]
    V -. reject .-> X2["X"]
```

Read the ordering carefully, because policy correctness depends on it:

1. **Authentication** establishes *who* the caller is — a user, a group, a ServiceAccount. It does not look at the object.
2. **Authorization (RBAC, and Node/ABAC/webhook authorizers)** decides whether that identity may perform this *verb* (`create` , `update` , `delete` , `patch`) on this *resource kind* in this *namespace*. RBAC is coarse: it reasons about verbs and resources, never about the *contents* of the object. RBAC can say "this ServiceAccount may create Pods." It cannot say "...but only Pods whose images come from `registry.example.com` and run as non-root." That gap is exactly what admission fills.

3. **Mutating admission** runs. Registered mutating webhooks and (newer) mutating policies may *change* the incoming object — inject a sidecar, set a default, pin a tag to a digest (Book 6, Chapter 5), add a label.
4. **Object schema validation and defaulting** run *after* mutation. The (possibly mutated) object is validated against the resource's OpenAPI/structural schema and API defaults are applied. This is why a mutating webhook must emit a patch that still validates — you cannot mutate an object into something the schema rejects.
5. **Validating admission** runs last. Validating webhooks and validating policies get the final, mutated, schema-valid object and may only say **yes** or **no** (optionally with warnings). They cannot change it.
6. If nothing rejected it, the object is **persisted to etcd** and becomes real.

Admission is the enforcement point because it is the **last checkpoint before the object exists**, and the first checkpoint that can reason about the object's *content* rather than merely the caller's identity. Authentication and authorization gate the *actor*; admission gates the *artifact*. A policy such as "no image may run unless it is signed by our build identity" is a statement about the object's contents and is therefore, structurally, an admission concern. There is nowhere else in the pipeline it can live.

One subtlety that trips people up: admission fires on the API path, not the kubelet path. If an object already in etcd violates a newly added policy, admission never re-examines it — admission only sees *writes*. Catching pre-existing violations is the job of **audit/background scanning** (discussed below), not the admission webhook.

Built-in admission controllers and Pod Security Admission

Before any of your webhooks run, a chain of **compiled-in admission controllers** executes inside the API server: `NamespaceLifecycle`, `LimitRanger`, `ServiceAccount`, `ResourceQuota`, `DefaultStorageClass`, `MutatingAdmissionWebhook`, `ValidatingAdmissionWebhook`, and a few dozen others, enabled via `--enable-admission-plugins`. Two of these plugins — `MutatingAdmissionWebhook` and `ValidatingAdmissionWebhook` — are the dispatchers that call *out* to your webhooks; the rest are in-process logic Kubernetes ships itself.

The most relevant built-in for supply-chain hardening is **Pod Security Admission (PSA)**, the in-tree successor to the removed **PodSecurityPolicy (PSP)**. PSP was deprecated in 1.21 and **removed entirely in Kubernetes 1.25** — if a document tells you to write a PSP, it is describing a mechanism that no longer exists. PSA replaced it with something deliberately simpler: a built-in validating controller that enforces the three **Pod Security Standards** — `privileged` (no restrictions), `baseline` (blocks known privilege escalations — `hostNetwork`, `hostPID`, `privileged containers`, `most hostPath`), and `restricted` (hardened — must run as non-root, drop all capabilities, `seccomp RuntimeDefault`, no privilege escalation). You opt a namespace into a level and mode with labels:

```
apiVersion: v1
kind: Namespace
metadata:
  name: payments
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/enforce-version: latest
    pod-security.kubernetes.io/warn: restricted
    pod-security.kubernetes.io/audit: restricted
```

PSA is fixed-function: three levels, no custom rules, and it only evaluates Pods. It is the right baseline for workload hardening and costs you no external webhook (it runs in-process, so none of the availability risk discussed later). But the moment you need "...and images must come from an approved registry, and must be signed, and must carry an owner label," you have left PSA's remit and need a general policy engine. That is the rest of this chapter.

Admission webhooks: the mechanism the engines are built on

OPA/Gatekeeper and Kyverno are both, underneath, **admission webhooks**. Understanding the webhook contract is understanding both engines' failure modes, so we spend a section on it.

A webhook is registered with a `MutatingWebhookConfiguration` or `ValidatingWebhookConfiguration` object. The registration tells the API server *when* to call the webhook and *how*:

```

apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingWebhookConfiguration
metadata:
  name: example-policy.example.com
webhooks:
- name: validate.example.com
  admissionReviewVersions: ["v1"]
  sideEffects: None
  clientConfig:
    service:
      name: policy-webhook
      namespace: policy-system
      path: /validate
    caBundle: <base64 CA cert>
  rules:
  - apiGroups: [""]
    apiVersions: ["v1"]
    operations: ["CREATE", "UPDATE"]
    resources: ["pods"]
    scope: "Namespaced"
  failurePolicy: Fail
  matchPolicy: Equivalent
  timeoutSeconds: 5
  namespaceSelector:
    matchExpressions:
    - key: kubernetes.io/metadata.name
      operator: NotIn
      values: ["kube-system", "kube-node-lease"]
  objectSelector: {}

```

When a matching request arrives, the API server serializes the object into an `AdmissionReview` and POSTs it to the webhook's `clientConfig` endpoint. The request half carries `request.uid`, the `object` (and `oldObject` on updates), the `operation`, `userInfo`, and `dryRun`. The webhook must reply with an `AdmissionReview` whose `response` echoes the same `uid` and carries `allowed: true|false`, an optional `status` message, optional `warnings`, and — for mutating webhooks — a base64 `patch` of type `JSONPatch`.

```
{
  "apiVersion": "admission.k8s.io/v1",
  "kind": "AdmissionReview",
  "response": {
    "uid": "705ab4f5-6393-11e8-b7cc-42010a800002",
    "allowed": false,
    "status": {
      "code": 403,
      "message": "image registry docker.io is not in the approved list"
    }
  }
}
```

The registration fields are the levers you tune, and each one is a security-vs-availability decision:

- **failurePolicy: Fail | Ignore** — what happens when the webhook *does not answer* (times out, returns 5xx, is unreachable, has no endpoints). **Fail = fail-closed**: the request is rejected, so nothing unpoliced gets in — but a dead webhook now blocks writes. **Ignore = fail-open**: the request is admitted unchecked — the cluster keeps working but your policy silently stops enforcing. This is the single most consequential setting in the chapter and we return to it under operational realities. It is the same availability-vs-security lever discussed in Book 5, Chapter 8 — Verification in Practice and Book 5, Chapter 10.
- **timeoutSeconds** (1-30, default 10) — how long the API server waits before applying **failurePolicy**. Every matching write pays up to this latency. Keep it small (1-5s) for anything in the hot path.
- **namespaceSelector / objectSelector** — scoping. The example above **excludes kube-system**, which is not optional hygiene — it is how you avoid the webhook policing (and potentially blocking) the very control-plane pods that keep it alive.
- **matchPolicy: Exact | Equivalent** — **Equivalent** (recommended) means the webhook is also called for requests that arrive via a different API version that maps to the same resource, so a policy written for **apps/v1 Deployments** isn't silently bypassed by a client using another version.
- **reinvocationPolicy: Never | IfNeeded** (mutating only) — if an *earlier* mutating webhook's patch changes the object after *this* webhook already ran, **IfNeeded** calls this webhook again so it can

react. Order among mutating webhooks is not guaranteed, so a webhook that must see the final state (e.g., inject a sidecar only if no sidecar is present) sets `IfNeeded`. Note the API server caps reinvocation at a bounded number of passes and does not guarantee convergence — order-independent mutations are still the safer design.

- **`sideEffects: None | NoneOnDryRun`** — declares whether the webhook mutates external state. It must be `None / NoneOnDryRun` for the webhook to be called on `dryRun` requests. Policy engines are pure evaluators and should always be `None`.

The mutating dispatcher runs *before* the validating dispatcher (matching the pipeline diagram), so a mutating policy can, for example, pin a tag to a digest and a validating policy can then require that all images be digest-pinned — the mutation makes the validation pass. Order matters and the engines exploit it.

OPA and Gatekeeper

Open Policy Agent (OPA) is a general-purpose, CNCF-graduated policy engine. It is not Kubernetes-specific: OPA evaluates a policy written in **Rego** against arbitrary JSON input and returns a decision. People use OPA for API authorization, Terraform plan checking, and CI gates as well as Kubernetes. **Gatekeeper** is the project that packages OPA as a Kubernetes validating (and, more recently, mutating) admission webhook and gives it a native CRD-driven policy model.

Rego in one section

Rego is a declarative, query-based language descended from Datalog. You do not write "if/else that returns allow"; you write **rules that define values**, and the absence of a violation *is* the allow. The mental model that matters: a Gatekeeper policy accumulates entries in a set called `violation`; if the set is empty, the object is admitted; if it contains messages, the object is rejected with those messages.

```

package k8sallowedrepos

violation[{"msg": msg}] {
  container := input.review.object.spec.containers[_]
  not startswith_any(container.image, input.parameters.repos)
  msg := sprintf("container <%v> image %v is not from an approved
registry %v",
    [container.name, container.image, input.parameters.repos])
}

startswith_any(str, prefixes) {
  startswith(str, prefixes[_])
}

```

Reading it: `input.review.object` is the admitted Kubernetes object (Gatekeeper wraps the `AdmissionReview` under `input.review`). `input.parameters` are values supplied by the *Constraint* (below). `container := input.review.object.spec.containers[_]` iterates every container — the `[_]` is Rego's "for all elements" iteration. The rule body is an implicit AND; it *fires* (adds a violation) when a container's image does **not** start with any approved prefix. Empty violation set $\Rightarrow$ admit. This is Rego's power and its notorious learning curve in one example: it is genuinely expressive — you can express constraints a YAML matcher cannot — but "define the negative space by accumulating a set" is a real cognitive shift for engineers used to imperative validation, and Rego debugging (`opa eval`, `opa test`, the trace explorer) is a skill of its own.

ConstraintTemplate + Constraint

Gatekeeper splits policy into two objects so that platform teams write Rego once and application teams (or the platform team, per environment) instantiate it many times with different parameters.

The **ConstraintTemplate** carries the Rego and *defines a new CRD* — the constraint kind:

```

apiVersion: templates.gatekeeper.sh/v1
kind: ConstraintTemplate
metadata:
  name: k8sallowedrepos
spec:
  crd:
    spec:
      names:
        kind: K8sAllowedRepos
      validation:
        openAPIV3Schema:
          type: object
          properties:
            repos:
              type: array
              items:
                type: string
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package k8sallowedrepos
        violation[{"msg": msg}] {
          container := input.review.object.spec.containers[_]
          not startswith_any(container.image, input.parameters.repos)
          msg := sprintf("image %v not from approved registry",
            [container.image])
        }
        startswith_any(str, prefixes) { startswith(str, prefixes[_]) }

```

The **Constraint** is an instance of that generated `K8sAllowedRepos` kind. It supplies parameters, scopes to resource kinds via `match`, and — critically — sets `enforcementAction`:

```

apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sAllowedRepos
metadata:
  name: prod-approved-registries
spec:
  enforcementAction: deny          # or: warn | dryrun
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
        namespaces: ["payments", "checkout"]
  parameters:
    repos:
      - "registry.example.com/"
      - "ghcr.io/example-org/"

```

```

flowchart TB
    subgraph authoring["Platform team authors"]
        CT["ConstraintTemplate  
Rego + CRD schema"]
        CO["Constraint  
params + match + action"]
    end
    CT -->|generates CRD| CO
    API["kube-apiserver"] -->|AdmissionReview| GK["Gatekeeper  
validating webhook"]
    CT --> GK
    CO --> GK
    GK -->|OPA evaluates Rego  
violation set empty?| DEC{"admit / deny / warn"}
    DEC --> API
    GK -->|periodic| AUD["Audit  
re-eval etcd objects  
write violations to status"]

```

`enforcementAction: dryrun` and `warn` are how you roll out without breaking anyone — `dryrun` records violations in the constraint's `status` and Gatekeeper's audit results but admits everything; `warn` returns an admission warning to the user but still admits; `deny` rejects. The progression `dryrun` → `warn` → `deny` is the safe path and we return to it.

Gatekeeper's other capabilities

- **Audit** — every `constraint_violations` interval (default 60s), Gatekeeper re-evaluates *existing* objects in etcd against all constraints and writes violations to each constraint's `status.violations`. This is how you find pre-existing offenders that admission never saw, and how you measure coverage before flipping to `deny`.
- **Mutation** — Gatekeeper added mutation via separate CRDs: `Assign`, `AssignMetadata` (labels/annotations only), and `ModifySet`. These are declarative field-setters, not Rego — deliberately weaker than validation, because a Turing-complete mutator is a footgun. To *verify image signatures*, Gatekeeper is typically paired with **Ratify** as an **external data provider** (Book 6, Chapter 5) — Rego calls out to Ratify, which does the cosign/Notary verification and returns a verdict the policy consumes.
- **External data** — the `external_data` builtin lets Rego call a provider mid-evaluation (registry lookups, signature verification, CMDB

checks). Powerful, but it puts a *second* network call in the admission hot path; budget the latency.

Gatekeeper's honest trade-off: maximum expressiveness and a mature ecosystem (a large community constraint-template library exists), at the cost of Rego. For a platform team that already runs OPA for other decision points, the leverage is real. For a team whose only policy engine is Kubernetes admission, Rego is often more language than the problem needs — which is precisely the gap Kyverno set out to fill.

Kyverno

Kyverno is a CNCF-incubating policy engine designed from the start for Kubernetes and *only* Kubernetes. Its central bet: policy is expressed as **Kubernetes YAML** — no new language — using resource-shaped patterns and overlays that will feel familiar to anyone who reads manifests. It runs as an admission webhook (both mutating and validating configurations) plus a background controller.

A Kyverno policy is a `ClusterPolicy` (cluster-scoped) or `Policy` (namespaced) containing one or more **rules**. Each rule has a `match` block and exactly one action: `validate`, `mutate`, `generate`, or `verifyImages`.

```
flowchart TB
    API["API[kube-apiserver]"] -->|AdmissionReview| KV["Kyverno webhooks  
mutating + validating"]
    KV --> CP["ClusterPolicy / Policy  
rules: match + action"]
    CP --> KV
    KV --> A1["validate  
pattern / deny → accept-reject"]
    KV --> A2["mutate  
patchStrategicMerge / patchesJson6902"]
    KV --> A3["verifyImages  
cosign / notary → mutateDigest"]
    KV --> A4["generate  
create/sync downstream objects"]
    KV -->|background scan| PR["PolicyReport /  
ClusterPolicyReport"]
```

validate

The workhorse. A `validate` rule either matches the object against a **pattern** (an overlay where `?/*` wildcards, `+/= /X()` anchors, and

operators like `>`, `!`, `|` express constraints) or uses `deny` with `conditions` for explicit boolean logic.

Require every container to run as non-root and drop all capabilities:

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-non-root-drop-caps
spec:
  validationFailureAction: Enforce      # or: Audit
  background: true
  rules:
    - name: containers-non-root
      match:
        any:
          - resources:
              kinds: ["Pod"]
      validate:
        message: "containers must run as non-root and drop ALL capabilities"
        pattern:
          spec:
            =(securityContext):
              runAsNonRoot: true
            containers:
              - name: "*"
                securityContext:
                  allowPrivilegeEscalation: false
                  capabilities:
                    drop: ["ALL"]
```

Restrict registries using `deny` with a JMESPath condition — semantically the Rego example from earlier, in YAML:

```

- name: approved-registries
  match:
    any:
      - resources:
          kinds: ["Pod"]
  validate:
    message: "images must come from registry.example.com or ghcr.io
/example-org"
  foreach:
    - list: "request.object.spec.containers"
      deny:
        conditions:
          any:
            - key: "{{ regex_match('^(registry.example.com|
ghcr.io/example-org)/.*', '{{ element.image }}') }}"
              operator: Equals
              value: false

```

`validationFailureAction: Audit vs Enforce` is Kyverno's audit-vs-block lever — the exact analogue of Gatekeeper's `dryrun/deny` and, as of newer Kyverno, this field is also expressible per-rule. `Audit` records results in a `PolicyReport` without blocking; `Enforce` rejects.

mutate

Set a default, inject a label, add a sidecar. Mutations use strategic-merge overlays or JSON6902 patches. Adding an owner label if missing:

```

- name: default-owner-label
  match:
    any:
      - resources:
          kinds: ["Pod"]
  mutate:
    patchStrategicMerge:
      metadata:
        labels:
          +(app.example.com/owner): "unassigned"
# +() = add only if absent

```

generate

Kyverno can *create* dependent objects when a trigger appears — the classic being: when a namespace is created, generate its default `NetworkPolicy`, `ResourceQuota`, and the `imagePullSecret` needed to pull from your registry. `generate` with `synchronize: true` keeps the

generated object in sync and recreates it if deleted. No other admission engine does this; it is a genuine Kyverno differentiator.

verifyImages

Covered in depth in Book 6, Chapter 5 — this is Kyverno's image-signature verification action. It resolves the tag, verifies the cosign signature and expected identity (and required attestations), and — importantly — **mutates the pod spec to the verified digest** (mutateDigest: true, default), closing the Book 6, Chapter 1 TOCTOU gap:

```
- name: verify-signed-by-our-ci
  match:
    any:
      - resources:
          kinds: ["Pod"]
  verifyImages:
    - imageReferences: ["registry.example.com/*"]
      mutateDigest: true
      attestors:
        - entries:
            - keyless:
                subject: "https://github.com/example-org/*.github/workflows/release.yml@refs/tags/*"
                issuer: "https://token.actions.githubusercontent.com"
                rekor:
                  url: "https://rekor.sigstore.dev"
```

Background scanning and policy reports

Kyverno's background controller periodically re-evaluates existing resources against background: true policies and emits **PolicyReport** (namespaced) and **ClusterPolicyReport** objects — the same role as Gatekeeper audit. `kubectl get policyreport -A` gives you fleet-wide pass/fail per policy per resource, which is the data you drive the rollout with. (Note: mutate, generate, and verifyImages rules can't run in background the way validate can, since they depend on the admission request; only validation-style checks fully populate reports.)

Why teams pick Kyverno

The pitch is simple and mostly true: policy authors already know YAML, patterns look like the manifests they constrain, and the four verbs

(`validate/mutate/generate/verifyImages`) cover the vast majority of Kubernetes policy needs without a second language. The cost is expressiveness at the edges — genuinely novel logic that a pattern can't express pushes you into JMESPath conditions and CEL expressions (Kyverno added CEL support in `validate.cel`), at which point the "no new language" advantage narrows. For most platform teams that trade is worth it, which is why Kyverno adoption has grown fast. But "no new language" becomes "well, some CEL" at the hard cases — know that going in.

Kubernetes-native: ValidatingAdmissionPolicy (VAP)

Both engines above share one structural liability: they are **external webhooks**. The API server makes a network call, out of process, for every matching request. That call can be slow, can fail, and — depending on `failurePolicy` — can take the cluster down with it. Kubernetes' answer is to move policy evaluation *back inside the API server*.

ValidatingAdmissionPolicy (VAP) is an in-tree admission mechanism that evaluates policy written in **CEL (Common Expression Language)** directly within the API server — **no webhook, no external process, no network hop**. It graduated to **beta in Kubernetes 1.28** and to **GA (admissionregistration.k8s.io/v1)** in **Kubernetes 1.30**. CEL is a small, non-Turing-complete expression language (the same one used in CRD validation rules and in Kyverno's `validate.cel`); every expression is guaranteed to terminate, which is precisely why it is safe to run inside the API server's request path.

VAP splits into two objects, mirroring Gatekeeper's template/constraint separation:

- **ValidatingAdmissionPolicy** — the CEL logic and (optional) parameter schema.
- **ValidatingAdmissionPolicyBinding** — binds the policy to resources/namespaces, supplies params, and sets `validationActions`.

```

apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicy
metadata:
  name: "approved-registries.example.com"
spec:
  failurePolicy: Fail
  matchConstraints:
    resourceRules:
      - apiGroups:  [""]
        apiVersions: ["v1"]
        operations: ["CREATE", "UPDATE"]
        resources:  ["pods"]
  variables:
    - name: prefixes
      expression: "['registry.example.com/', 'ghcr.io/example-org/']"
  validations:
    - expression: >-
      object.spec.containers.all(c,
        variables.prefixes.exists(p, c.image.startsWith(p)))
      message: "all images must come from an approved registry"
      reason: Forbidden
---
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicyBinding
metadata:
  name: "approved-registries-binding"
spec:
  policyName: "approved-registries.example.com"
  validationActions: ["Deny"]           # any of: Deny, Warn, Audit
  matchResources:
    namespaceSelector:
      matchExpressions:
        - key: kubernetes.io/metadata.name
          operator: NotIn
          values: ["kube-system"]

```

The CEL reads almost like the intent: "every container's image starts with one of the approved prefixes." The variables `object`, `oldObject`, `request`, `params`, `namespaceObject`, and `authorizer` are available to expressions; `object.spec.containers.all(...)` and `.exists(...)` are CEL macros. `validationActions` is the rollout lever, and note you can set **multiple** — `["Audit", "Warn"]` during rollout, then `["Deny"]` — and even list `["Deny", "Audit"]` to both block and record.

```

flowchart LR
    subgraph webhook["Webhook engines - out of process"]
        API1["kube-apiserver"] -->|network call| ENG["Gatekeeper / Kyverno pod"]
        ENG -->|down + failurePolicy Fail| BLK["writes blocked cluster-wide"]
    end
    subgraph native["VAP - in process"]
        API2["kube-apiserver evaluates CEL inline"] --> OK["no network hop no external dependency"]
    end

```

Why in-tree matters: for any policy VAP can express, there is *no webhook to be down*, no extra latency beyond a bounded in-process CEL evaluation, and no `failurePolicy: Fail` scenario that wedges the control plane. Policies that were previously the scariest to run fail-closed become safe to run fail-closed. This is a real reduction in tier-0 risk and the reason VAP is the emerging default for the *expressible* subset of policy.

The catch is that CEL is deliberately limited. It cannot call out to a registry, verify a Sigstore signature, or consult external data — those require I/O, which VAP forbids by design. So signature/attestation verification (Book 6, Chapter 5) and anything needing a network lookup **stay on webhooks** (Kyverno `verifyImages`, policy-controller, Ratify+Gatekeeper). VAP handles the large class of *structural* policy — approved registries, required labels, non-root, no `:latest`, replica limits — and takes it off the webhook, shrinking the webhook's blast radius to only the policies that genuinely need out-of-process evaluation.

A **MutatingAdmissionPolicy** (in-tree, CEL-based, using `ApplyConfiguration/JSONPatch`) is the mutating counterpart. It is newer — introduced alpha in Kubernetes 1.32 and promoted to beta in 1.34 — so treat it as emerging rather than settled, and check your cluster version before depending on it; the validating VAP is the GA, production-ready piece today.

Choosing an engine

	OPA / Gatekeeper	Kyverno	ValidatingAdmissionPolicy (VAP)
Language	Rego	Kubernetes YAML (+ some CEL/JMESPath)	CEL
Runs as	External webhook	External webhook	In-tree (API server)
CNCF status	Graduated (OPA)	Incubating	Kubernetes core (GA 1.30)
Scope	K8s + anything (general engine)	Kubernetes only	Kubernetes only
Actions	validate; mutate (Assign/ ModifySet); external data	validate, mutate, generate, verifyImages	validate (mutate via MAP, newer)
Image signature verify	via Ratify external- data	native verifyImages	no (needs I/O)
Availability risk	Webhook in critical path	Webhook in critical path	None external
Audit / background	Audit controller	Background scan + PolicyReports	validationActions: Audit
Learning curve	High (Rego)	Low-moderate (YAML)	Moderate (CEL)

	OPA / Gatekeeper	Kyverno	ValidatingAdmissionPolicy (VAP)
Best for	Orgs already running OPA; maximally expressive policy; cross-domain	Most Kubernetes teams; image verification; generate/mutate	Structural policy you want off the webhook and safely fail-closed

The pragmatic 2026 pattern is not "pick one" but **layer them**:

1. **Pod Security Admission** for the workload-hardening baseline (in-tree, free, no webhook).
2. **VAP** for every *structural* policy it can express — approved registries, required labels, no `:latest`, resource limits — because moving these in-tree removes them from the webhook's blast radius entirely.
3. **A webhook engine (Kyverno for most, Gatekeeper for OPA shops)** for what VAP cannot do: **image signature/attestation verification**, `generate`, complex external-data lookups.

This minimizes the number of policies that depend on an external webhook being up, which is the whole game for tier-0 risk.

What to enforce: the supply-chain policy catalogue

Admission is where the entire preceding curriculum becomes *mandatory* rather than *advisory*. Each policy below consumes evidence produced upstream; the enforcement point is the cluster boundary.

Image provenance (consumes Books 4-5). Only images **signed by the expected build identity**, carrying **required attestations** (SLSA provenance, SBOM, vuln scan, optionally VEX). This is the Book 6, Chapter 5 material — Kyverno `verifyImages`, Sigstore policy-controller `ClusterImagePolicy`, or Ratify+Gatekeeper. It is the one policy VAP *cannot* do, because it requires verifying signatures against a registry and Rekor.

Approved registries only (Book 6, Chapter 2). No pulls from `docker.io` or arbitrary hosts — only `registry.example.com` and specific

vendored mirrors. Expressible in all three engines; put it in **VAP**. This is a surprisingly high-value, low-cost policy: it forecloses a whole class of typosquat/dependency-confusion image attacks at the cluster.

Approved base images (Book 6, Chapter 3). Harder, because the base isn't in the pod spec — you enforce it upstream (build-time) and *attest* it, then verify the attestation at admission. Admission can enforce the *registry/repo* of the final image directly and the *base* via a provenance attestation predicate.

No `:latest`, require digests (Book 6, Chapter 1). Mutable tags defeat reproducibility and digest pinning. A structural VAP rule — `object.spec.containers.all(c, c.image.contains('@sha256:'))` — or Kyverno pattern rejects any un-pinned image; a `verifyImages` mutation pins tag→digest for you.

Workload hardening (defense in depth, adjacent to supply chain). Non-root, no `privileged`, drop `ALL` capabilities, read-only root filesystem, `seccomp RuntimeDefault`, no `hostPath/hostNetwork/hostPID`. Mostly covered by **PSA restricted**; anything beyond the three standard levels goes in VAP/Kyverno. These aren't supply-chain policies per se, but they cap what a compromised image can do once admitted, which is why they belong in the same policy bundle.

Required metadata (Book 3 inventory; Book 8 governance). Ownership labels (`app.example.com/owner`, `team`), cost-allocation labels, criticality/tier annotations. Cheap to enforce, disproportionately valuable at fleet scale: it is what makes "which team owns the thing running CVE-X" answerable during an incident (Book 8). A VAP or Kyverno `validate` requiring the label set, plus a `mutate`/MAP to default it, is the standard construction.

All of these are **policy-as-code**: versioned in a repo, reviewed, tested, and GitOps'd — the governance dimension of which is Book 8, Chapter 4.

Operational realities

This is the section that separates a policy engine that hardens your fleet from one that pages you at 3 a.m. because nobody can create a pod.

Roll out audit-first, always

Never enforce blind. A policy that looks obviously correct will, on contact with a real fleet, match objects you didn't anticipate: a monitoring DaemonSet that legitimately needs `hostPath`, a vendor operator whose images come from `quay.io`, a namespace full of Jobs that predate your label convention. Enforcing that policy `deny` on day one turns those into an outage.

The disciplined path, identical in shape across all three engines:

Stage	Gatekeeper	Kyverno	VAP
Observe	<code>enforcementAction: dryrun</code>	<code>validationFailureAction: Audit</code>	<code>validationActions: ["Audit"]</code>
Nudge	<code>warn</code>	(per-rule <code>Audit</code> + admission warnings)	<code>["Warn"]</code>
Enforce	<code>deny</code>	<code>Enforce</code>	<code>["Deny"]</code>

Run in observe mode, read the audit output (Gatekeeper constraint `status.violations`, Kyverno `PolicyReport`s, VAP audit annotations in the API audit log), and *fix the real violations you find* — either by remediating the workloads or by adding a scoped exception (below). Only when the audit report is clean for a namespace do you flip that namespace to enforce. This is the Book 1, Chapter 10 and Book 5, Chapter 10 rollout discipline applied to admission: measure, then enforce, per scope, never fleet-wide in one move.

The webhook is tier-0

A webhook-based engine (Gatekeeper, Kyverno, any `verifyImages` controller) is in the admission path of **every matching write in the cluster**. Think through the failure modes explicitly:

- **failurePolicy: Ignore (fail-open).** If the engine is down, writes are admitted *unchecked*. Your policy silently stops enforcing during exactly the window an attacker would exploit. Safe for availability, unsafe for security.
- **failurePolicy: Fail (fail-closed).** If the engine is down, matching writes are *rejected*. Secure, but now a crashed policy pod blocks deployments — and worse, can **deadlock its own recovery**: if the

webhook matches on `Pods` cluster-wide and its own pods get rescheduled, the API server calls a webhook that is down to admit the pod that would bring the webhook back up. The cluster wedges.

Mitigations you apply regardless of engine:

- **Exclude the control plane and the engine's own namespace.** The `namespaceSelector` must exclude `kube-system` and the policy engine's namespace so a broken policy cannot block the components that recover it. This is non-negotiable for fail-closed.
- **Scope rules tightly.** Match only the resources you actually police. A webhook matching `"*"` on every resource multiplies both latency and blast radius.
- **Run the engine HA** with multiple replicas, a `PodDisruptionBudget`, anti-affinity across nodes, and generous priority (`system-cluster-critical`) so it is scheduled and stays scheduled.
- **Bound `timeoutSeconds`** to a few seconds so a slow engine degrades to `failurePolicy` quickly rather than adding seconds of latency to every write.
- **Prefer a hybrid `failurePolicy`.** A common production posture: *fail-open* for broad, low-severity mutating/labeling webhooks (availability wins), *fail-closed* only for the high-severity validating policies where admitting an unverified image is worse than a deployment freeze — and even then, only after the exclusions above are proven correct.

This is the single strongest argument for VAP: every policy you can move in-tree is a policy that *has no webhook to be down*, so its fail-closed behavior can never wedge the control plane. Push structural policy to VAP precisely to shrink the set of policies that carry this risk.

Exceptions, exclusions, and break-glass

Real fleets need principled escape hatches, or teams route around the engine entirely.

- **Namespace / label exclusions** for system and infra namespaces that legitimately violate baseline policy. Keep the list small and reviewed.
- **Scoped, expiring exceptions.** Kyverno has a first-class `PolicyException` CRD — it names the exact policy, rule, and resource

selector that is exempted. Gatekeeper does exceptions via `excludedNamespaces` in its `Config`, per-constraint `match`, and (newer) `Assign` /constraint-level exemptions. The discipline that matters more than the mechanism: **every exception has an owner, a reason, and an expiry**. An exception without an expiry is a permanent hole nobody remembers opening.

```
apiVersion: kyverno.io/v2
kind: PolicyException
metadata:
  name: monitoring-hostpath-exception
  namespace: monitoring
  annotations:
    owner: platform-observability
    reason: "node-exporter requires hostPath /proc; JIRA PLAT-4821"
    expires: "2026-10-01"
# convention; enforce via CI/cleanup job
spec:
  exceptions:
    - policyName: disallow-host-path
      ruleNames: ["host-path"]
  match:
    any:
      - resources:
          kinds: ["Pod"]
          namespaces: ["monitoring"]
          names: ["node-exporter-*"]
```

- **Break-glass** for incidents: a documented, audited procedure to disable or scope-down a policy when it is *itself* the outage (a bad policy blocking a critical deploy). This means the ability to quickly flip `Enforce` → `Audit` or delete a constraint — via the same GitOps path, with an audit trail — not a shared admin credential that bypasses everything. And because `expires` / `reason` are annotations, not enforced fields, back them with a CI job that fails the repo when an exception is past expiry.

Test policies before they touch a cluster

Policy is code; untested policy is untested code in the critical path. Test it in CI:

- **Rego:** `conf test` and `opa test` run Rego unit tests against fixture inputs. Gatekeeper's `gator` CLI (`gator test`, `gator verify`)

evaluates ConstraintTemplates + Constraints against sample manifests offline.

- **Kyverno:** the `kyverno` CLI — `kyverno test` runs a declarative test manifest (policy × resource × expected result), and `kyverno apply` evaluates policies against manifests locally. This runs in CI on every policy PR.
- **VAP:** validate CEL expressions with the same CEL tooling used for CRD rules, and dry-run the policy with `validationActions: ["Audit"]` in a non-prod cluster before adding `Deny`.

```
# Kyverno: unit-test a policy against fixtures in CI
kyverno test ./policies/
```

```
# Gatekeeper: evaluate templates+constraints against manifests offline
gator test --filename=constraint.yaml --filename=template.yaml -f pod.y
aml
```

A policy PR should not merge without passing tests, exactly like application code (Book 4, Chapter 4).

Distributed-systems lens

Admission policy is the **cluster-boundary enforcement of fleet-wide supply-chain requirements**. Its power comes from being non-bypassable at exactly the point where an artifact becomes a running workload — and its risk comes from that same universality.

The move that makes it a *distributed-systems* control rather than a per-cluster hack is **authoring once and distributing everywhere**. The platform/security team writes one bundle — image verification, approved registries and bases, digest pinning, provenance requirements, workload hardening, ownership metadata — and **GitOps'ss it to every cluster** in the fleet (Book 6, Chapter 7 — GitOps and Cluster Reconciliation; Book 8, Chapter 4 — Policy as Code and Governance). The policy repo is the source of truth; a reconciler (Argo CD / Flux) applies it to each cluster; drift is corrected automatically. This is the paved road: application teams inherit the secure default without writing a line of policy, and "secure" becomes the *only* state the cluster will accept, uniformly, everywhere — not a checklist each team may or may not follow.

The engine is the **consumer** at the end of the produce-and-verify loop. Books 3-5 produced the evidence: SBOMs (Book 3), SLSA provenance from the build platform (Book 4), cosign signatures and in-toto attestations in the registry (Book 5). None of that evidence *does* anything until something demands it. Admission is that something. `verifyImages` demanding a signature by the build workflow's identity is the cluster consuming what CI produced — closing the loop at the boundary where it matters.

And it is **tier-0**: in the critical path of every object in every cluster. That is why the availability engineering in this chapter is not optional polish. The fleet-wide version of the audit-first rule is stark: a bad policy pushed via GitOps to fifty clusters is a fifty-cluster outage if it was enforcing and fail-closed. So the fleet discipline is *audit-first across the fleet* — roll the policy out in `Audit/dryrun` to all clusters, read the aggregated `PolicyReports`, fix the real violations, then progressively flip scopes to enforce, canarying clusters the way you canary code. The emergence of in-tree **VAP** is the structural answer to the tier-0 problem: every policy that moves from a webhook into the API server is one fewer external dependency in every cluster's critical path — which is why, at fleet scale, "what can we push to VAP?" is now a first-order design question.

Key takeaways

- **Admission is the policy enforcement point** because it is the last checkpoint before an object is persisted and the first that can reason about the object's *content* rather than the caller's identity. RBAC gates the actor; admission gates the artifact.
- The pipeline order is fixed and load-bearing: **authn → authz → mutating admission → schema validation → validating admission → etcd**. Mutating runs first so a mutation (e.g., digest pinning) can make a later validation pass.
- A webhook's behavior lives in its registration: **failurePolicy (Fail=fail-closed vs Ignore=fail-open)** is the availability-vs-security lever, `timeoutSeconds` is latency in the hot path, `namespaceSelector` must exclude the control plane, `reinvocationPolicy` handles mutation ordering.

- **Pod Security Admission** (privileged/baseline/restricted) is the in-tree hardening baseline and the PSP successor — **PSP was removed in 1.25**. Use PSA for the standard levels; a general engine for anything more.
- **Three engines, layered, not chosen:** PSA for the baseline; **VAP (CEL, in-tree, GA 1.30)** for structural policy you want off the webhook and safely fail-closed; a **webhook engine — Kyverno (YAML) for most teams, Gatekeeper (Rego) for OPA shops** — for signature verification, `generate`, and external-data policy that VAP cannot express.
- **VAP's decisive advantage is that it has no webhook** — no network hop, no external dependency, no `failurePolicy: Fail` scenario that wedges the control plane. Push everything expressible to VAP to shrink the webhook's blast radius.
- **Enforce the whole book at the boundary:** signed images from expected identities with required attestations, approved registries and bases, digest pins (no `:latest`), workload hardening, and ownership metadata — all as versioned, tested, GitOps'd policy-as-code.
- **Operate it like tier-0:** audit/warn before enforce (per scope, never blind), HA + control-plane exclusions + bounded timeouts for fail-closed webhooks, exceptions with owner/reason/expiry, and policy tested in CI (`kyverno test` , `gator test` , `confest`) before it reaches a cluster.

Further reading

- **Kubernetes — Dynamic Admission Control.** Mutating/validating webhooks, `failurePolicy` , `reinvocationPolicy` , selectors, ordering, `AdmissionReview` . <https://kubernetes.io/docs/reference/access-authn-authz/extensible-admission-controllers/>
- **Kubernetes — Validating Admission Policy.** The in-tree CEL mechanism, `ValidatingAdmissionPolicy`
- `ValidatingAdmissionPolicyBinding` , `validationActions` , variables. <https://kubernetes.io/docs/reference/access-authn-authz/validating-admission-policy/>
- **Kubernetes — Pod Security Admission** and the **Pod Security Standards** (privileged/baseline/ restricted). <https://kubernetes.io/>

[docs/concepts/security/pod-security-admission/](https://kubernetes.io/docs/concepts/security/pod-security-admission/) and <https://kubernetes.io/docs/concepts/security/pod-security-standards/>

- **Open Policy Agent** and **Rego** language reference. <https://www.openpolicyagent.org/docs/> and <https://www.openpolicyagent.org/docs/policy-language>
- **OPA Gatekeeper**. ConstraintTemplates, Constraints, audit, mutation, external data, `gator` CLI. <https://open-policy-agent.github.io/gatekeeper/website/docs/>
- **Kyverno documentation**. ClusterPolicy/Policy, validate/mutate/generate/verifyImages, PolicyException, policy reports, the `kyverno` CLI. <https://kyverno.io/docs/>
- **CEL language definition** (the expression language behind VAP and CRD validation). <https://github.com/google/cel-spec>
- **Kubernetes Enhancement Proposals** — KEP-3488 (mutating admission policies / MutatingAdmissionPolicy) and the ValidatingAdmissionPolicy KEP-3492, for the design rationale and version history. <https://github.com/kubernetes/enhancements>

Chapter 7 — Kubernetes Delivery Chains: Helm, Operators, GitOps

What this chapter covers. Chapters 1-5 of this book secured the **container image** as an artifact: how it is built (Book 4), scanned (Chapter 4), signed (Chapter 5), and verified at admission (Chapters 5-6). But an image is not a deployment. Something has to *package* that image together with the manifests, RBAC, config, secrets, and dozens of other images it depends on, and something has to *deliver* that package into a running cluster. That packaging-and-delivery layer — **Helm charts, Kubernetes Operators, and GitOps controllers** — is its own supply chain, with its own artifacts, its own repositories, its own transitive dependencies, and its own attack surface. It is also the *least* secured link in most organizations: teams that meticulously sign and scan images will `helm install` a chart pulled from a public index over TLS-and-a-prayer, granting it the power to deploy anything. This chapter treats the delivery layer as a supply chain in its own right. We cover Helm's architecture and its provenance and OCI-signing story; the Operator pattern and why an

in-cluster controller with broad RBAC is a supply-chain risk of the first order; GitOps (Argo CD and Flux) as the modern pull-based delivery model, its genuine security advantages, and its genuine new failure modes; and finally how these compose into a single **delivery chain of custody** — signed commit → verified chart → verified image → admission backstop — where every link must hold or the whole chain is theatre.

Learning goals — after this chapter you should be able to:

- Explain why **what gets deployed is not just an image** but a graph of manifests, charts, subcharts, CRDs, operators, and images, each a tamperable supply-chain artifact.
- Describe **Helm 3's architecture** (client-side, no Tiller), state precisely why **Helm 2's Tiller** was a security disaster, and reason about charts as *code* that can deploy anything.
- Secure a Helm workflow: **provenance files (.prov) and helm verify** , **OCI-registry charts signed with cosign**, digest pinning, internal chart repositories, and **rendering-then-policy- checking** chart output before it ever reaches a cluster.
- Analyze **Operators** as privileged in-cluster software: the CRD-plus-controller pattern, OperatorHub/OLM as a distribution channel, over-broad RBAC as a least-privilege failure, and how to scope and monitor them.
- Analyze **GitOps** (Argo CD, Flux): git as source of truth, pull-based reconciliation, the security appeal *and* the fact that the repo and the controller become **tier-0**, plus what Flux and Argo actually do (and do not) verify.
- Compose the whole into a **defensible chain of custody** and place admission (Chapter 6) as the backstop that verifies whatever the delivery chain produces.

The delivery layer is a supply chain

Return to the mental model from Book 2, Chapter 1 — Dependencies as a Supply Chain. A package manager takes a *manifest* of what you want, resolves it against one or more *repositories*, downloads *artifacts* (often with *transitive* dependencies you never named), and assembles them into something that runs. Every one of those steps is an integrity boundary: a

compromised repository, a typosquatted artifact, an unpinned transitive dependency, an unsigned download.

Kubernetes delivery is exactly this pattern, one level up. The thing you *want* is "run version 1.4.2 of the payments service." The thing that actually lands in etcd is a Deployment, a Service, an Ingress, a HorizontalPodAutoscaler, a ServiceAccount, three Roles and RoleBindings, a ConfigMap, a couple of Secrets, and — pulled in behind your back — an init-container image, a sidecar image, and whatever a charted subdependency decided to deploy. None of that is the image you signed. All of it is a supply-chain artifact.

```
flowchart TD
    I["Requested: run payments v1.4.2"]
    I --> M["Manifests  
Deployment, Service, RBAC"]
    I --> H["Helm chart  
templates + values"]
    H --> SC["Subcharts  
redis, common, ..."]
    I --> OP["Operators + CRDs  
in-cluster controllers"]
    M --> IMG1["App image"]
    H --> IMG2["Chart-pulled images  
sidecars, init, deps"]
    SC --> IMG3["Subchart images"]
    OP --> IMG4["Operator image  
+ operand images"]
    CFG["Config + values + secrets"] --> M
    CFG --> H
    classDef risk fill:#7a1f1f,stroke:#d66,color:#fff
    class SC,IMG2,IMG3,IMG4,OP risk
```

The highlighted nodes are the ones teams routinely *fail* to vet. You chose the app image; you did not choose the Redis subchart's image, the operator's operand images, or the sidecar a chart injects. Each is pulled "from wherever" — often a public registry, often by a floating tag, often with no signature check. And unlike an application dependency (Book 2), a Kubernetes delivery artifact is *directly executable at the highest privilege in your cluster*: a chart template is Go code that emits manifests, and a manifest can request a privileged pod, a hostPath mount, a cluster-admin binding, or an exfiltration sidecar. There is no sandbox between "the chart rendered this" and "the API server persisted this" except the one you build: admission (Chapter 6).

This is the framing for the whole chapter. **Securing images (Chapters 1-5) secures one node in this graph.** The delivery layer is the rest of the graph, and it is where an attacker who cannot forge your signed image will instead attack the chart that deploys it, the operator that manages it, or the git repo that describes it.

Helm: the Kubernetes package manager

Helm is the de facto package manager for Kubernetes. A **chart** is a directory (or a `.tgz` archive of one) containing:

- `Chart.yaml` — metadata: name, version, `appVersion`, and a `dependencies` list.
- `values.yaml` — the default configuration, a tree of keys the templates read.
- `templates/` — Go `text/template` files that, rendered against the values, produce Kubernetes manifests.
- optionally `charts/` — vendored **subcharts** (dependencies), and a `Chart.lock`.

You render a chart with `helm template` (produce YAML, apply it yourself) or install it with `helm install`, which renders *and* submits the objects to the API server, recording the result as a **release** — a named, versioned, revisionable unit you can `upgrade`, `rollback`, or `uninstall`. Charts live in **chart repositories**: historically an HTTP server hosting an `index.yaml` plus `.tgz` files (the model behind **Artifact Hub**, the public index of thousands of charts); increasingly, **OCI registries** (Book 6, Chapters 1-2), where a chart is just another OCI artifact stored next to your images. The repository model is a direct parallel to Book 2, Chapter 1 (npm/PyPI/Maven registries) and Book 2, Chapter 8 (running an internal registry): same trust questions, same typosquatting and dependency-confusion exposure, same "who can publish" problem.

The Tiller lesson: Helm 2's server-side agent

You cannot reason about Helm's security without understanding what Helm 3 *removed*. Helm 2 had a server-side component called **Tiller**: an in-cluster pod that received your rendering requests, did the templating, and applied the resulting objects to the API server on your behalf. The client (`helm`) talked to Tiller over gRPC; Tiller talked to the API server.

The security consequences were severe and are worth stating precisely because they are the archetype of a badly-designed delivery agent:

- **Tiller held a broad, usually cluster-admin, ServiceAccount.** It had to, because it deployed arbitrary charts into arbitrary namespaces. So *every* release ran with Tiller's privileges, not the caller's. A developer who could reach Tiller could deploy anything Tiller could — which was everything.
- **In its default install, the gRPC endpoint had no authentication and no TLS.** Anyone who could reach Tiller's port in the cluster network could ask it to install charts. TLS and cert-based auth were opt-in and widely skipped. This turned an unauthenticated in-cluster endpoint into a cluster-admin remote-code-execution primitive; it was a staple of Kubernetes pentest findings for years.
- **RBAC was effectively bypassed.** The point of Kubernetes RBAC is that *your* identity bounds what you can do. Tiller collapsed every user's authority into its own.

Helm 3 deleted Tiller entirely. Helm 3 is a **client-side** tool: it renders templates locally and talks to the API server *as you*, using your kubeconfig and your RBAC. Release state moved from Tiller's memory into Secrets in the release namespace. The lesson generalizes to everything else in this chapter: **a delivery agent that runs in-cluster with broad privilege and accepts work over the network is a cluster-takeover primitive.** Hold that thought — it is exactly the shape of an Operator (below) and a GitOps controller (further below), and the reason both must be secured with care Helm 2 did not take.

Charts are code: the risk surface

A chart is not data. `templates/` is Turing-adjacent Go templating with functions (`tpl`, `lookup`, `include`, Sprig helpers), conditionals, and loops, and its *output* is unrestricted Kubernetes YAML. A malicious or compromised chart can therefore deploy anything the applying identity is permitted to create:

- **Privileged workloads** — `securityContext.privileged: true`, `hostPID`, `hostNetwork`, `hostPath: /`, or a pod that mounts the node's container runtime socket and breaks out.

- **Bad RBAC** — a ClusterRoleBinding to `cluster-admin` for the chart's ServiceAccount, quietly granting the workload the keys to the cluster.
- **Exfiltration sidecars** — an extra container in the pod that ships secrets or traffic outbound.
- **Attacker-controlled images** — the chart's `values.yaml` points `image.repository` at a registry you do not control, or an unpinned tag that can be re-pushed.

And the risk is transitive. `Chart.yaml` dependencies pull **subcharts** from other repositories. `helm dependency update` resolves them into `charts/` — a classic transitive-dependency graph (Book 2, Chapter 3 — Transitive Dependencies), except each node can emit privileged manifests. A subchart you never audited can inject a sidecar into a pod defined by the parent chart. Worse, the *images* a chart deploys are a second, parallel dependency graph: the chart references image references, and those images are pulled at pod-creation time from wherever the values say — sources you almost never vet with the rigor you apply to your own app image.

Add **values injection**: values flow from `values.yaml`, `-f overrides.yaml`, and `--set` flags on the command line, with later sources overriding earlier ones. A value that lands in a template position that renders into a command, an image reference, or an annotation consumed by another controller is an injection point. `--set image.repository=evil.example.com/x` is a one-line supply-chain substitution, and it is *the intended interface*.

```

flowchart TD
    R["Chart repo / Artifact Hub  
(varying trust)"] --> C["Chart .tgz  
Chart.yaml + templates + values"]
    C --> D1["Subchart A"]
    C --> D2["Subchart B"]
    D1 --> D3["Subchart A.1 (transitive)"]
    C --> V["Values: file + -f + --set  
(injection surface)"]
    C --> IR["Image references"]
    D1 --> IR2["Subchart images"]
    IR --> REG["Any registry  
floating tags"]
    IR2 --> REG
    C -->|helm template| Y["Rendered manifests  
can be ANYTHING"]
    Y --> K["API server"]
    classDef risk fill:#7a1f1f,stroke:#d66,color:#fff
    class D1,D2,D3,IR2,V,REG risk

```

Securing Helm

There is no single control; you layer several, mirroring the image controls from earlier chapters but applied to the chart artifact and to its rendered output.

1. Chart provenance: .prov files and helm verify. Helm has a native provenance mechanism predating Sigstore. When you `helm package --sign`, Helm computes a checksum of the chart archive and writes a **provenance file** (`<chart>.tgz.prov`) containing the `Chart.yaml`, the archive's SHA-256, and a **clear-signed PGP block** over that data. Consumers run `helm verify chart.tgz` (which requires the `.prov` alongside the `.tgz` and a keyring), or `helm install --verify`, to check that the signature is valid and the checksum matches:

```

# Publisher: package and sign with a PGP key
helm package --sign --key 'release@example.com' \
  --keyring ~/.gnupg/secring.gpg ./payments

# produces payments-1.4.2.tgz and payments-1.4.2.tgz.prov

# Consumer: verify integrity + signature before install
helm verify --keyring ./pubring.gpg payments-1.4.2.tgz
helm install payments ./payments-1.4.2.tgz --verify --keyring ./
pubring.gpg

```

This works, but it is PGP: key distribution and trust are on you, there is no transparency log, and in practice adoption on public repos is thin. Treat it as available and better-than-nothing, not as the modern default.

2. The modern default: OCI charts signed with cosign. Since Helm 3.8, charts are first-class **OCI artifacts** — you `helm push chart.tgz oci://registry.example.com/charts` and it lands in the registry by digest, exactly like an image (Book 6, Chapters 1–2). Because it is an OCI artifact with a digest, you can sign it with **cosign** and verify it with the *same* Sigstore machinery you use for images (Book 5, Chapter 3), including keyless signing from CI identity (Book 5, Chapter 4) and Rekor transparency (Book 5, Chapter 5):

```
# Publish the chart to an OCI registry (returns a digest)
helm push payments-1.4.2.tgz oci://registry.example.com/charts
# Pushed: registry.example.com/charts/payments:1.4.2
# Digest: sha256:5c9e...a1

# Sign the chart artifact by digest, keyless, from CI
cosign sign registry.example.com/charts/payments@sha256:5c9e...a1

# Consumer verifies signer identity before pulling
cosign verify registry.example.com/charts/payments@sha256:5c9e...a1 \
  --certificate-identity-regexp 'https://github.com/example/charts/.*' \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com
```

This is the recommendation: **sign charts like images, verify at pull/deploy time, and pin by digest**. One trust model, one toolchain, one transparency log across every artifact in the delivery graph.

3. Pin chart *and* image digests. A chart pinned to `payments:1.4.2` is pinned to a mutable tag; pin the chart to its `@sha256:` digest, and inside the chart pin every image to a digest too. This closes the same re-push window as digest-pinning images (Book 6, Chapter 1) and lets admission enforce “no floating tags” against chart output (Chapter 6).

4. Vet, then host internally: golden charts. Do not `helm repo add` a public index and install straight from it any more than you would `pip install` from an unknown source into production. Vet charts once, then serve them from an **internal chart repository** (an OCI registry works fine — Book 2, Chapter 8’s internal-registry pattern). This is the delivery-layer analogue of **golden base images** (Book 6, Chapter 3): a curated, versioned, signed set of **golden charts** that encode your security

defaults (non-root, pinned images, sane RBAC, resource limits). Teams consume the paved road; the surface of "any chart from anywhere" collapses to "our charts."

5. Render, scan, and policy-check the *output*. The single most under-used Helm control: `helm template` renders the chart to plain YAML *without touching a cluster*, and you can inspect what it actually produces before anything is applied. Run policy over that output in CI:

```
# Render, then policy-check the manifests the chart would create
helm template payments ./payments -f prod-values.yaml > rendered.yaml

# conftest (OPA/Rego) against the rendered manifests
conftest test rendered.yaml --policy ./policy

# or Kyverno CLI applying the same ClusterPolicies you enforce at
admission
kyverno apply ./policy -r rendered.yaml
```

```
# policy/deny_privileged.rego – reject privileged output from ANY chart
package main

deny contains msg if {
  input.kind == "Deployment"
  c := input.spec.template.spec.containers[_]
  c.securityContext.privileged == true
  msg := sprintf("container %q requests privileged mode", [c.name])
}

deny contains msg if {
  input.kind == "ClusterRoleBinding"
  input.roleRef.name == "cluster-admin"
  msg := "chart binds a subject to cluster-admin"
}
```

Now a chart that *tries* to deploy a privileged pod or a cluster-admin binding fails in CI, before it is ever proposed to a cluster — regardless of which subchart smuggled it in. Reuse the *same* policies you enforce at admission (Chapter 6) so CI and the cluster agree.

6. Backstop at admission. Everything above is shift-left. The cluster-side backstop is Chapter 6: admission verifies image signatures, rejects floating tags and privileged pods, and enforces approved registries on *whatever the chart rendered*, no matter how it was rendered. CI policy and admission policy are defense in depth, not alternatives.

Operators: the CRD-plus-controller pattern

An **Operator** extends Kubernetes with domain knowledge. It is two things: one or more **Custom Resource Definitions (CRDs)** that add new object kinds to the API (e.g., `PostgresCluster`, `Kafka`, `CertificateRequest`), and a **controller** — a pod running a reconciliation loop that watches those custom resources and drives the real world to match them. You create a `PostgresCluster` object; the operator's controller creates the StatefulSets, Services, Secrets, and backup CronJobs that constitute a running database, and keeps reconciling them forever.

That reconciliation power is exactly the problem. To manage StatefulSets, Secrets, PVCs, Services, and RBAC on your behalf, the controller needs RBAC to *create and modify* all of those — often across all namespaces. **An operator is privileged, long-running, in-cluster software with broad authority.** In the taxonomy of the Tiller lesson, it is Tiller-shaped: a persistent agent holding wide permissions. The difference is that a good operator's authority is *scoped*, and its interface is the API server (subject to RBAC and admission) rather than an unauthenticated gRPC port. Those differences are only real if you enforce them.

The risks

- **A compromised or malicious operator is a cluster compromise.** If the operator holds `secrets: [get, list]` cluster-wide (many do, to wire credentials into operands), an attacker who controls the operator image or its inputs can read every secret in the cluster. If it can create Deployments and RBAC, it can schedule attacker workloads and escalate. The operator *image* is a supply-chain artifact (Book 6, Chapters 1–5): sign it, scan it, demand provenance, exactly as you would any tier-0 workload.
- **The distribution channel.** Operators are distributed through **OperatorHub** and installed via the **Operator Lifecycle Manager (OLM)**, which packages an operator as a **bundle** (CSV — `ClusterServiceVersion` — plus CRDs and RBAC) and resolves it from a **catalog**. This is a package registry with the same trust question as any other (Book 2, Chapter 1): *who published this, and did anyone*

verify it? Community operators vary wildly in quality and permission hygiene.

- **Over-broad RBAC.** The CSV *declares* the RBAC the operator wants, and OLM grants it at install. Many operators request far more than they need — cluster-wide `*` verbs on `secrets`, or blanket access to core resources — because it is easier than scoping. Installing such an operator is a least-privilege failure (Book 1, Chapter 6) baked in at delivery time. **Read the CSV's `install` permissions before you install; treat a request for cluster-wide secret access or `*/*` as a finding.**
- **Operands pull their own images.** The operator deploys operand images (the actual Postgres, Kafka, etc.), frequently by tag from upstream registries — a second image supply chain you did not choose, arriving through the operator.
- **CRDs are API surface.** A CRD can register conversion/validation webhooks that sit in the admission path (Chapter 6); a malicious CRD is another way into the request pipeline.

```
flowchart TD
    OH["OperatorHub / catalog  
(trust of source)"] --> OLM["OLM installs bundle  
CSV + CRDs + RBAC"]
    OLM --> RBAC["Grants declared RBAC  
(often over-broad)"]
    OLM --> CTRL["Operator controller pod  
(operator image)"]
    RBAC --> CTRL
    CTRL --> CR["watches| CR|\"Custom Resources  
PostgresCluster, ...\""]
    CTRL --> RES["creates/modifies| RES|\"StatefulSets, Secrets,  
Services, RBAC, PVCs\""]
    CTRL --> OPND["deploys| OPND|\"Operand images  
(upstream, tagged)\""]
    classDef risk fill:#7a1f1f,stroke:#d66,color:#fff
    class OH,RBAC,OPND risk
```

Securing operators

- **Vet the source and the image.** Prefer operators you can point to a signed image, an SBOM (Book 3), and SLSA provenance (Book 5) for. Verify the operator image's signature at admission like any workload (Chapter 5). Mirror it into your internal registry (Book 6, Chapter 2) rather than pulling from upstream at install.

- **Scope RBAC to least privilege.** Read the CSV. If the operator only manages resources in the namespaces it owns, it should hold **namespaced** Roles, not ClusterRoles. Constrain secret access to specific namespaces. An operator that demands `cluster-admin` should be rejected or run in a tightly isolated cluster. This is the single highest-leverage control: it bounds the blast radius of a compromise to what the operator can actually touch.
 - **Use OLM's operator groups and install modes** to constrain which namespaces an operator watches; a single-namespace install mode is far safer than `AllNamespaces` for an operator that does not need cluster-wide scope.
 - **Policy the operator's *output* at admission.** The operator creates objects like anything else; those objects pass through admission. Enforce your image, registry, and hardening policies (Chapter 6) on operator-created pods so a compromised operator still cannot schedule an unsigned or privileged workload. The operator is not exempt from the boundary.
 - **Monitor behavior.** An operator's normal action set is narrow and knowable. Audit-log its ServiceAccount (Book 8, incident response) and alert on out-of-profile actions — creating RBAC it never created before, reading secrets outside its namespaces, scheduling pods with unusual specs.
-

GitOps: git as the source of truth

GitOps is the delivery model that ties this all together, and it is where the delivery layer becomes most clearly a supply chain. The idea: **the desired state of the cluster lives in git**, and an in-cluster controller **continuously reconciles** the real cluster to match what git says. You do not `kubectl apply` to production; you open a pull request, it merges, and the controller (**Argo CD** or **Flux**) notices the change and converges the cluster to it. The two dominant implementations differ in ergonomics but share the model: Argo CD is application-centric with a UI and an `Application` CRD; Flux is a set of composable controllers (source, kustomize, helm, image-automation) driven by CRDs.

The security appeal is real and worth naming precisely, because it is the reason GitOps is a *better* delivery model, not just a fashionable one:

- **Declarative and auditable.** The entire desired state is in git. What is deployed is what is in the repo at a commit, and git history is a signed-or-signable, immutable-ish audit log of who changed what and when (Book 7 — Source Security).
- **Pull-based.** The controller *pulls* from git and reconciles from *inside* the cluster. There is no CI system holding cluster-admin credentials and pushing to the API server from outside; the cluster's write path does not require inbound access. This shrinks the credential-exposure surface compared to push-based CI deploys dramatically — there is no long-lived kubeconfig in a runner to steal (Book 4, Chapter 8).
- **Reviewable.** Changing production is a pull request. Branch protection, required reviews, and CI checks (Book 7, Chapters 2-3) now gate deployment the same way they gate code.

```
flowchart LR
    DEV["Developer"] -->|PR + review| GIT["Git repo  
(desired state)"]
    GIT -->|controller PULLS| CTRL["GitOps controller  
Argo CD / Flux  
(in-cluster)"]
    CTRL -->|render| REN["Helm / kustomize"]
    REN -->|verify + reconcile| API["API server"]
    API --> ADM["Admission  
(Ch 6 backstop)"]
    ADM --> CLU["Cluster state"]
    CLU -.observed drift.-> CTRL
    classDef tier0 fill:#1f3a7a,stroke:#6ad,color:#fff
    class GIT,CTRL tier0
```

The supply-chain implication: repo and controller are tier-0

The pull-based model does not remove risk; it **relocates** it. Two things become the most security-critical assets in your entire delivery chain, highlighted above:

1. The GitOps repository is now production. Whoever can merge to the GitOps repo can deploy anything to the cluster the controller manages. The repo *is* the deploy button. A compromised commit — a malicious `Deployment`, an image swap in a values file, a new `ClusterRoleBinding` — reconciles straight into production with no further

human in the loop, because reconciliation is the whole point. **Book 7's source-control security is now deployment security.** Every SCM control from Book 7 applies with production stakes: branch protection with required reviews, **signed commits** (verify them — see below), protected `main`, no direct pushes, restricted merge rights, and CI that scans and policy-checks the manifests before merge. A GitOps repo with a weak review policy is a cluster with a weak deploy policy.

2. The controller is Tiller, done right — if you do it right. Argo CD and Flux run in-cluster with broad permissions: to reconcile arbitrary objects into arbitrary namespaces, they need wide create/update/delete RBAC. Compromise the controller — its image (Book 6, Chapters 1-5), its service account token, or its config — and you have cluster-wide deploy access. This is the Tiller shape again, and the mitigations are the Tiller lessons: the controller must run with **scoped RBAC** where possible (Argo's app-project restrictions, per-cluster/per-namespace destinations; Flux's per-Kustomization `serviceAccountName` impersonation so a reconciliation runs as a bounded identity, not the controller's full authority), its image must be verified, and its blast radius bounded.

Beyond those two, GitOps inherits everything above:

- **It renders Helm and kustomize** — so every Helm risk (malicious chart, unvetted subchart, chart-pulled images) and every kustomize risk applies to what the controller renders.
- **Multi-source and remote refs.** A `Kustomization` can reference remote bases (`github.com/...` URLs), and an Argo `Application` or Flux `HelmRelease` can pull charts from remote repos. These are **untrusted remote content pulled at reconcile time** — a remote base can change under you unless pinned to a commit/digest. Pin remote references; do not float them.
- **Secrets.** You must not commit plaintext secrets to the GitOps repo; git history is forever and the repo is widely readable. The three standard answers: **Sealed Secrets** (encrypt to a cluster-held key with `kubeseal`; only the in-cluster controller can decrypt, so the ciphertext is safe in git), **SOPS** (encrypt values with age/KMS/PGP; Flux and Argo can decrypt at reconcile with a key they hold), and **External Secrets Operator** (commit only a *reference*; the operator fetches the real value from Vault/cloud KMS at runtime). All three keep plaintext out of git; they differ in where the trust root sits.

- **Drift and manual overrides.** A human `kubectl edit` in production is drift. Reconciliation will revert it (self-heal) — good for integrity, but it also means the controller's view of git is authoritative, which is exactly why the repo must be locked down. Argo can be set to auto-heal; treat manual changes as incidents, not workflow.

What Argo CD and Flux actually verify

Be precise here, because it is easy to overstate. GitOps controllers are not, by default, verifying signatures on what they deploy. You must turn verification on, and each tool supports a specific, bounded set of checks.

Flux has the more built-in verification story:

- **Git commit signature verification.** The `GitRepository` source can require that the tip commit (or tag) be signed by a key in a configured keyring, via `spec.verify`. Flux refuses to reconcile a source whose commit is not signed by a trusted key — enforcing Book 7's signed-commit control at the *deployment* boundary.
- **Cosign verification of OCI artifacts.** For **OCI-stored** content — an `OCIRepository` (charts or manifests pushed as OCI artifacts) — Flux can verify a **cosign** signature (keyed or keyless with identity/issuer matching) via `spec.verify.provider: cosign` before reconciling. Flux's Helm/OCI path can thus verify a signed chart artifact.
- **Image signature verification for image automation** is *not* a general admission-time image gate; Flux's image controllers automate updating image tags in git, and signature enforcement on running pods still belongs at admission (Chapter 6).

```
# Flux: require a cosign-verified OCI chart, signed by our CI identity
apiVersion: source.toolkit.fluxcd.io/v1
kind: OCIRepository
metadata:
  name: payments
  namespace: apps
spec:
  interval: 5m
  url: oci://registry.example.com/charts/payments
  ref:
    digest: sha256:5c9e...a1      # pin by digest, not tag
  verify:
    provider: cosign
    matchOIDCIdentity:
      - issuer: "https://token.actions.githubusercontent.com"
        subject: "https://github.com/example/charts/*"
```

```
# Flux: refuse to reconcile a Git source unless the commit is signed
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: fleet
  namespace: flux-system
spec:
  interval: 1m
  url: https://github.com/example/fleet-infra
  ref:
    branch: main
  verify:
    mode: HEAD      # verify the tip commit
    secretRef:
      name: git-signing-pubkeys  # keyring of trusted signers
```

Argo CD verifies GPG signatures on git commits: you configure a set of trusted GPG public keys, enable signature verification **per AppProject**, and Argo refuses to sync an application whose target commit is not signed by a trusted key. Argo does **not** ship a built-in cosign image/chart-signature gate the way Flux's OCI path does; for image-signature enforcement you pair Argo with an admission-time verifier (policy-controller, Kyverno `verifyImages`, Ratify — Chapter 5). The accurate summary: **Argo enforces commit provenance; image/artifact-signature verification is delegated to admission.** Do not claim Argo verifies image signatures natively; claim it verifies *commits* and relies on admission for the rest.

```
# Argo CD: require signed commits for everything in this project
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: payments
  namespace: argocd
spec:
  signatureKeys:
    - keyID: 4AEE18F83AFDEB23    # trusted GPG key; unsigned/foreign
    commits refused
  sourceRepos:
    - "https://github.com/example/fleet-infra.git"
  destinations:
    - server: https://kubernetes.default.svc
      namespace: payments
```

Securing GitOps: the checklist

- **Protect the repo like production:** branch protection, required review, signed commits, no direct push to `main`, least-privilege merge rights, and CI that renders + policy-checks manifests before merge (Book 7, Chapters 2-3; the `helm` template | `conftest` pattern above).
- **Make the controller verify provenance:** Flux `spec.verify` for commits and cosign for OCI charts; Argo `signatureKeys` for commits. Turn these on — they are opt-in.
- **Pin everything:** commits (not branches) for remote bases, digests (not tags) for OCI charts and images. Floating refs reintroduce mutability the whole model is meant to remove.
- **Least-privilege the controller:** Flux per-Kustomization service-account impersonation; Argo AppProject destination and source restrictions. The controller should not be able to deploy anywhere it does not manage.
- **Manage secrets out of plaintext:** Sealed Secrets, SOPS, or External Secrets — never commit a plaintext Secret.
- **Backstop at admission (Chapter 6):** verify image signatures, enforce registries and hardening on whatever reconciles. Defense in depth: the controller verifying charts and commits does not remove the need for the cluster to verify images at the boundary.
- **GitOps the policies themselves.** Manage your Kyverno/Gatekeeper/VAP policies (Chapter 6) *in the GitOps repo* so the

enforcement layer is versioned, reviewed, and fleet-distributed exactly like the workloads it governs.

Bringing it together: the delivery chain of custody

Each preceding section secured one link. Composed, they form a continuous chain of custody from a developer's commit to a running pod, where every artifact is signed by an identity you trust and verified by the next link before it is used. A break in any link is a break in the chain; the value is in the *composition*, not any single control.

```
flowchart LR
    subgraph Source["Source (Book 7)"]
        C["Signed commit  
reviewed PR"]
    end
    subgraph Build["Build (Books 4-5)"]
        IMG["Signed image  
+ SBOM + SLSA prov"]
        CH["Signed chart  
(cosign / OCI)"]
    end
    subgraph Deliver["Delivery (this chapter)"]
        GC["GitOps controller  
verifies commit sig  
+ cosign OCI chart"]
        REN["Render Helm / kustomize  
(pinned digests)"]
        subgraph Enforce["Admission (Chs 5-6)"]
            ADM["Verify image sig + identity  
+ attestations  
+ registry/hardening policy"]
        end
    end
    C --> GC
    CH --> GC
    IMG --> ADM
    GC --> REN --> ADM --> POD["Running pod"]
    classDef ok fill:#1f5a2f,stroke:#6c9,color:#fff
    class C,IMG,CH,GC,ADM ok
```

Read the chain as a sequence of *verify-before-use* boundaries, each consuming the evidence produced by the link before:

Link	Artifact	Signed by (Book)	Verified by (this chain)
Source	Git commit	Developer / CI key (Book 7)	Flux <code>spec.verify</code> / Argo <code>signatureKeys</code>
Package	Helm chart (OCI)	CI keyless identity (Book 5)	Flux cosign OCI verify; cosign in CI
Image	Container image + attestations	CI keyless identity (Books 4–5)	Admission verifier (Chs 5–6)
Config	Rendered manifests	(integrity via pinning)	CI conftest/Kyverno + admission (Ch 6)
Deploy	Persisted objects	—	Admission is the final gate (Ch 6)

The crucial property is that **no single compromised link deploys a malicious workload**, because each link's output is re-verified downstream. Forge a chart but not a commit signature, and the GitOps controller rejects the source. Compromise the git repo but not the image-signing identity, and admission rejects the unsigned image. Slip past the chart-rendering CI check, and admission still enforces registry and hardening policy on the output. The chain is defensible precisely because verification is *layered and redundant*, not concentrated at one point that, if bypassed, opens everything.

Distributed-systems lens

At fleet scale the delivery chain's economics and risk both concentrate, and the design has to account for that.

GitOps centralizes fleet delivery — and thereby concentrates risk. One control plane (an Argo CD or a Flux install, often a management cluster) reconciles desired state onto *many* clusters. This is the operational win of GitOps at scale: you change a golden config once and it propagates across the fleet, with git as the audit trail. But it makes the **GitOps repo and the controller tier-0 for the entire fleet** (Book 1, Chapter 9 — Tiered Trust and Blast Radius). Compromise of the repo is fleet-wide deploy access; compromise of the management controller is

fleet-wide cluster access. The mitigation is the same one that makes any tier-0 dependency tolerable: bound its blast radius (scoped per-cluster/per-namespace destinations, impersonation, network isolation of the management cluster), verify what it consumes (signed commits, signed charts), and monitor it as the critical asset it is.

Sign and verify at every layer so no single link is load-bearing alone. The recurring theme of Books 4-6 — commit, chart, image, admission — is not four independent controls; it is one chain where each link verifies the previous. At fleet scale you cannot manually review every deploy, so the chain of custody is the review: an artifact reaches a pod only by carrying signatures that verify at each boundary. This is what lets you deploy thousands of times a day across many clusters without a human in every loop — the machines verify what humans cannot re-check by hand.

The paved road shrinks the delivery-layer surface. An internal chart repository of **golden charts** (this chapter) is the delivery analogue of golden base images (Book 6, Chapter 3): most teams consume a small, curated, signed set that encodes your security defaults, so the effective attack surface is your charts, not the public long tail. Least-privilege for controllers and operators bounds what any single compromise can reach. And **admission (Chapter 6) is the backstop** that catches whatever the delivery chain — with all its rendering, remote refs, and third-party charts and operators — actually produces, at the one point every object must pass through. The result is a delivery chain of custody that is auditable (git history + signed commits), verifiable (signed charts and images checked at each hop), and bounded (least-privilege controllers, admission-enforced output) — which is the standard a distributed backend platform, deploying constantly across a fleet, has to meet.

Key takeaways

- **What deploys is not just an image.** It is a graph of manifests, charts, subcharts, CRDs, operators, and the images each pulls in — every node a supply-chain artifact, most of them never vetted with image-level rigor. The delivery layer is the under-secured attack surface.

- **Helm charts are code.** Go templating emits unrestricted Kubernetes YAML: a malicious chart can deploy privileged pods, cluster-admin bindings, and exfil sidecars, and its subcharts and chart-pulled images are a transitive dependency graph you did not choose.
- **Helm 2's Tiller was the archetypal bad delivery agent** — an in-cluster, usually cluster-admin, unauthenticated gRPC endpoint that collapsed every user's RBAC into its own. **Helm 3 removed it; Helm runs client-side as you.** The lesson recurs in operators and GitOps controllers.
- **Secure Helm by signing charts like images.** Native provenance (`.prov` + `helm verify`) exists but is PGP-bound; the modern default is **OCI charts + cosign** (Books 5–6), pinned by digest, served from an internal repo of **golden charts**, with `helm template` output policy-checked in CI and re-checked at admission.
- **Operators are privileged in-cluster software.** A CRD-plus-controller holds broad RBAC (often cluster-wide secret access); a compromised operator is a cluster compromise. Vet the source and image, **read the CSV and scope RBAC to least privilege**, constrain watched namespaces, and policy the operator's output at admission.
- **GitOps is a better delivery model that relocates risk.** Pull-based, declarative, auditable, reviewable — but the **repo and controller become tier-0**: the repo is the deploy button (Book 7 is now deployment security) and the controller is Tiller-shaped.
- **Turn on verification; do not overstate it.** Flux verifies signed **git commits** (`spec.verify`) and **cosign-signed OCI charts/images**; Argo CD verifies **GPG-signed commits** per AppProject and delegates image/artifact-signature checks to admission. Pin commits and digests; keep plaintext secrets out of git (Sealed Secrets / SOPS / External Secrets).
- **The chain of custody is the point.** Signed commit → verified chart → verified image → admission backstop, each link re-verified downstream, so no single compromised link deploys a malicious workload. Admission (Chapter 6) is the final gate on whatever the delivery chain produces.

Further reading

- **Helm documentation** — charts, dependencies, releases, and the security-model page (Helm 3 architecture, no Tiller). <https://helm.sh/docs/> and <https://helm.sh/docs/topics/security/>
- **Helm — Provenance and Integrity**. The `.prov` provenance file format, `helm package --sign`, and `helm verify / --verify`. <https://helm.sh/docs/topics/provenance/>
- **Helm — Registries (OCI)**. Storing and signing charts as OCI artifacts; `helm push / helm pull` against OCI registries. <https://helm.sh/docs/topics/registries/>
- **Sigstore cosign** — signing and verifying OCI artifacts (images *and* charts), keyless identity, Rekor. <https://docs.sigstore.dev/> and <https://github.com/sigstore/cosign>
- **Operator Framework / OLM**. ClusterServiceVersion, bundles, catalogs, install modes and operator groups, and the RBAC an operator declares. <https://olm.operatorframework.io/docs/> and <https://operatorhub.io/>
- **Kubernetes — Operator pattern and Custom Resources / CRDs**. <https://kubernetes.io/docs/concepts/extend-kubernetes/operator/> and <https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/>
- **Flux — security and verification**. `GitRepository/OCIRepository spec.verify` (Git commit signatures and cosign), Kustomization service-account impersonation. <https://fluxcd.io/flux/security/> and <https://fluxcd.io/flux/components/source/>
- **Argo CD — signature verification** (GPG-signed commits per AppProject) and **AppProject** restrictions. <https://argo-cd.readthedocs.io/en/stable/user-guide/gpg-verification/> and <https://argo-cd.readthedocs.io/en/stable/user-guide/projects/>
- **OpenGitOps principles** (declarative, versioned/immutable, pulled automatically, continuously reconciled). <https://opengitops.dev/>
- **Secrets in GitOps**: Sealed Secrets (<https://github.com/bitnami-labs/sealed-secrets>), Mozilla SOPS (<https://github.com/getops/sops>), and External Secrets Operator (<https://external-secrets.io/>).

- **Conftest / Open Policy Agent** for policy-checking rendered manifests, and **Kyverno CLI** for applying admission policies in CI. <https://www.conftest.dev/> and <https://kyverno.io/docs/>

Chapter 8 — Infrastructure as Code Supply Chain Risks

What this chapter covers. Every prior chapter in this book treated the *workload* as the thing you defend: images (Book 6, Chapter 1 — Container Images), the registries that store them (Book 6, Chapter 2 — Registries), and the admission gate that decides whether they run (Book 6, Chapter 6 — Admission Control and Policy Engines). This chapter descends one layer, to the thing that *creates the cluster in the first place*, and the network it lives in, and the IAM roles that grant it power, and the data stores it talks to. Infrastructure as Code — Terraform, OpenTofu, Pulumi, CloudFormation, CDK, Ansible, Crossplane — is program text that provisions your cloud. It is code, so it has a supply chain: reusable **modules**, plugin **providers**, persistent **state**, and an **execution pipeline** that runs with credentials powerful enough to build or destroy the whole account. Compromise any of those four and you do not compromise one service — you compromise the foundation every service stands on. That is the argument of the chapter, and everything else is mechanism and defense.

Learning goals — after this chapter you should be able to:

- Frame IaC as a **four-part supply chain** — modules, providers, state, execution identity — and say precisely what trust and what blast radius sits at each part.
- Explain the **Terraform/OpenTofu** specifics concretely: module sourcing and pinning, why a **provider is a privileged plugin** that runs with your cloud credentials, the role of `.terraform.lock.hcl`, registry **GPG signing**, and why **state** is a high-value target.
- Map the same structure onto **Pulumi, CloudFormation/CDK, Ansible, and Crossplane**, and see that the common risk is *reusable components executed at high privilege*.
- Deploy the defenses: **static misconfiguration scanning** (Checkov, tfsec/Trivy, Terrascan, KICS, cfn-nag), **policy as code** (OPA/Conftest,

Sentinel), **pinning and lockfile verification**, **internal registries/mirrors**, **plan/apply separation with approval gates**, and **drift detection**.

- Reason about the **IaC execution identity as a tier-0 credential** and design the pipeline so a compromised module or a poisoned pipeline step cannot silently hand your cloud account to an attacker.
-

Why IaC is a supply chain, and a uniquely dangerous one

Start from what IaC *is*, mechanically. A Terraform configuration is a set of declarations — "a VPC with these CIDRs, an EKS cluster, an RDS instance, an IAM role with this trust policy." Terraform reads the desired state, compares it to a recorded *actual* state, computes a diff (the **plan**), and then calls cloud APIs to make reality match (the **apply**). To call those APIs it must hold credentials, and because it provisions *everything* — networking, identity, compute, data — those credentials are, in the general case, near-omnipotent in the account. An IaC runner that can create IAM roles can create an IAM role that can do anything; the ceiling on its power is the account itself.

Now layer on reuse. Nobody writes a production VPC from primitives every time. They pull a **module** — someone else's parameterized bundle of resources — the same way an application pulls a library (Book 2 — Dependency Management and Open Source Risk). They pull **providers**, the plugins that translate Terraform's resource graph into AWS, GCP, Azure, Cloudflare, Datadog, or Kubernetes API calls. Both modules and providers are fetched from registries over the network, both have versions, both have transitive dependencies, and both are *executed* — modules by expansion into the resource graph, providers as actual binaries running on the IaC host. Reuse plus execution plus high privilege is the exact recipe that makes any supply chain dangerous; IaC just turns all three dials to maximum.

The consequence is a blast radius argument you should internalize before the mechanics. When an application dependency is malicious, the attacker gets that application's privileges — one service's database, one service's secrets. When an *IaC* dependency is malicious, the attacker gets a foothold in the layer that *defines* privilege. A single extra resource in a

module — an IAM role whose trust policy names an attacker's AWS account, a security-group rule opening `0.0.0.0/0` to a database port, an S3 bucket policy granting `*` — is provisioned by *your* trusted pipeline, with *your* blessed credentials, and it looks like infrastructure because it is infrastructure. This is why IaC belongs in the same tier-0 conversation as your signing keys and your root secrets (Book 4, Chapter 6 — Secrets Management in CI/CD; Book 1, Chapter 9 — Supply Chain Security in Distributed Backend Systems).

```
flowchart TB
    subgraph SC["The IaC supply chain"]
        M["Modules  
reusable IaC  
(like packages)"]
        P["Providers  
plugins to cloud APIs  
(binaries w/ creds + network)"]
        ST["State  
record of managed infra  
(holds secrets)"]
        EX["Execution identity  
cloud creds in CI  
(near-omnipotent)"]
    end
    M --> ENG["IaC engine  
plan + apply"]
    P --> ENG
    ST --> ENG
    EX --> ENG
    ENG --> INFRA["Provisioned infrastructure  
VPC / IAM / compute / data stores"]
    INFRA -->|"foundation for"| APPS["Every workload in the account"]
```

Read the diagram as four independent trust decisions feeding one execution. A defect in *any* of the four — a backdoored module, a malicious provider, a tampered state file, a stolen pipeline credential — flows through the engine into real infrastructure, and from there into everything that runs on it. The rest of the chapter walks the four, Terraform first because it is the concrete, dominant case, then the other ecosystems, then the defenses that harden all four.

Terraform and OpenTofu: the four parts in detail

Terraform is the reference implementation of this model, and OpenTofu — the Linux Foundation fork created after HashiCorp relicensed Terraform to the BUSL in 2023 — is wire- and language-compatible with it, sharing the same module and provider protocols, the same `.terraform.lock.hcl` format, and in most cases the same registry ecosystem. Everything in this section applies to both unless noted.

Modules: reuse and its trust boundary

A module is a directory of `.tf` files that other configurations call with a `source` and a set of input variables. The `source` argument is the trust boundary, and it takes several forms, each with a different threat profile:

```
# Public Terraform Registry – namespace/name/provider, version-
constrained
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "5.8.1"           # exact pin; not ">= 5.0" and not floating
  # ...
}

# Git source pinned to an immutable commit SHA (not a branch, not a
tag)
module "network" {
  source = "git::https://git.example.com/infra/network.git//modules/
vpc?ref=6f2c1e9a3b7d..."
}

# Internal/private registry
module "logging" {
  source = "app.terraform.io/acme/logging/aws"
  version = "2.3.0"
}
```

The risks track the parallels from Book 2 almost line for line:

- **Typosquatting and untrusted sources.** The public registry namespace is `<namespace>/<name>/<provider>`, and `terraform-aws-modules/...` is a well-known community organization. `terraform-aws-modules` or a look-alike personal namespace is a classic squat (Book 2, Chapter 3 — Dependency Confusion, Typosquatting, and Namespace Attacks). A registry module ultimately resolves to a git repository or archive; you are trusting whoever controls that source.

- **Backdoor infrastructure.** The dangerous module is not one that *fails* — it is one that provisions everything you asked for *plus one more resource*. An extra `aws_iam_role` with a cross-account `assume_role` trust policy. An extra ingress rule on a security group. An `aws_iam_user` with an access key exported to a place the attacker can read. Because the module also does its advertised job, the diff looks plausible and the review is where it must be caught — which is precisely why unreviewed module upgrades are so dangerous.
- **Unpinned versions.** `version = ">= 5.0"` or a git `?ref=main` means the `next terraform init` can silently pull a new, possibly compromised, release — the IaC form of the floating-dependency problem (Book 2, Chapter 2 — Versioning, Resolution, and Lockfiles). Note a sharp limitation: Terraform's `version` constraint on modules is honored *only* for registry sources. Git and HTTP sources are pinned solely by their `ref` /URL, so an immutable **commit SHA** is the only real pin for a git module — a tag can be moved.
- **Transitive modules.** Modules call modules. `terraform-aws-modules/vpc/aws` may itself invoke sub-modules. Your pin covers the top level; you inherit whatever *it* pins (or fails to pin) below. There is no module lockfile in Terraform — only the provider lockfile — so transitive module integrity rests entirely on the sources each module hard-codes.

Defenses: pin every module to an exact registry version or a commit SHA; vet modules before adoption and re-vet on upgrade (read the diff, not just the changelog); and route module consumption through an **internal/private registry** so teams pull from a curated, scanned catalog rather than the open internet — the IaC analog of an internal package registry (Book 2, Chapter 8 — Vendoring, Mirroring, and Internal Registries) and of the golden-base-image paved road (Book 6, Chapter 3 — Base Image Strategy).

Providers: the privileged plugin problem

Providers are where IaC's threat model diverges most sharply from ordinary dependency risk, and it is the part engineers most often underestimate. A Terraform provider is not configuration — it is an **executable binary**. During `terraform init`, Terraform downloads the provider plugin for your platform from a registry. During `plan` and `apply`, Terraform launches that binary as a subprocess and speaks to it

over a local gRPC protocol. The provider is the component that actually holds and uses your cloud credentials: it reads `AWS_ACCESS_KEY_ID` or assumes the runner's role, opens network connections to `*.amazonaws.com`, and makes the create/read/update/delete calls.

That means a malicious or compromised provider has, by construction, everything an attacker wants: code execution on the IaC host, the cloud credentials in that host's environment, and outbound network access. It does not need a clever exploit. It can read every environment variable and POST it to an attacker endpoint; it can make out-of-band API calls that never appear in your `.tf` files or your plan; it can provision backdoor resources that the plan does not show because the plan only reflects your declared configuration, not whatever else the provider chose to do with your credentials.

```
flowchart LR
    TF["terraform apply"] -->| "gRPC subprocess" | PR["Provider plugin  
(binary)"]
    PR --> ENVV["Cloud creds in env  
AWS_* / role"]
    ENVV --> PR
    PR -->| "declared CRUD calls" | API["Cloud API  
(intended)"]
    PR -->| "out-of-band calls  
not in the plan" | EXFIL["Attacker endpoint  
+ backdoor resources"]
    API --> INFRA["Your infrastructure"]
    EXFIL --> INFRA
```

The defense is a chain of provenance and pinning that Terraform builds in, and which you must not bypass:

Registry signing. Providers published to the public Terraform Registry are distributed with a GPG signature over the release's `SHA256SUMS` file, signed by the publisher's key (registered with the registry). On `terraform init`, Terraform downloads the sums file and its signature and verifies the signature against the registry-supplied public key before trusting the checksums. This binds the downloaded binary to a key the publisher controls — analogous in spirit to package signing, though it is publisher self-attestation via the registry, not a third-party transparency log.

The dependency lock file, `.terraform.lock.hcl`. This is the single most important control and the direct analog of `package-lock.json` or `Cargo.lock` (Book 2, Chapter 2). `terraform init` generates or updates it, and it records, per provider, the *exact resolved version*, the *version*

constraints that produced it, and — critically — a set of `h1:` / `zh:` **hashes** covering the provider package. `zh:` hashes are the zip-archive hashes taken from the registry's signed `SHA256SUMS`; `h1:` is Terraform's own content hash of the extracted package. On every subsequent `init`, Terraform verifies the provider it downloads against these recorded hashes and **fails** if they do not match. Commit this file to version control. It is what turns "we use AWS provider ~> 5.0" into "we use exactly this bit-for-bit provider, and CI will refuse anything else."

```
# .terraform.lock.hcl – generated; commit this to the repo
provider "registry.terraform.io/hashicorp/aws" {
  version      = "5.62.0"
  constraints = ">= 5.0.0, < 6.0.0"
  hashes = [
    "h1:8f2e...==",          # Terraform h1 content hash
    "zh:0alb...",            # signed registry archive
  ]
  hashes = [
    "zh:1c2d...",
    # ... one zh: per platform archive
  ]
}
```

Two operational rules make the lock file actually protective. First, when you add or change providers, run `terraform providers lock -platform=linux_amd64 -platform=darwin_arm64 ...` to record hashes for every platform your team and CI use — otherwise a developer on an unlocked platform fetches without verification and can repopulate the file with attacker-supplied hashes. Second, in CI run `terraform init` (and ideally a plan) and treat any lock-file modification as a change that must be reviewed and committed deliberately, never auto-accepted.

Trusted sources and private mirrors. For a stronger posture, run a **provider network mirror** (a filesystem or HTTP mirror configured via `provider installation` in the CLI config, or the `terraform providers mirror` command) so runners fetch only vetted providers from infrastructure you control, never directly from `registry.terraform.io`. This is the provider analog of an internal module registry and closes the "init reaches the internet" gap in locked-down environments.

State: the record that holds your secrets

Terraform's **state** is the file that records the mapping between your configuration and the real resources it manages — resource IDs,

dependency ordering, and, importantly, resource *attributes*. Many of those attributes are sensitive: an `aws_db_instance` password, a generated `tls_private_key`, an IAM access key, the contents of a secret. Terraform stores these in state in **plaintext** by default, regardless of whether you marked them `sensitive` in outputs — the `sensitive` flag hides them from CLI display, not from the state file. State is therefore a secrets store whether you intend it to be or not, and it is a first-class target.

Two distinct risks:

- **Confidentiality.** Anyone who can read the state can read those secrets. Local state on a laptop or, worse, committed to git, is a credential leak waiting to happen.
- **Integrity — state tampering.** State is Terraform's source of truth for "what exists." An attacker who can *write* state can lie to Terraform: mark a resource as already-configured when it is not (suppressing a corrective change), or — via `terraform import /state` manipulation — bring an attacker-controlled resource under management. Tampered state produces malicious or misleading plans; drift between state and reality is both an attack symptom and an attack tool.

The defenses are standard but non-negotiable for any shared use: **remote state** with encryption and access control. The common patterns are an S3 backend with **SSE-KMS** encryption and a DynamoDB table (or S3 native locking) for state locking, or **Terraform Cloud / HCP Terraform** which stores state encrypted at rest and never writes it to the runner's disk in the clear. Lock down the backend like a secrets store: KMS key policy and bucket policy restricting read/write to the IaC pipeline identity and break-glass admins only, versioning enabled (so you can detect and roll back tampering), and access logging on. Never commit state to source control; never leave it as local `terraform.tfstate` for anything shared.

The execution environment: where the crown-jewel credential lives

The fourth part is not a file — it is the pipeline that runs `terraform apply`, and it is the part with the highest privilege of all. This is a CI/CD problem, and Book 4 is its home, but the IaC-specific severity is worth stating plainly: **an attacker who can run code in the IaC apply**

pipeline controls your infrastructure. Not one build, not one artifact — the account.

That makes IaC pipelines the premium target for **Pipeline Poisoning / Poisoned Pipeline Execution (PPE)** (Book 4, Chapter 7 — Pipeline Poisoning: PPE, Cache, and Artifact Attacks). The generic PPE move — get attacker-controlled code to execute in a pipeline that holds privileged credentials, via a malicious PR that alters the CI definition, a poisoned pre-commit hook, or a compromised build step — is catastrophic here because the credential the pipeline holds is the cloud admin role. Two structural defenses matter most:

- **Plan/apply separation.** Run `terraform plan` on untrusted input (e.g., a PR from a fork or an unprivileged contributor) with **read-only** or *no* cloud credentials, and gate `terraform apply` behind merge to a protected branch and a human approval. The plan job can render the diff without the power to change anything; only the trusted, post-review apply job gets write credentials. This neutralizes the most common PPE vector — malicious code in a PR trying to reach the apply credential — because the PR path never touches it.
- **Short-lived, workload-identity credentials.** The apply job should assume a role via OIDC federation (GitHub Actions OIDC → AWS IAM role, GCP Workload Identity Federation, Azure workload identity) and receive a **short-lived** token scoped to that job, never a long-lived static cloud key sitting in a CI secret (Book 4, Chapter 6 — Secrets Management in CI/CD; Book 5, Chapter 4 — Keyless Signing and Workload Identity). Run applies in ephemeral, isolated runners (Book 4, Chapter 8 — Ephemeral and Isolated Build Environments) so a poisoned step cannot persist.

The blast radius, made concrete

Before the other ecosystems, hold on one worked example, because it is the whole reason the chapter exists. Suppose an internal module `acme/eks-cluster/aws`, used by forty teams, gains one extra resource in a new minor version — a version the teams pull because their `version` constraint was `~> 3.2` and nobody read the 3.4.0 diff:

```
# Slipped into an otherwise-legitimate module version
resource "aws_iam_role" "cluster_ops" {          # innocuous name
  name = "eks-cluster-ops"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect   = "Allow"
      Principal = { AWS = "arn:aws:iam::209876543210:root" } #
attacker's account
      Action    = "sts:AssumeRole"
    }]
  })
}

resource "aws_iam_role_policy_attachment" "cluster_ops_admin" {
  role           = aws_iam_role.cluster_ops.name
  policy_arn     = "arn:aws:iam::aws:policy/AdministratorAccess"
}
```

Every team that applies this module now has an admin-equivalent role in their account whose trust policy lets an *external* AWS account assume it. The attacker calls `sts:AssumeRole` from their account and has administrator access to forty of yours. No malware ran on a workload; no image was tampered with; your admission controller and image signing did their jobs perfectly and are entirely irrelevant, because the compromise happened one layer below them, provisioned by your own trusted pipeline.

```
flowchart LR
  UP["Upstream module  
v3.4.0 (backdoored)"] --> INIT["terraform init  
(unpinned ~> 3.2)"]
  INIT --> PLAN["plan (diff not read)"]
  PLAN --> APPLY["apply  
(cloud-admin creds)"]
  APPLY --> ROLE["Backdoor IAM role  
trust: attacker account"]
  ROLE --> STS["Attacker sts:AssumeRole"]
  STS --> OWN["Admin in your account  
x40 teams"]
```

The same shape applies to a security-group rule opening a management port to the internet, an S3 bucket made public, a VPC peering to an attacker VPC, or a Lambda that exfiltrates on a schedule. The lesson: at IaC's privilege level, the diff is the security boundary. Everything in the

defenses section exists to make sure a human or a policy engine sees that diff before it becomes real.

The other IaC ecosystems

Terraform is the archetype, but the four-part model — reusable components executed at high privilege — generalizes. What changes across ecosystems is *which* supply chain the reusable components come from.

Pulumi expresses infrastructure in general-purpose languages — TypeScript, Python, Go, C#. That is ergonomically pleasant and a supply-chain multiplier: a Pulumi program is an ordinary npm or PyPI project, so *the entire language package supply chain applies directly* (Book 2). Your `package.json` or `requirements.txt` pulls in dependencies with `postinstall` scripts, transitive packages, and typosquat risk — and this code runs *in the same process that holds your cloud credentials and drives provisioning*. Pulumi also has its own providers (many wrapping the Terraform providers) and a registry for reusable components. So Pulumi has two overlapping supply chains: the language ecosystem (defend with lockfiles, `npm ci` / pinned installs, SCA — Book 2, Chapter 6) and the provider/component ecosystem (defend with pinning and vetting). State and execution-identity concerns are identical to Terraform's; Pulumi state lives in the Pulumi Service or a self-managed backend (S3/GCS/Azure Blob) and holds secrets, though Pulumi encrypts secret values in state with a per-stack key by default — a meaningful improvement over Terraform's plaintext default.

CloudFormation and the CDK. CloudFormation is AWS's native template engine; a template is YAML/JSON and is executed by the AWS CloudFormation *service*, not on your runner, so there is no local privileged-plugin risk — but the template's supply chain lives in **CloudFormation modules/macros** and in the **CDK**. The AWS CDK compiles high-level constructs in a real language (TypeScript, Python, Java, Go) down to a CloudFormation template, so, like Pulumi, it inherits the npm/PyPI supply chain for its dependencies and for third-party **Construct Library** packages published to those registries. A malicious CDK construct can inject resources into the synthesized template exactly as a malicious Terraform module injects them into the plan. Execution privilege lives in the CloudFormation **service role** and the deploying

principal — treat it as the tier-0 credential the way you treat the Terraform apply role.

Ansible. Ansible is configuration management more than provisioning, but the model holds: reuse comes from **roles and collections** distributed via **Ansible Galaxy** (and Automation Hub). A role is downloaded and its tasks executed — often with `become /root` on managed hosts or with cloud credentials for cloud modules. So a malicious Galaxy role is code execution on your fleet or in your cloud. Pin roles/collections by version in `requirements.yml`, vet them, and prefer a curated internal Galaxy/Automation Hub over open Galaxy. Ansible has no lockfile-with-hashes equivalent as strong as `.terraform.lock.hcl`, so source curation carries more weight.

Crossplane makes infrastructure Kubernetes-native: you declare cloud resources as Kubernetes custom resources, and Crossplane **providers** (installed as OCI **packages**) reconcile them via cloud APIs, with reusable **Compositions** and **Configuration packages** playing the module role. This folds IaC supply chain into the container/OCI supply chain of this entire book — Crossplane packages are OCI images pulled from registries, so image provenance, signing, and admission (Chapters 1, 2, 5, 6) apply to them. The provider pods run with cloud credentials (via IRSA/workload identity), so they are the privileged-plugin risk wearing a Kubernetes hat.

Ecosystem	Reusable component	Its supply chain	Privileged execution
Terraform/ OpenTofu	Modules, providers	Registry + git; provider binaries	<code>apply</code> runner + cloud creds
Pulumi	Providers, components, language deps	npm/PyPI/Go + Pulumi registry	Program process + cloud creds
CloudFormation/ CDK	Modules/macros, CDK constructs	npm/PyPI (CDK) + CFN registry	CFN service role
Ansible	Roles, collections	Ansible Galaxy / Automation Hub	<code>become /root</code> or cloud creds

Ecosystem	Reusable component	Its supply chain	Privileged execution
Crossplane	Providers, Compositions	OCI packages (this book)	Provider pods + workload identity

The through-line: whatever the syntax, you are trusting reusable code from a registry and then running it — or something it emits — at a privilege level that can reshape your infrastructure. Defend accordingly, and reuse the controls you already built for the language and container supply chains.

Defenses I: static analysis and misconfiguration scanning

There are two distinct classes of IaC risk, and it is worth keeping them separate because the tooling differs. The first, the subject of this chapter's framing, is **supply-chain tampering**: a component or pipeline is compromised and provisions something malicious. The second, related and far more common in day-to-day practice, is **misconfiguration**: the IaC is exactly what its author intended, but what they intended is insecure — a security group open to the world, a public S3 bucket, an over-broad IAM policy, an unencrypted volume. Misconfiguration scanners target the second class, and they matter to the supply chain story because *they are also how you catch the backdoor a malicious module tried to slip past review* — an attacker-added public-ingress rule trips the same rule an honest mistake would.

Static IaC scanners parse your Terraform/CloudFormation/Kubernetes/etc. (usually the HCL/YAML directly, and better tools also the Terraform *plan* JSON) and evaluate it against a library of checks:

- **Checkov** (Prisma/Bridgecrew) — hundreds of built-in policies across Terraform, CloudFormation, Kubernetes, Helm, ARM, and more; supports custom policies in Python or YAML.
- **tfsec** — Terraform-focused; its checks and engine have been folded into **Trivy**, which is now the maintained path (Trivy also does image and dependency scanning — Book 6, Chapter 4 — Image Scanning

and Vulnerability Management — so it can be one tool across images and IaC).

- **Terrascan** (Tenable) — OPA/Rego-based policy engine over IaC.
- **KICS** (Checkmarx) — broad IaC coverage (Terraform, CloudFormation, Ansible, Kubernetes, Helm, Docker) with a large query set.
- **cfn-nag** — CloudFormation-specific linter for insecure patterns.
- **Cloud-native / runtime** — **AWS Config, Security Hub**, GCP Security Command Center, Azure Policy catch misconfiguration *after* apply, as a backstop for what static scanning missed or what drifted.

What they check, concretely: publicly-open security groups and NACLs (0.0.0.0/0 to sensitive ports); public S3 buckets / GCS buckets / blob containers; IAM policies with `Action: "*" / Resource: "*" or iam:PassRole` sprawl; missing encryption at rest (unencrypted EBS, RDS, S3); missing encryption in transit; disabled logging (no CloudTrail, no VPC flow logs, no access logs); publicly-exposed databases; and hardcoded secrets in IaC. Wire the scanner into pre-commit and CI so a finding fails the plan job before it reaches apply:

```
# CI step – fail the build on high/critical misconfig, and scan the
*plan*, not just source,
# so computed values and injected resources are visible
terraform plan -out=tf.plan
terraform show -json tf.plan > tf.plan.json
checkov -f tf.plan.json --framework terraform_plan \
    --compact --quiet --soft-fail-on LOW
# non-zero exit on MEDIUM+ fails the pipeline
```

Scanning the **plan JSON** rather than only the raw `.tf` is the higher-signal choice: it sees the fully-resolved graph, including resources contributed by modules and values computed from variables, which is exactly where a backdoor module would hide. Static source scanning can miss what a module injects; plan scanning cannot.

A caveat consistent with this book's realism: these tools find *known-pattern* misconfigurations. A sufficiently clever backdoor that uses only "valid" configuration — a legitimate-looking IAM role whose only sin is a trust-policy principal pointing at an attacker account — may pass every built-in check, because "cross-account trust" is a normal, sometimes-necessary pattern. That is why scanning is necessary but not sufficient,

and why the next layer — policy as code you write to your own invariants — and human review of the diff both remain essential.

Defenses II: policy as code for IaC

Misconfiguration scanners ship opinions; **policy as code** lets you encode *your organization's* invariants and gate IaC changes on them before apply. This is the same discipline as admission control for Kubernetes (Book 6, Chapter 6) and the same engines, applied to infrastructure definitions instead of runtime objects — and it is the connective tissue to Book 8, Chapter 4 — Policy as Code and Continuous Compliance.

OPA / Conftest against the Terraform plan. Conftest runs Rego policies against structured configuration. Feed it the plan JSON and you can assert arbitrary invariants over the *exact changes* about to be applied:

```
# policy/terraform.rego – deny public S3 and mandate encryption
package main

deny contains msg if {
    rc := input.resource_changes[_]
    rc.type == "aws_s3_bucket_public_access_block"
    not rc.change.after.block_public_acls
    msg := sprintf("S3 bucket %q must block public ACLs", [rc.address])
}

deny contains msg if {
    rc := input.resource_changes[_]
    rc.type == "aws_db_instance"
    rc.change.after.storage_encrypted != true
    msg := sprintf("RDS instance %q must have storage_encrypted =
true", [rc.address])
}

# Supply-chain-flavored invariant: no IAM role may trust an account
outside our org
deny contains msg if {
    rc := input.resource_changes[_]
    rc.type == "aws_iam_role"
    pol := json.unmarshal(rc.change.after.assume_role_policy)
    principal := pol.Statement[_].Principal.AWS
    not allowed_account(principal)
    msg := sprintf("IAM role %q trusts non-org principal %v", [rc.address, principal])
}
```

```
terraform show -json tf.plan > tf.plan.json
conftest test --policy policy/ tf.plan.json
```

That last policy is the important one for this chapter: it is a *supply-chain* control, not a generic misconfiguration check. Even if a malicious module injects the backdoor IAM role from the worked example, a policy that enumerates allowed trust principals rejects it at the gate, because the attacker's account is not on the list. Policy as code lets you defend against the class of tampering that pattern-scanners miss, *provided you write the invariant*.

Sentinel (HCP Terraform / Terraform Enterprise). HashiCorp's policy-as-code framework integrates into the Terraform Cloud run pipeline, evaluating policies against the plan between plan and apply, with **enforcement levels** — `advisory` (warn), `soft-mandatory` (block, but an authorized user may override), and `hard-mandatory` (block, no

override). It can read the plan, the config, the prior state, and run cost estimates. The enforcement-level model maps cleanly onto a rollout strategy: introduce a new policy as advisory, watch what it would have blocked, then promote to mandatory — exactly the audit-then-enforce pattern from admission control.

Checkov custom policies cover the same ground for teams standardized on Checkov, letting you add organization-specific rules alongside the built-ins.

Whichever engine, the placement is the same and it is the point: **policy runs on the plan, between plan and apply, and a failure blocks apply**. The infrastructure change is evaluated while it is still a proposed diff, never after it is real.

Defenses III: the IaC pipeline, end to end

Assemble the controls into one pipeline and the shape is an assembly line of gates, each one a chance to stop a bad change — whether the badness is a mistake, a misconfiguration, or a supply-chain compromise — before it reaches infrastructure holding a privileged credential.

```
flowchart LR
    PR["PR: IaC change  
(untrusted input)"] --> INIT["init  
(verify .terraform.lock.hcl,  
pinned modules)"]
    INIT --> PLAN["plan  
(read-only / no write creds)"]
    PLAN --> SCAN["scan  
Checkov / Trivy on plan JSON"]
    SCAN --> POL["policy  
Conftest / Sentinel on plan"]
    POL --> REV["human review  
read the diff"]
    REV --> APPROVE["approval gate  
protected branch"]
    APPROVE --> APPLY["apply  
(short-lived scoped creds)"]
    APPLY --> DRIFT["drift detection  
plan on schedule"]
    SCAN -- fail --> STOP1["block"]
    POL -- fail --> STOP2["STOP"]
    REV -- reject --> STOP3["STOP"]
```

Walk the gates:

1. **init with verification.** `terraform init` verifies providers against `.terraform.lock.hcl` hashes and fails on mismatch; module pins are honored. Any change to the lock file is itself a reviewable event. This is the supply-chain integrity gate.
2. **plan without write power.** The plan runs with read-only or no cloud credentials, especially on PRs from untrusted contributors — the PPE mitigation. It emits the diff as an artifact.
3. **scan.** Checkov/Trivy on the plan JSON, failing on high-severity misconfiguration.
4. **policy.** Conftest/Sentinel on the plan, enforcing organizational invariants (no public storage, mandatory encryption, only-org IAM trust principals, approved regions, required tags).
5. **human review.** A person reads the *diff* — the `+/ - / ~` resource changes — because IaC is code and reviewing it is reviewing infrastructure (Book 7 — Source, Code, and Insider Threat Security; in particular Chapter 3 — Branch Protection, Review, and Two-Person Rules). This is the catch-all for the clever backdoor that passed the automated gates.
6. **approval + apply.** Only after merge to a protected branch and explicit approval does `apply` run, assuming a short-lived, scoped role via OIDC — never a long-lived cloud-admin static key.
7. **drift detection.** On a schedule, run `terraform plan` (with read access) against production. A non-empty diff means reality diverged from the declared state — an out-of-band change, which is either an unmanaged human edit or the fingerprint of tampering. Drift detection is IaC's equivalent of continuous compliance and a genuine tamper-detection signal.

A minimal apply gate in GitHub Actions, showing plan/apply separation and OIDC:

```
# .github/workflows/terraform.yml (excerpt)
permissions:
  id-token: write          # OIDC for short-lived cloud creds
  contents: read

jobs:
  plan:
    runs-on: ephemeral-runner          # runs on PRs – NO write creds
    steps:
      - uses: actions/checkout@<pinned-sha>
      - run: terraform init              # verifies .terraform.lock.hcl
      - run: terraform plan -out=tf.plan
      - run: terraform show -json tf.plan > tf.plan.json
      - run: checkov -f tf.plan.json --framework terraform_plan
      - run: conftest test --policy policy/ tf.plan.json

  apply:
    runs-on: ephemeral-runner          # runs only on main, after approval
    if: github.ref == 'refs/heads/main'
    needs: plan
    environment: production            # GitHub environment = required
    reviewers gate
    runs-on: ephemeral-runner
    steps:
      - uses: actions/checkout@<pinned-sha>
      - uses: aws-actions/configure-aws-credentials@<pinned-sha>
        with:
          role-to-assume: arn:aws:iam::111122223333:role/tf-apply #
            scoped, OIDC-assumed
          aws-region: us-east-1
      - run: terraform init
      - run: terraform apply -auto-approve
```

The `environment: production` line is doing real work: GitHub environments support required reviewers, so `apply` physically cannot start until a designated human approves the run — the approval gate, enforced by the platform rather than convention. Action versions are pinned by SHA (Book 4, Chapter 5 — Hardening GitHub Actions), because your CI actions are themselves a supply chain.

The controlled execution plane

Rolling all of this yourself in raw CI is possible but fragile. The mature pattern is a **dedicated IaC execution plane** that centralizes the gates, the state, and the credentials:

- **HCP Terraform / Terraform Enterprise** — managed runs, remote encrypted state, Sentinel/OPA policy sets, run approvals, and a private module registry, all in one control plane.
- **Atlantis** — self-hosted; listens on PRs, runs `plan` and comments the diff back on the PR, and runs `apply` on an approving comment, keeping the whole flow in the PR where review happens.
- **Spacelift** — a commercial platform with policy (OPA), drift detection, and private worker pools.

The value is the same as a paved-road build platform (Book 4, Chapter 10 — Designing a Secure Build Platform at Scale): the privileged credential lives in *one* controlled, audited place instead of scattered across team pipelines; state is centrally encrypted; policy is enforced uniformly; and every run is logged. This is the IaC analog of a golden-image paved road (Book 6, Chapter 3) and of admission control as a central enforcement substrate (Book 6, Chapter 6) — you concentrate the risky capability so you can afford to guard it well.

The distributed-systems lens

IaC is where the "many services, many teams, many repos" reality of backend engineering meets the foundation those services run on, and every theme of this suite reappears sharpened.

Concentration and propagation. IaC provisions the fleet's foundation, so a compromised module, provider, or pipeline propagates to *all* infrastructure it manages. A backdoored internal module used by forty teams is forty compromised accounts from one change (Book 1, Chapter 9 — Supply Chain Security in Distributed Backend Systems). The same concentration that makes reuse efficient makes compromise systemic. The mitigation is not to abandon reuse — it is to make the reused thing a *paved road*: internal module and provider registries/mirrors, pinning, scanning, and policy gates, so that the standardized path is also the

secured path, and the concentration works *for* you (one place to vet, one place to patch) instead of against you.

The IaC execution identity is a crown-jewel credential. It sits alongside your signing keys and root secrets in the tier-0 tier. Treat it as such: **workload identity** and short-lived tokens over long-lived static keys (Book 4, Chapter 6; Book 5, Chapter 4 — Keyless Signing and Workload Identity), **least privilege** scoped as tightly as the reconciliation actually requires (genuinely hard for a tool whose job is to create IAM, but at minimum separate plan-read from apply-write, scope per environment, and deny the ability to touch the logging/audit and the IaC state backend itself), and **never** a long-lived cloud-admin key sitting in a CI variable. The apply pipeline holding a static `AdministratorAccess` key is the single worst credential-management decision available to an infrastructure team, and it is depressingly common.

Gates as admission control for infrastructure. Plan review, policy evaluation, and approval gates are to infrastructure changes what admission webhooks are to Kubernetes objects (Book 6, Chapter 6): the last checkpoint that can reason about the *content* of a proposed change before it becomes real, and the point where you enforce "no public buckets, mandatory encryption, only-org IAM trust" as hard invariants. Drift detection is the continuous-verification companion — the audit that reality still matches the declared, policy-passing state. Together they make infrastructure changes governable at fleet scale instead of trusted per-engineer.

Reviewing IaC is reviewing infrastructure. Because IaC is code, the entire discipline of source security applies (Book 7): branch protection, required review, two-person rules on the highest-blast-radius repos, commit signing, and secret scanning on the IaC repo itself. But the review has a heightened stakes profile — a subtle diff in an app repo risks a bug; a subtle diff in the IaC repo risks the account. Reviewers of infrastructure changes should read the *plan diff*, not just the code diff, and should be specifically wary of new IAM trust relationships, new network exposure, and changes to the state backend or the pipeline definition itself.

The controlled execution plane as the coordination point. In a distributed system you centralize the things that must be consistent and guarded — service discovery, config, secrets. The IaC execution plane (HCP Terraform, Atlantis, Spacelift) is that coordination point for

infrastructure change: the one place the crown-jewel credential lives, state is encrypted, policy is uniform, and every apply is attributable. Standardizing teams onto it is how you move from "every team's CI can nuke a cloud account" to "infrastructure change is a governed, observable, policy-gated pipeline."

Key takeaways

- **IaC is a four-part supply chain — modules, providers, state, execution identity — executed at near-omnipotent privilege.** Compromise any part and you compromise the foundation, not one app. This is the blast-radius argument, and it is why IaC deserves tier-0 treatment.
- **A provider is a privileged plugin, not config.** It runs as a binary during plan/apply with your cloud credentials and network access; a malicious one can exfiltrate and provision out-of-band, invisibly to the plan. Defend with registry GPG signing, `.terraform.lock.hcl` hash verification (commit it; lock all platforms), and private mirrors.
- **Pin modules to exact versions or commit SHAs.** For git sources, the SHA is the only real pin — tags move, and `version` constraints are honored only for registry modules. There is no module lockfile, so transitive-module integrity rests on curation and internal registries.
- **State is a plaintext secrets store and a tamper target.** Use encrypted remote state (S3+KMS, HCP Terraform), lock down access like a secret, enable versioning, and never commit state to git.
- **The apply pipeline is the premium PPE target.** Separate plan (read-only, untrusted input) from apply (post-review, short-lived OIDC-scoped creds); never store a long-lived cloud-admin key in CI.
- **Two risk classes, complementary tools.** Misconfiguration scanners (Checkov, Trivy/tfsec, Terrascan, KICS, cfn-nag) catch insecure-but-intended config *and* many injected backdoors; scan the **plan JSON**, not just source. Policy as code (Conftest/OPA, Sentinel) enforces *your* invariants — including supply-chain ones like "no IAM role may trust a non-org account" — that generic scanners miss. Neither replaces human review of the diff.
- **The gate order is plan → scan → policy → review → approval → apply → drift-detect.** Every change is evaluated while it is still a

proposed diff, and a controlled execution plane concentrates the credential, state, and policy so you can guard them well.

Further reading

- HashiCorp, "Terraform: Dependency Lock File" and "Provider Requirements" — official documentation for `.terraform.lock.hcl`, hash types, and `terraform providers lock`.
- HashiCorp, "Terraform Registry: Provider Signing" and "Module Sources" — GPG signing of providers and the `source / version` semantics for modules.
- OpenTofu documentation — provider/module protocols, lock file, and the fork's compatibility with Terraform (opentofu.org/docs).
- HashiCorp, "Manage Terraform State" and "Backends: S3" — remote state, encryption, and locking.
- HashiCorp, "Sentinel" documentation — policy-as-code enforcement levels and the run-pipeline integration.
- Open Policy Agent / Conftest documentation, and "Terraform plan testing with OPA" — Rego against plan JSON.
- Checkov (bridgecrewio/checkov), Trivy (aquasecurity/trivy — the maintained home of tfsec), Terrascan, and KICS project documentation — check catalogs and CI integration.
- Pulumi documentation, "Secrets" and "How Pulumi Works" — encrypted state secrets and the program execution model.
- AWS, "AWS CDK" and "CloudFormation modules" documentation — construct libraries and template supply chain.
- Crossplane documentation, "Packages" and "Providers" — OCI-distributed providers and Compositions.
- Cross-references in this suite: Book 2, Chapter 2 — Versioning, Resolution, and Lockfiles; Book 2, Chapter 8 — Vendoring, Mirroring, and Internal Registries; Book 4, Chapter 6 — Secrets Management in CI/CD; Book 4, Chapter 7 — Pipeline Poisoning: PPE, Cache, and Artifact Attacks; Book 5, Chapter 4 — Keyless Signing and Workload

Chapter 9 — Serverless, Managed Services, and the Cloud Provider Chain

What this chapter covers. The previous eight chapters of this book treated the artifact you ship — a container image — as a thing you own end to end: you choose the base (Book 6, Chapter 3 — Base Image Strategy), you scan it (Chapter 4), you sign it (Chapter 5), and an admission controller you configure decides whether it runs (Chapter 6). Serverless and managed services move that boundary. When you deploy an AWS Lambda function, a Google Cloud Run job, a managed Postgres, or a Cognito user pool, a large part of the stack that you used to own — the operating system, the language runtime, the patch cadence, the host kernel, the hypervisor — becomes the provider's responsibility. That is a genuine security win: there is no base-image CVE for you to patch in the Lambda runtime, because you do not build or own that runtime. But the supply chain does not disappear when the boundary moves. It splits. On your side of the line you still own your code and its dependencies (Book 2 — Dependency Management and Open Source Risk), and you have acquired a new, large, mostly-opaque dependency: the cloud provider's own supply chain — their runtime, their control plane, their marketplace, their build systems. This chapter is about where that boundary actually sits for FaaS and managed services, what supply-chain controls survive on your side of it, and how to reason about the irreducible trust you place in the provider on the other side.

Learning goals — after this chapter you should be able to:

- State the **shared-responsibility shift** precisely for FaaS and managed services: what the provider now owns (OS, runtime, patching, infra), what you still own (code, dependencies, config, IAM), and the fact that you now *depend on* the provider's supply chain as a new upstream.
- Map the **serverless supply chain** concretely on AWS Lambda, Google Cloud Functions, and Azure Functions — the deployment

package, dependency bundling, **Lambda layers** as a shared-dependency vector, the managed runtime as a visibility gap, and **container-image Lambda** as an OCI artifact you scan and sign like any other.

- Describe **Lambda code signing with AWS Signer** accurately — what it signs, when it verifies, and its scope limits — as the FaaS equivalent of image signing (Book 6, Chapter 5).
 - Treat **managed services and third-party SaaS** in the runtime path as supply-chain dependencies, and understand the **polyfill.io** and **Ledger connect-kit** incidents as runtime third-party compromises.
 - Reason about the **cloud provider as an accepted, attested trust relationship**, vet marketplace and community images as third-party artifacts, and document the shared-responsibility boundary per service for both security and incident response.
-

The boundary moves; it does not disappear

Start with the mental model everyone already carries and then correct it. The **shared responsibility model** — AWS's phrasing, but Google, Microsoft, and every other provider publish their own version of the same idea — draws a line between *security of the cloud* (the provider's job) and *security in the cloud* (yours). For raw IaaS the line sits low: on EC2 you own the guest OS, every package on it, the runtime, the app, and its dependencies; AWS owns the hypervisor, the physical host, the network fabric, and the facility. As you move up the abstraction ladder — from EC2 to a managed container platform to Lambda to a fully managed SaaS-like service — the line rises, and the provider absorbs more of the stack.

The supply-chain consequence is the part that is usually left implicit. Everything below the line is a slice of the supply chain the provider now operates *on your behalf*. When AWS patches the Amazon Linux base that the Lambda Python runtime is built on, that is a base-image update you did not have to schedule, test, or roll out. That is real and it is good: a meaningful fraction of the CVE churn that dominates Chapter 4's vulnerability-management treadmill simply becomes someone else's treadmill. But two things remain true. First, everything *above* the line is still yours, unchanged: your handler code, the libraries it imports, the

transitive dependency tree behind them, the same malicious-package and known-vulnerability risks Book 2 spent ten chapters on. A dependency-confusion attack (Book 2, Chapter 3) or a compromised transitive package (Book 1, Chapter 4 — event-stream, ua-parser-js) lands in your Lambda deployment package exactly as it would in a container. Second, the stack *below* the line did not vanish — it became a dependency. You now consume the provider's runtime the way you consume any upstream, except you cannot see inside it, cannot generate an SBOM for most of it, and cannot verify it independently. You have traded operational burden for opacity.

```
flowchart TB
    subgraph YOURS["Your supply chain (still yours)"]
        CODE["Handler code"]
        DEPS["Your dependencies  
(libraries, transitive tree)"]
        CFG["Config + IAM roles  
event source wiring"]
    end
    subgraph SHARED["Shared / provider-operated"]
        LAYER["Shared layers  
(you pick, provider hosts)"]
    end
    subgraph PROVIDER["Provider's supply chain (now a dependency)"]
        RT["Managed language runtime"]
        OS["Guest OS / base image"]
        MICROVM["Isolation: microVM / kernel"]
        HV["Hypervisor + host + facility"]
    end
    YOURS -->|"you build, scan, sign"| SHARED
    SHARED -->|"runs on"| PROVIDER
    PROVIDER -->|"you trust, cannot fully verify"| ATTEST["Compliance  
attestations  
SOC 2 / ISO 27001 / FedRAMP"]
```

The rest of the chapter walks this diagram from top to bottom: your code and dependencies first (where nothing has changed and everything from Book 2 still applies), then the shared and managed runtime layers (where the interesting new mechanics live), then the provider trust relationship at the bottom (where the only honest answer is "documented, accepted risk backed by attestation").

The serverless supply chain, concretely

What you actually ship

A serverless function is deployed in one of two forms, and the form determines which chapters of this book apply to it.

The first form is a **deployment package**: a ZIP archive on AWS Lambda (up to 250 MB unzipped, 50 MB zipped for direct upload), a source or ZIP upload on Google Cloud Functions, or a package deployed through the Functions host on Azure. The archive contains your handler plus your bundled dependencies — `node_modules/`, a `site-packages` / vendored directory, compiled Go or Rust binaries. This is your supply chain in the most literal sense: whatever your package manager resolved is now sitting in that ZIP, and it was resolved by the same npm/pip/Go tooling, from the same registries, with the same trust model, that Book 2 dissected. There is no container base image here, but there is absolutely a dependency tree, and it deserves the same treatment: a lockfile with pinned, hashed versions (Book 2, Chapter 2 — Versioning, Resolution, and Lockfiles), SCA scanning (Book 2, Chapter 6), and an **SBOM generated from the built package**, not from the source repo. Book 3, Chapter 8 — SBOMs for Services covers exactly this: `syft` and equivalent tools can inventory a Lambda ZIP or a function source tree, and that SBOM is what feeds your vulnerability correlation and VEX workflow (Book 3, Chapter 6). The absence of a Dockerfile does not mean the absence of a bill of materials — it means the bill of materials is easier to overlook.

The second form is a **container image** — Lambda has supported OCI images (up to 10 GB) since 2020, and Cloud Run, Azure Container Apps, and Google Cloud Functions (2nd gen, which builds a container under the hood) are container platforms by construction. When your function is an image, *everything* in Book 6 Chapters 1–5 applies unchanged: it has a base image you should choose deliberately (distroless or a minimal provider base — Chapter 3), layers you can inspect (Chapter 1), a digest you scan (Chapter 4) and sign (Chapter 5), and a registry it lives in (Chapter 2 — for Lambda, an ECR repository in your account). Container-image serverless is the case where the serverless supply chain is *least* special: it is a container supply chain that happens to be invoked by an event instead of scheduled by Kubernetes.

Deployment model	What you own	What the provider owns	Supply-chain controls available to you
ZIP / package Lambda or function	Handler + bundled deps	Language runtime, OS, microVM, host	Lockfile + hashes; SCA; SBOM of the ZIP; AWS Signer code signing ; least-privilege IAM
Container-image Lambda / Cloud Run / Container Apps	Image (base + deps + code)	Runtime base beneath your base; microVM/host	Full Book 6 Ch 1–5: base choice, scan, cosign sign/verify , digest pinning, registry controls
Lambda layer (shared dependency)	The layer you build/own	Hosting of the layer	Build reproducibly; pin by version ARN; SBOM the layer; avoid unvetted third-party layers
Managed service (DB, queue, auth, gateway)	Configuration (IaC) + your integration code	The service software + its supply chain	IaC security (Book 6 Ch 8); secure config; least-privilege; rely on provider attestation
Third-party SaaS in runtime path	The decision to depend + integration	The SaaS vendor's entire stack	Vendor risk (Book 8 Ch 3); SRI/pinning for scripts; egress monitoring; SaaS BOM

Lambda layers: shared dependencies with a shared blast radius

A **Lambda layer** is a ZIP archive of libraries, a runtime, or other content that Lambda extracts to `/opt` in the execution environment before your handler runs. A function can attach up to five layers, and their contents are merged onto the filesystem in order. The intent is DRY: share a fat dependency (the AWS SDK, a data-science stack, an image-processing library, a custom runtime) across many functions instead of bundling it into each ZIP. Extensions — long-running sidecar processes for

observability, secrets injection, or security agents — are also commonly delivered as layers.

Structurally, a layer is to a function what a base image is to a container: shared, reused code that executes with the function's privileges but that the function author did not necessarily write or review. That makes it the same class of supply-chain vector. A layer you build yourself carries your own dependency risk (bundle a malicious npm package into a layer and every function that attaches it inherits the package). A **third-party layer** — published by a vendor, a community author, or a partner, referenced by ARN — is code you are executing without owning, and a compromise of that layer's publisher poisons every function across every account that references it. The observability agents shipped as extension layers are a particularly sharp example: they run as separate processes in the execution environment, often with network egress, precisely so they can see and export your telemetry — which is exactly the position a malicious extension would want.

Layers are versioned and referenced by a fully qualified ARN that ends in a version number (`arn:aws:lambda:us-east-1:123456789012:layer:my-layer:7`). That version is immutable, which is your friend: pin to a specific version ARN, never to a floating alias, so that a re-publish of the layer cannot silently change what your functions execute — the same digest-pinning discipline Chapter 1 argued for images, applied to layers. The defenses are the familiar ones: prefer layers you build in your own pipeline over third-party layers; when you must use a third-party layer, treat its publisher as a vendor (Book 8, Chapter 3 — Vendor and Third-Party Software Risk); build layers reproducibly and SBOM them like any other artifact; and re-publish trusted third-party layer contents into your own account rather than referencing a foreign account's ARN across a trust boundary.

```

flowchart LR
    subgraph BUILD["Build / deploy time"]
        SRC["Handler source"]
        DEP["Dependency resolution  
(npm / pip / go)"]
        PKG["Deployment package  
ZIP or OCI image"]
        LYR["Lambda layers  
(shared deps / extensions)"]
        SIGN["Sign: AWS Signer (ZIP  
or cosign (image))"]
    end
    subgraph RUN["Run time (provider environment)"]
        VERIFY["Signature verification  
(code signing config)"]
        RUNTIME["Managed language runtime"]
        EXEC["Execution environment  
(Firecracker microVM)"]
        ROLE["IAM execution role  
(function privileges)"]
    end
    SRC --> DEP --> PKG --> SIGN --> VERIFY
    LYR --> VERIFY
    VERIFY --> RUNTIME --> EXEC
    ROLE -.->|"grants power to"| EXEC

```

The managed runtime: the visibility gap

Beneath your handler and your layers sits the **managed runtime**: the language runtime (a specific Node.js, Python, Java, .NET, Go, or Ruby version), the bootstrap that invokes your handler, and the Amazon Linux (or provider equivalent) userland it all runs on, inside a Firecracker microVM on AWS's side. You do not build this, you do not patch it, and you cannot fully see into it. That is the trade at its purest. The upside is concrete: when a CVE lands in the OpenSSL that ships with the managed runtime, AWS patches the runtime fleet and you inherit the fix without a deploy — no base image to rebuild, no rollout to coordinate, none of Chapter 4's operational load for *that* layer. The downside is equally concrete: you cannot generate a complete, authoritative SBOM of the managed runtime, so there is a slice of your running software's bill of materials that is permanently below your visibility line (Book 3, Chapter 7 — SBOM Quality, Completeness, and Limitations; Chapter 8 — SBOMs for Services names this gap explicitly). Providers publish the runtime's included libraries and patch bulletins, and you can and should consume those, but that is *their* attestation about *their* software, not something

you verified. This is the trust relationship of Book 1, Chapter 6 (Trust, Threat Models, and the Economics of Supply Chain Risk) made physical: you have chosen to trust the provider's runtime supply chain, and the correct posture is to make that choice explicit rather than to pretend the runtime is not part of your attack surface.

One operational wrinkle that is genuinely a supply-chain event: **runtime deprecation**. Providers do not maintain old language runtimes forever. AWS publishes a deprecation schedule; once a runtime (say, an old Node.js or Python minor) reaches end of support, Lambda first blocks you from *updating* functions on it and later can block *creating* them, and the runtime stops receiving security patches. The provider is, in effect, force-upgrading your dependency on their schedule, not yours. For a fleet of hundreds of functions this is a recurring migration obligation you must track and budget for — the managed-runtime analogue of a base image reaching EOL (Chapter 3), except the timeline is the provider's to set.

Deployment integrity: AWS Signer and Lambda code signing

For container-image functions the signing story is the one you already know: sign the image with cosign, record it in a transparency log (Book 5, Chapter 3 — Sigstore Architecture; Chapter 5 — Transparency Logs), and verify the signature before the image is allowed to run. For **ZIP-package** Lambda functions there is a purpose-built equivalent: **AWS Signer** and **Lambda code signing**.

Mechanically: you create a **signing profile** in AWS Signer, which is backed by a code-signing certificate AWS manages. When you build a deployment package, you sign it with that profile; Signer produces a signed object stored in S3. On the Lambda side, you attach a **code signing configuration** to the function. That configuration lists the ARNs of the signing profiles you trust and sets two policies: what to do on an **untrusted signature** (a package not signed by a listed profile, or an altered package whose signature no longer validates) and what to do on an **expired or revoked** signature. Each policy is either **Warn** (allow but log) or **Enforce** (reject the deployment). With **Enforce**, Lambda **verifies the signature at the point of deployment** — when you publish a new version or update the function's code — and refuses to accept a package that is unsigned, signed by an untrusted profile, or tampered with after signing. This is the FaaS realization of the same control Chapter 5 built for images and Chapter 6 enforced at admission: a cryptographic check

that the code about to run is the code an authorized identity produced, gating the moment it enters the runtime.

Two accuracy caveats you must carry, because getting them wrong leads to a false sense of coverage. First, **Lambda code signing applies to ZIP deployment packages, not to container-image functions**. If your function is an OCI image, AWS Signer's Lambda code-signing flow does not cover it; you secure it as a container (sign the image, verify it in your delivery pipeline and via your registry/policy controls — Book 6, Chapters 5 and 6). Do not assume a code-signing configuration protects an image-based function. Second, signing establishes *authenticity and integrity* — that a trusted identity produced this exact package — it does **not** establish that the package's *contents* are free of vulnerable or malicious dependencies. A signed ZIP with a compromised npm dependency is a faithfully signed compromised artifact. Signing and SCA/SBOM are orthogonal controls; you need both, for the same reason a signed container still has to pass a scan.

Event sources and the IAM execution role as blast radius

A function does not run in a vacuum; it is wired to **event sources** (an API Gateway route, an SQS queue, an S3 bucket notification, an EventBridge rule, a Kinesis stream) and it runs with an **IAM execution role** that grants it whatever permissions its code needs — read this bucket, write that table, publish to this topic. That role is the function's blast radius, and it is entirely on your side of the responsibility line. If a supply-chain compromise reaches into the function — a malicious dependency, a poisoned layer, a tampered package that somehow bypassed signing — the attacker's reach is bounded by exactly what that role can do. An over-broad role (`dynamodb:*` on all tables, `s3:*` on all buckets, or the catastrophic `AdministratorAccess`) converts a single compromised function into an account-wide incident; a least-privilege role scoped to the specific resources and actions the function genuinely uses contains the same compromise to one bucket or one table. This is the serverless instance of the argument Book 6, Chapter 8 made about IaC execution identity and Book 1, Chapter 9 made about blast radius generally: at fleet scale, with thousands of functions, the aggregate over-provisioning of execution roles is your serverless attack surface. Least-privilege per function is not hygiene theater here — it is the primary containment mechanism for exactly the supply-chain failures the rest of this chapter is about.

Managed services as supply-chain dependencies

Serverless functions are the most visible managed offering, but the larger and quieter dependency is the fleet of **managed services** a modern backend leans on: RDS/Aurora/Cloud SQL for databases, SQS and Pub/Sub and managed Kafka for queues, ElastiCache/Memorystore for caches, Cognito and Auth0 and Azure AD B2C for authentication, API Gateway for the edge, managed OpenSearch, managed secrets stores, and so on. You do not run the software behind any of these. That means you have taken on three distinct supply-chain dependencies for each one, and it helps to name them separately because they are defended differently.

The first is the **provider's own software supply chain**. The managed database is running the provider's build of PostgreSQL or their proprietary engine, patched by their pipeline, on their infrastructure. A compromise in *their* build or patch process would reach you, and there is essentially nothing you can do to detect or prevent it directly — this is the same irreducible trust as the managed runtime, and it is discharged the same way: through the provider's compliance attestations and your acceptance of the residual risk. The second is the **configuration supply chain**: managed services are provisioned and configured by IaC (Book 6, Chapter 8 — Infrastructure as Code Supply Chain Risks), and a poisoned module or a misconfigured resource is entirely yours to own. The single most common managed-service breach is not a provider compromise at all; it is a storage bucket, database, or search cluster left publicly accessible or with default weak settings. Secure defaults and IaC misconfiguration scanning (Checkov, Trivy, Terrascan — Chapter 8) are your controls here, and they matter far more day to day than the exotic provider-compromise scenario.

The third dependency is the sharpest one for supply-chain purposes: **managed services that run your code**. A managed database executes stored procedures and functions you wrote. Edge platforms run functions you deploy — **Lambda@Edge** and CloudFront Functions on AWS, Cloudflare Workers, Fastly Compute — pushing your code and its dependencies out to hundreds of points of presence. Managed event platforms run your transformation logic. In every one of these, your dependency tree has been shipped into an environment the provider operates but that executes code you authored, which means your Book 2 dependency risk now runs at the edge, in the database, or in the event

pipeline — often with weaker SBOM and scanning coverage than your core services get, precisely because these deployment surfaces are easy to forget when you inventory "your software." The rule is simple and frequently violated: if it runs your code, it is in scope for your SBOM, your SCA, and your signing — no matter how managed the platform around it is.

Third-party SaaS in the runtime path

There is a category of dependency that never appears in your `package.json`, your Dockerfile, or your Terraform, yet executes in the critical path of every request: **third-party SaaS invoked at runtime**. An external auth provider your login flow redirects through. A payments API your checkout calls synchronously. A feature-flag service your code polls to decide behavior. An observability agent that runs in-process and exports telemetry. And — the most dangerous of all because it executes in your users' browsers with your origin's privileges — **third-party scripts** loaded from a vendor's CDN. These are runtime supply-chain dependencies in the fullest sense (Book 1, Chapter 9 — Supply Chain Security in Distributed Backend Systems; Book 3, Chapter 8): you did not build them, you cannot scan them, and if the vendor turns malicious or is compromised, their code runs with your application's trust, on your users, immediately, with no deploy on your part.

Two incidents make this concrete and should anchor how you think about the category.

Ledger connect-kit, December 2023. `@ledgerhq/connect-kit` is a library that decentralized applications embed to let users connect Ledger hardware wallets. On 14 December 2023, an attacker phished a former Ledger employee whose npm publishing access was still live, and pushed malicious versions of the package (in the 1.1.x line) to npm. Crucially, many dApps loaded connect-kit not as a pinned npm dependency but via a **CDN**, pulling the latest published build at runtime — so the malicious code propagated into live front ends within minutes of publication. The injected payload was a wallet-draining script that prompted users to approve transactions sending their assets to the attacker. It was live for a few hours and drained on the order of hundreds of thousands of dollars before Ledger and the ecosystem rotated the package and CDN. This is simultaneously an npm account compromise (Book 2, Chapter 4 — Malicious Packages) and a runtime third-party-script compromise: the

CDN loading pattern is what turned a bad npm publish into an immediate live-site incident.

polyfill.io, June 2024. For years, `cdn.polyfill.io` served JavaScript polyfills to hundreds of thousands of sites that embedded a `<script src="https://cdn.polyfill.io/...">` tag — a classic third-party-script dependency loaded fresh on every page view. In early 2024 the domain and its associated GitHub project changed hands to a new operator (widely reported as a Chinese company, Funnul). In June 2024 the service began serving **malicious code** to a subset of visitors — targeting mobile users, dynamically injecting redirects to scam and gambling sites, and evading detection by varying the payload. The original polyfill author publicly warned that no site should ever have trusted the domain after the handover, and Cloudflare and Fastly stood up clean mirrors so sites could repoint. The mechanism is the one that matters: a script you embed by URL is a standing grant of code execution to whoever controls that URL, and control can change hands or be compromised without any change on your side. The number of affected sites was in the six figures.

```
flowchart TB
    USER["End user / browser"]
    subgraph YOU["Your application"]
        APP["Your service"]
        FE["Your front end"]
    end
    end
    subgraph SAAS["Runtime third-party dependencies"]
        AUTH["Auth provider  
(Cognito / Auth0)"]
        PAY["Payments API"]
        FLAG["Feature flags"]
        OBS["Observability agent  
(in-process)"]
        CDNJS["Third-party script  
(CDN-loaded JS)"]
    end
    end
    USER --> FE
    FE -->|"embeds by URL"| CDNJS
    APP --> AUTH
    APP --> PAY
    APP --> FLAG
    APP --> OBS
    CDNJS -->|"runs with your origin's trust  
on your users"| USER
```


The defenses are a blend of vendor risk management and technical pinning. For browser scripts: prefer **Subresource Integrity (SRI)** hashes so a changed script fails to load rather than silently executing; better still, self-host vetted copies so there is no standing third-party grant at all; and use a Content Security Policy to constrain what embedded code can reach. For CDN-loaded npm packages like connect-kit: pin an exact version *and its integrity hash*, never "latest." For server-side SaaS in the request path: treat each vendor as exactly what it is — a dependency whose compromise is your incident — and manage it as vendor risk (Book 8, Chapter 3), with contractual attestations, egress monitoring to catch a SaaS integration suddenly talking to somewhere new, and a tested fallback for when the dependency is unavailable or has to be cut off. And inventory them: the CycloneDX notion of a services BOM — a **SaaS BOM** — exists precisely so that your third-party *runtime* dependencies are enumerated with the same rigor as your build-time ones (Book 3, Chapter 8). You cannot manage a dependency you have not written down.

The cloud provider as a supply-chain dependency

Step back to the widest frame. Underneath the functions, the managed services, and even the IaaS VMs, you are trusting the cloud provider's supply chain in its entirety: the hypervisor that isolates your workload from your neighbors, the control plane that fulfills your API calls, the managed runtimes and service software already discussed, and the catalog of **images and marketplace offerings** they hand you as starting points. This is the largest single trust relationship in your stack and the most opaque. You are not going to audit AWS's or Google's or Microsoft's internal build systems. The honest framing, straight out of Book 1, Chapter 6, is that this is a **documented, accepted risk** — you have decided the provider is trustworthy, and the engineering task is to make that decision explicit, bounded, and backed by the best evidence available rather than by assumption.

What you *can* rely on: the provider's **compliance attestations and certifications** — SOC 2 Type II, ISO 27001, PCI DSS, FedRAMP authorizations for government workloads, and the audit reports they make available (through AWS Artifact, Google's compliance reports portal, the Microsoft Service Trust Portal). These are independent third-party assessments of the provider's controls, including their software-

development and change-management practices. They are not the same as you verifying the provider's runtime yourself — no attestation is — but they are structured, audited evidence, and they are the currency in which cloud provider trust is actually denominated. Relying on them is not a cop-out; it is the correct control for a dependency you cannot inspect directly, and it is what your own auditors and customers will expect you to have done.

What you cannot do is verify the provider's internals, and you should be clear-eyed that a **provider-side supply-chain compromise would be catastrophic and largely undetectable to you**. A backdoor introduced into a managed runtime's build, a compromise of the control plane, or a tampered service binary would reach every customer on that surface at once, and you would inherit the breach with no signal on your side of the line. This is not a reason for paralysis — the probability is low, the provider's incentives and scrutiny are enormous, and the attestations exist to raise the bar — but it is a reason to keep the residual risk written down in your threat model rather than pretending the managed stack has no attack surface because you cannot see it.

The place where provider trust becomes a concrete, everyday supply-chain decision is the **image and marketplace catalog**. Cloud providers offer three tiers of pre-built images, and they are not equally trustworthy. Provider-owned base images (Amazon Linux, the official managed-runtime bases) carry the provider's own trust. **Marketplace** images and machine images are published by vendors and are subject to some provider vetting, but the security posture and patch discipline of the *contents* are the publisher's, not the provider's. **Community images** — community AMIs on AWS being the canonical example — can be published by essentially anyone, and there is a documented history of malicious and backdoored community AMIs: images seeded with cryptocurrency miners, hard-coded SSH keys, or credential-exfiltration payloads, waiting for someone to launch them by name. Launching a random community AMI is exactly the same act as `docker pull`-ing an unknown image from an untrusted registry (Book 6, Chapter 2) or `npm install`-ing an unvetted package (Book 2) — it is running someone else's code at high privilege on the strength of a name. Vet these artifacts like any third-party dependency: prefer provider-owned or your own hardened images (build your own golden images — Book 6, Chapter 3), treat marketplace images as vendor software subject to Book 8, Chapter 3 review, and treat community images as untrusted until proven

otherwise. The convenience of "just launch this AMI / deploy this marketplace stack" is the exact convenience that makes it a supply-chain vector.

```
flowchart TB
    YOU["Your workloads + config + IAM"]
    subgraph INHERITED["What you inherit from the provider"]
        IMG["Marketplace / community images  
(vet like any artifact)"]
        RT["Managed runtimes + service software"]
        CP["Control plane"]
        HV["Hypervisor + host"]
    end
    end
    ATTEST["Compliance attestations  
SOC 2 / ISO 27001 / FedRAMP / PCI"]
    YOU -->|"runs on"| INHERITED
    INHERITED -->|"trust backed by"| ATTEST
    ATTEST -->|"evidence, not verification"| RESIDUAL["Residual,  
accepted risk  
(documented in threat model)"]
```

Concern	You can control	You must trust (backed by attestation)
Handler / function code + dependencies	Yes — lockfiles, SCA, SBOM, signing	—
Lambda layers / extensions you build	Yes — build, pin, SBOM, own the ARN	—
Third-party layers / marketplace / community images	Vetting decision + re-hosting	Publisher's internal integrity
IAM roles, event wiring, service config	Yes — least privilege, IaC scanning	—
Managed language runtime + its patching	Consume bulletins; track deprecation	Provider's runtime build + patch pipeline
Managed service software (DB, queue, auth)	Configuration only	Provider's build + operation of the service
Hypervisor, control plane, host	—	Provider entirely (SOC 2 / ISO / FedRAMP)

What you can actually do

Strip away the framing and a concrete program falls out — a checklist that is deliberately the same set of controls you have met throughout this suite, re-aimed at the serverless and managed surface.

- **Own your code and dependencies as if nothing changed** — because for that layer nothing did. Pinned, hashed lockfiles; SCA on every function and every managed-service integration; and an SBOM generated from the *built* artifact (the ZIP, the image, the edge bundle), not the source repo (Book 2; Book 3, Chapter 8). The most common serverless mistake is assuming "serverless" means "someone else's dependencies."
- **Sign what you deploy and verify before it runs.** AWS Signer with a `Enforce` code-signing configuration for ZIP-package Lambdas; cosign signing plus registry/admission verification for container-image functions and Cloud Run (Book 5; Book 6, Chapters 5–6). Remember the scope limit: Lambda code signing covers ZIPs, not images.
- **Vet layers, marketplace images, and SaaS as vendors.** Prefer your own layers and golden images; re-host trusted third-party layer contents into your own account and pin by immutable version ARN; treat marketplace and community images as untrusted third-party artifacts; and run every runtime SaaS and third-party script through vendor risk review (Book 8, Chapter 3), with SRI/self-hosting for browser scripts and exact-version-plus-hash pinning for CDN-loaded packages.
- **Least-privilege everything.** Scope every function's IAM execution role and every managed service's access to exactly what it uses. This is the containment layer that decides whether a supply-chain compromise is one bucket or the whole account.
- **Secure the configuration supply chain.** Managed services are provisioned by IaC; scan it for misconfiguration and enforce policy on it (Book 6, Chapter 8). Public buckets and default-open databases are a bigger real-world risk than provider compromise.
- **Document the shared-responsibility boundary per service** and rely on provider attestations for their side. Write down, for each managed surface, what is yours and what is theirs — for security design *and* for incident response, where "who patches this / who can

even see the logs" must be answered before the incident, not during it (Book 8, Chapter 6 — Incident Response for Supply Chain Events).

- **Monitor the runtime third-party dependencies** you cannot pin away — egress monitoring to catch a SaaS integration or edge script suddenly reaching a new destination, and a tested plan to cut off a compromised dependency fast, because polyfill.io and connect-kit both measured their damage window in hours.

Distributed-systems lens

At the scale this suite assumes — many teams, many repos, high deploy frequency — serverless and managed services are a genuine reduction in supply-chain burden *and* a genuine expansion in supply-chain surface, and mature organizations hold both facts at once. The reduction is real: across a fleet of hundreds of functions and dozens of managed services, the provider has absorbed the OS and runtime patching that would otherwise be the dominant line item in Chapter 4's vulnerability program. You get to stop patching what you no longer own. The expansion is equally real: that same fleet is now hundreds of small deployment surfaces (every function a ZIP or image with its own dependency tree), plus dozens of managed services, plus a long tail of runtime SaaS integrations and edge scripts that never show up in a source-tree scan. The failure mode at scale is not any single dramatic compromise; it is *sprawl* — functions and integrations that no one has inventoried, deployed by teams that treated "serverless" as "not my supply chain."

The operating discipline follows directly. **Inventory everything** with SBOMs for what you build and SaaS SBOMs for what you consume at runtime (Book 3, Chapter 8), so the fleet is enumerable rather than folkloric. **Sign and verify** every artifact you deploy, uniformly, so provenance holds across the whole surface and not just the container-shaped part of it. **Least-privilege everything**, because with hundreds of execution roles the aggregate blast radius is set by your worst-provisioned function, not your best. **Make the shared-responsibility boundary explicit per service** — a written line, not an assumption — because at fleet scale the ambiguity of "who owns this layer" is what turns a routine CVE or a provider deprecation notice into an unowned, unpatched gap, and it is what makes incident response flounder when the affected component sits on the provider's side of a line nobody drew (Book 8, Chapter 6). And **accept the provider trust deliberately** —

write it into the threat model as a residual risk backed by SOC 2 / ISO 27001 / FedRAMP attestation, so that the one part of the chain you truly cannot control is a conscious, bounded decision rather than a blind spot.

The sentence to leave with is the one this chapter has been arguing from the first paragraph: managed and serverless is not "no supply chain." It is a *different, provider-shared* supply chain — smaller on your side of the line, larger and more opaque on theirs — and securing it means being precise about where the line sits, owning your side of it completely, and trusting the other side explicitly.

Key takeaways

- **The boundary moves; it does not disappear.** FaaS and managed services shift the OS, runtime, patching, and infrastructure to the provider — a real win, no managed-runtime CVEs for you to patch — but your code and dependencies remain fully yours, and the provider's supply chain becomes a new upstream dependency you cannot fully verify.
- **Your function dependencies are your supply chain, unchanged.** ZIP or image, the dependency tree is resolved by the same tooling from the same registries with the same risks as Book 2. Lockfile, SCA, and SBOM the built artifact — the absence of a Dockerfile does not mean the absence of a bill of materials.
- **Lambda layers are shared dependencies with a shared blast radius.** A poisoned or unvetted third-party layer or extension affects every function that attaches it. Prefer your own layers, pin by immutable version ARN, and re-host trusted third-party layers into your own account.
- **The managed runtime is a visibility gap you trust, plus a forced-upgrade obligation.** You cannot fully SBOM it, and runtime deprecation force-migrates you on the provider's schedule — a recurring fleet-wide task to budget for.
- **Sign what you deploy — with the right tool and its real scope.** AWS Signer plus an `Enforce` code-signing configuration verifies ZIP deployment packages at deploy time; container-image functions are signed and verified as containers (Book 6, Chapters 5–6). Signing proves authenticity, not the absence of vulnerable dependencies — you need both.

- **Managed services are three dependencies:** the provider's software supply chain (trust + attestation), the configuration supply chain (yours — IaC security, Book 6, Chapter 8), and any surface that runs *your* code (edge functions, stored procedures — in scope for your SBOM/SCA/signing).
- **Runtime SaaS and third-party scripts execute with your trust.** polyfill.io (June 2024) and Ledger connect-kit (December 2023) show that a CDN-loaded script or SaaS integration turning malicious is your live incident within hours. Pin with SRI/self-hosting, exact-version-plus-hash for CDN packages, manage as vendor risk, and inventory with a SaaS BOM.
- **The cloud provider is a documented, accepted trust relationship** backed by SOC 2 / ISO 27001 / FedRAMP attestations — evidence, not verification. Vet marketplace and especially community images like any untrusted third-party artifact; a malicious community AMI is `npm install` at host privilege.
- **Least-privilege IAM per function and per service is the containment layer** that decides whether a supply-chain compromise is one bucket or the whole account.

Further reading

- AWS, "Shared Responsibility Model" — the canonical statement of security *of* vs *in* the cloud; Google Cloud and Microsoft publish equivalent shared-responsibility documentation.
- AWS Lambda Developer Guide — "Lambda layers," "Lambda extensions," "Deploying Lambda functions as container images," and "Runtime deprecation policy."
- AWS, "Configuring code signing for AWS Lambda" and AWS Signer Developer Guide — signing profiles, code-signing configurations, and the Warn / Enforce signature-validation policies (note the ZIP-only scope).
- Google Cloud Functions and Cloud Run documentation; Azure Functions documentation — deployment models, runtimes, and the managed-runtime patching posture for the other major providers.
- The Sansec and BleepingComputer reporting on the **polyfill.io** supply-chain attack (June 2024), and Cloudflare's and Fastly's mirror announcements.

- Ledger's incident disclosure and community post-mortems on the **@ledgerhq/connect-kit** compromise (14 December 2023).
- W3C, "Subresource Integrity (SRI)" specification, and MDN's SRI and Content Security Policy guides — the browser-side controls for third-party scripts.
- CycloneDX documentation on **services** and the SaaS BOM concept — enumerating runtime third-party dependencies.
- Provider compliance portals — AWS Artifact, Google Cloud compliance reports, Microsoft Service Trust Portal — for SOC 2, ISO 27001, PCI DSS, and FedRAMP attestations.
- Cross-references in this suite: Book 1, Chapter 6 — Trust, Threat Models, and the Economics of Supply Chain Risk; Book 1, Chapter 9 — Supply Chain Security in Distributed Backend Systems; Book 2 — Dependency Management and Open Source Risk (Chapters 2, 3, 4, 6); Book 3, Chapter 7 — SBOM Quality, Completeness, and Limitations; Book 3, Chapter 8 — SBOMs for Services: Containers, Serverless, and SaaS; Book 5, Chapters 3 and 5 — Sigstore Architecture and Transparency Logs; Book 6, Chapters 1-6 and Chapter 8 — the container and IaC supply chain; Book 8, Chapter 3 — Vendor and Third-Party Software Risk; Book 8, Chapter 6 — Incident Response for Supply Chain Events.

Chapter 10 — A Cloud-Native Supply Chain Reference Architecture

What this chapter covers. Every prior chapter in this book handed you one control. Chapter 1 (Container Images) explained the OCI format and content-addressable digests. Chapter 2 (Container Registries) made the registry the storage chokepoint and the referrers API the place attestations live. Chapter 3 (Base Image Strategy) gave you golden, minimal, continuously-rebuilt bases. Chapter 4 (Image Scanning) gave you vulnerability visibility on push and at rest. Chapter 5 (Image Signing and Verification in Kubernetes) put a cryptographic signature on the image and a verifier at the cluster edge. Chapter 6 (Admission Control and Policy Engines) turned that verifier into an enforcing gate. Chapter 7 (Kubernetes Delivery Chains) wired GitOps between the registry and the cluster. Chapter 8 (Infrastructure as Code Supply Chain Risks) secured

the substrate the cluster runs on. Chapter 9 (serverless) extended the model past long-lived pods.

This chapter does something different: it **integrates** them, and integrates them with the rest of the suite — build provenance from Book 4, signing and attestation from Book 5, SBOMs from Book 3, source integrity from Book 7 — into a single, concrete, buildable reference architecture for securing the supply chain of a cloud-native, containerized, distributed backend. It is the capstone. It is opinionated on purpose. The thesis is simple and load-bearing: **no single control secures a cloud-native supply chain; security is an unbroken, verified chain of custody from a signed commit to a running pod, in which every artifact carries verifiable metadata and every boundary verifies it.**

Learning goals — after this chapter you should be able to:

- State the **chain-of-custody thesis** precisely and name what evidence each stage produces and what each boundary verifies.
- Walk the **end-to-end reference architecture** stage by stage — source, build, base, registry, metadata store, delivery, admission, runtime — and say what artifact and what attestation crosses each edge.
- Describe the **paved-road control plane**: what the platform team owns so that application teams inherit the entire secure chain by default, and where central policy is enforced fleet-wide.
- **Map the architecture to frameworks** — SLSA v1.0 Build L3, image signing and verification, SBOM coverage, verified deployment — and to the regulatory drivers behind them.
- **Validate the design against a threat model**: for each classic supply-chain attack, name the control that stops it and the chapter it came from.
- Sequence a realistic **crawl/walk/run adoption roadmap** with warn-then-enforce rollout, tiered policy for internal versus third-party images, and an exception process with expiry.
- Choose **coverage and MTTR metrics** that prove the chain is closed — and answer the end-state test: can an unsigned, unscanned, unauthorized-source, tag-mutable image reach production?

The thesis: chain of custody, not a pile of controls

Most organizations accumulate supply-chain controls the way they accumulate monitoring dashboards: one at a time, each solving a local pain, none aware of the others. They turn on registry scanning after an audit finding. They add image signing after reading about a registry breach. They stand up Kyverno after a crypto-miner runs in a namespace. Each control is real, and each is nearly worthless in isolation, because an attacker does not need to defeat your strongest control — they need to find the one edge no control watches.

Signing without verification is theatre: you sign every image and admit unsigned ones anyway. Verification without provenance proves *someone* signed the image, not that a *trusted build* produced it — a signature from a leaked key or a compromised laptop verifies just fine. Provenance without admission enforcement produces a beautiful, unread attestation sitting in the registry while the cluster pulls whatever digest the deployment names. Scanning without a gate produces a report nobody blocks on. SBOMs without a queryable store are JSON nobody can answer "am I affected?" with when the next Log4Shell (Book 1, Chapter 5 — Case Studies III) drops at 2 a.m.

The architecture in this chapter is built on one idea, borrowed from physical evidence handling and from **in-toto** (Book 5, Chapter 6 — in-toto: Attestations, Layouts, and Policies): a **chain of custody**. Each stage of the pipeline is a link. Each link *produces verifiable evidence* about what it did — a signed provenance statement, a signed SBOM, a signed scan result, a signature over the image digest. Each *boundary between links verifies* the evidence from the prior link before it acts. The image digest is the thread that runs through the whole chain: because an OCI digest is the SHA-256 of the manifest (Book 6, Chapter 1 — Container Images), every attestation binds to a digest, and every verifier can confirm that the thing it is about to run is *the exact bytes* the evidence describes. Break any link and the next boundary refuses to proceed. That is the whole design. The rest of the chapter is mechanism.

```

flowchart LR
    subgraph chain["Chain of custody – evidence produced, evidence verified"]
        C["Commit signed"] -->|"verify sig"| B["Build provenance + SBOM"]
        B -->|"verify provenance"| R["Registry image + attestations"]
        R -->|"verify signature"| A["Admission policy gate"]
        A -->|"verified digest"| P["Pod running"]
    end
    style C fill:#1f6f43,color:#fff
    style B fill:#1f6f43,color:#fff
    style R fill:#1f6f43,color:#fff
    style A fill:#7a1f1f,color:#fff
    style P fill:#1f4f7a,color:#fff

```

Two properties make this more than a slogan. First, the evidence is **cryptographically bound** to content, not to names: a signature over `sha256:abc...` cannot be transplanted onto a different image, and a tag pointing somewhere else cannot smuggle unverified bytes past a digest-checking gate. Second, the verification is **enforced at a chokepoint that cannot be bypassed** — the admission controller — so that even if every earlier link is compromised, the last boundary still refuses an image that lacks the required, valid evidence. Produce verifiable evidence at every step; verify at every boundary. Everything below is the concrete wiring.

The end-to-end reference architecture

Here is the whole thing. Read it top to bottom; the sections that follow walk each stage and say exactly what crosses each edge.

```

flowchart TB
    subgraph src["1. Source – Book 7"]
        DEV["Developer"] -->|"signed commit"| REPO["App repo"]
        protected branch
        required review"
    end

    subgraph bld["2. Build – Book 4 (SLSA L3)"]
        REPO -->|"trigger on merge"| CI["Hermetic, ephemeral"]
        build on shared platform
        OIDC workload identity"]
        BASE["3. Golden base images"]
        minimal, signed,
        continuously rebuilt – Ch 3"] --> CI
        CI --> IMG["Image + digest"]
        CI --> PROV["SLSA provenance"]
        in-toto attestation"]
        CI --> SBOM["SBOM – syft"]
        SPDX / CycloneDX"]
    end

    subgraph reg["4. Registry – Ch 2 (chokepoint)"]
        IMG --> STORE["Internal registry"]
        PROV -->|"cosign attest"]
        OCI referrers"| STORE
        SBOM -->|"cosign attest"| STORE
        CI -->|"cosign sign"]
        keyless"| STORE
        STORE --> SCAN["Scan on push"]
        Trivy / Gripe – Ch 4"]
        SCAN -->|"signed scan result"| STORE
    end

    subgraph meta["5. Metadata / attestation store – Book 3, Book 5"]
        STORE -->|"ingest"| GUAC["GUAC /"]
        Dependency-Track"]
        CI -->|"signing entries"| REKOR["Rekor"]
        transparency log"]
        VEX["VEX – OpenVEX"] --> GUAC
    end

    subgraph del["6. Delivery – Ch 7 (GitOps)"]
        STORE -->|"digest"| GITOPS["Config repo"]
        signed commits"]
        GITOPS --> ARGO["Argo CD / Flux"]
        renders signed Helm"]
    end

    subgraph adm["7. Admission – Ch 5 & 6 (backstop)"]
        ARGO -->|"apply manifest"| GATE["Admission controller"]
    end

```

```

policy-controller / Kyverno"]
    POLICY["Central policy
Book 8 Ch 4"] --> GATE
    STORE -->|"fetch + verify
sig, provenance,
SBOM, scan, digest"| GATE
    REKOR -->|"inclusion proof"| GATE
end

subgraph run["8. Runtime"]
    GATE -->|"admit"| POD["Running pod"]
    GATE -->|"deny"| REJECT["Rejected"]
    POD --> RESCAN["Continuous re-scan – Ch 4
runtime monitoring
drift detection"]
end

style GATE fill:#7a1f1f,color:#fff
style STORE fill:#1f4f7a,color:#fff
style CI fill:#1f6f43,color:#fff
style REJECT fill:#7a1f1f,color:#fff

```

Stage 1 — Source: establishing a trusted origin

The chain starts before any container exists, at the commit. Book 7 (Source, Code, and Insider Threat Security) is the authority here; the reference architecture *requires* three things from it. Commits to the release branch are **signed** — Sigstore `gitsign` for keyless commit signing tied to an OIDC identity, or GPG/SSH signing — so the build system can attribute a commit to a real developer identity rather than trusting an unauthenticated `Author:` header. The release branch is **protected**: no direct pushes, linear history or merge-commit rules enforced, force-push disabled. And every change is **reviewed** under a required-reviewers rule, so no single account — human or compromised token — can move code to the branch the build system trusts. The output of this stage is not an artifact; it is a *property*: the commit the build will consume is attributable, reviewed, and tamper-evident. That property is the first link, and the build stage verifies it before it starts.

Stage 2 — Build: producing signed evidence (SLSA Build L3)

The build is where evidence is born. This is the domain of Book 4, and specifically of Chapter 4.10 (Designing a Secure Build Platform at Scale) and Chapter 4.3 (SLSA Build Levels and Provenance). The reference build

has four non-negotiable properties, and they map directly onto **SLSA v1.0 Build Level 3**:

- **Hermetic** (Book 4, Chapter 2 — Hermetic and Reproducible Builds): dependencies are fetched before the build from pinned, verified sources; the build step itself has no arbitrary network egress, so a malicious `postinstall` cannot phone home or pull an attacker payload mid-build.
- **Isolated and ephemeral** (Book 4, Chapter 8 — Ephemeral and Isolated Build Environments): each build runs in a fresh environment that is destroyed afterward, so one tenant's build cannot influence another's and no persistent implant survives between runs.
- **Provenance-generating**: the platform — not the user's build script — generates a **SLSA provenance** in-toto attestation describing what was built, from which source commit, with which builder, and records it. Because the platform controls provenance generation and the build cannot forge it, this is what lifts the build from L2 to **L3**.
- **Signed by build identity**: the image, the provenance, and the SBOM are all signed using **keyless signing** (Book 5, Chapter 4 — Keyless Signing and Workload Identity). The build's OIDC workload identity is exchanged with **Fulcio** for a short-lived certificate; the signing event is recorded in **Rekor**. There is no long-lived signing key to steal.

Concretely, on the shared build platform, the tail of a build looks like this:

```
# 1. Build the image (hermetic; base pulled from the internal golden-
base registry, by digest)
IMAGE="registry.internal/team-payments/api"
DIGEST=$(buildctl build ... --output type=image,name=$IMAGE,push=true
| grep -oE 'sha256:[a-f0-9]+')
REF="$IMAGE@$DIGEST"

# 2. Generate the SBOM from the built image and attest it (keyless)
syft "$REF" -o spdx-json > sbom.spdx.json
COSIGN_EXPERIMENTAL=1 cosign attest --yes \
  --predicate sbom.spdx.json --type spdxjson "$REF"

# 3. Sign the image itself (keyless – OIDC identity -> Fulcio cert ->
Rekor entry)
COSIGN_EXPERIMENTAL=1 cosign sign --yes "$REF"

# 4. SLSA provenance is emitted by the *platform*, not this script, and
attested to the same digest:
#   cosign attest --type slsaprovenance --predicate provenance.json
"$REF"
```

The critical discipline: **the build script cannot opt out**. Signing, SBOM generation, and provenance are platform behaviors wrapped around the user's build, not steps the user's YAML is trusted to include. A team cannot ship an unsigned image by deleting a line, because the line is not theirs to delete. This is the paved-road principle applied to the most sensitive stage.

Stage 3 — Base images: a golden, signed foundation

The build does not pull `FROM ubuntu:latest` off the public internet. It pulls a **golden base** (Book 6, Chapter 3 — Base Image Strategy) from the internal registry, *by digest*, and the base is itself a signed artifact produced by the same secure build platform. Golden bases are minimal (distroless or a hardened slim image), which shrinks the vulnerability surface the scanner will find in Stage 4; they are **continuously rebuilt** so that a CVE in the base is fixed by a rebase-and-roll rather than by a per-team scramble; and they are signed and attested exactly like application images. The base is a link in the chain of custody too: its signature and provenance are verifiable, so an application image's SBOM transitively covers a *trusted* base rather than an arbitrary one. A poisoned public base (the analog of the attack the golden-base program exists to stop) never enters the chain because the build platform is configured to pull bases only from the internal, signed set.

Stage 4 — Registry: the storage chokepoint

Every artifact converges on the internal registry (Book 6, Chapter 2 — Container Registries). This is deliberately the **chokepoint** of the architecture: one place through which all images flow, which means one place to scan, one place to store attestations, and one place admission can trust as the source of truth. The registry stores, keyed to each image digest:

- the **image** manifest and layers;
- the **image signature** (a `cosign` signature is itself an OCI artifact);
- the **SBOM attestation** and the **provenance attestation**, discoverable via the **OCI referrers API** (Book 6, Chapter 1) so a verifier can ask "what attestations reference `sha256:abc...`?" and get back the signed SBOM, provenance, and scan results;
- the **scan result**, produced by **scan-on-push** (Book 6, Chapter 4 — Image Scanning) with Trivy or Gype, and itself signed and attested so admission can trust it later.

Because attestations are attached *by digest* through referrers, moving or deleting a tag cannot orphan or swap the evidence — the evidence is bound to bytes. The registry is a tier-0 dependency of the whole platform; its availability gates every deploy, and its integrity gates every verification.

Stage 5 — Metadata and attestation store: making evidence queryable

Attestations attached in the registry answer "what is true about this one image?" A separate **metadata store** answers the fleet-wide questions: "which running images depend on `log4j-core`?", "which images were built from an unpatched base?", "does this CVE actually reach code we run?" (Book 3, Chapter 5 — SBOM Distribution, Storage, and Querying at Scale). The architecture centralizes SBOMs, provenance, and **VEX** (Vulnerability Exploitability eXchange — OpenVEX; Book 3, Chapter 6 — VEX and Vulnerability Correlation) into a queryable graph — **GUAC** ingesting attestations and SBOMs, or **Dependency-Track** for continuous component-level monitoring. VEX is what converts a raw scan finding ("this CVE is present in a bundled library") into an actionable one ("...and it is reachable, so it is affected" versus "...but the vulnerable path is not built in, so it is not_affected"), which is the difference between a page and

a note (Book 2, Chapter 7 — Reachability, Exploitability, and Prioritization).

Orthogonally, every signing event lives in the **Rekor transparency log** (Book 5, Chapter 5 — Transparency Logs). Rekor gives the whole system a tamper-evident, append-only record of *what was signed, by which identity, when* — so that even a later compromise of a signing identity cannot rewrite history, and admission can demand a Rekor inclusion proof as part of verification.

Stage 6 — Delivery: GitOps between registry and cluster

Nothing pushes to the cluster imperatively. Deployment is **GitOps** (Book 6, Chapter 7 — Kubernetes Delivery Chains): the desired state of each cluster lives in a config repository, changes to it are **signed commits** through the same protected-branch discipline as source, and **Argo CD** or **Flux** reconciles the cluster to match. Helm charts are rendered from pinned, signed chart versions. Critically, the deployment manifest names images **by digest, not by tag** — the digest that came out of the build and through the registry — so the thing GitOps asks the cluster to run is the exact thing the evidence describes. The config repo is a high-value target (whoever controls it controls what deploys), which is why it does not stand alone: the next stage verifies regardless of what GitOps applied.

Stage 7 — Admission: the backstop that cannot be bypassed

The admission controller is the last boundary and the one that makes the architecture enforcing rather than advisory (Book 6, Chapter 5 — Image Signing and Verification in Kubernetes; Chapter 6 — Admission Control and Policy Engines). Every pod-creating request in every cluster passes through it. For each image, the controller — Sigstore's **policy-controller** or **Kyverno** — verifies, against **central policy**:

1. the image is **signed** by the expected identity;
2. the signer is the **expected build workload identity** (the OIDC subject and issuer of the CI platform), not merely *some* valid Fulcio certificate — this is the check that distinguishes "our build signed it" from "someone with a working laptop signed it";
3. the required **attestations exist and verify**: a SLSA provenance whose builder and source repo match policy, an SBOM, and a passing scan result;

4. the reference is **digest-pinned** — tags are rejected, so nothing mutable can slip through;
5. optionally, a **Rekor inclusion proof** confirms the signature is logged.

A Kyverno rule that ties several of these together:

```

apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: verify-supply-chain
spec:
  validationFailureAction: Enforce
  # audit first, then Enforce (Ch 6)
  rules:
    - name: require-signed-and-attested
      match:
        any:
          - resources:
              kinds: ["Pod"]
      verifyImages:
        - imageReferences:
            - "registry.internal/*"
            # keyless: identity must be OUR build platform, not any
            Fulcio cert
            attestors:
              - entries:
                  - keyless:
                      issuer: "https://
token.actions.githubusercontent.com"
                      subject: "https://github.com/acme/build-
platform/.github/workflows/build.yaml@refs/heads/main"
                      rekor:
                          url: "https://rekor.internal"
            # require SLSA provenance whose source repo matches
            attestations:
              - type: "https://slsa.dev/provenance/v1"
                conditions:
                  - all:
                      - key: "{{ buildDefinition.externalParameters.sourc
e }}"
                      operator: Equals
                      value: "git+https://github.com/acme/*"
        - name: reject-mutable-tags
          match:
            any:
              - resources: { kinds: ["Pod"] }
          validate:
            message: "images must be pinned by digest"
            pattern:
              spec:
                containers:
                  - image: "*/sha256:*"

```

Admission is where the whole chain of custody is *cached in*. Every earlier link produced evidence; this boundary is where the evidence is finally

required. Note the design property that gives the architecture its resilience: **admission verifies independently of how the image arrived**. If the GitOps repo is compromised and points at a malicious digest, admission still demands a valid signature from the expected build identity — which the attacker's image will not have. The gate does not trust the pipeline; it trusts the evidence.

Stage 8 — Runtime: the chain does not end at admission

Admission verifies a point in time; production is continuous. A CVE disclosed tomorrow was invisible to yesterday's scan. So the architecture keeps working after the pod starts: **continuous re-scanning** of running images against fresh vulnerability data (Book 6, Chapter 4), correlated back through the metadata store so "which running workloads are affected by CVE-2026-xxxx?" is a query, not an investigation; **runtime monitoring** for anomalous process, file, and network behavior (the runtime analog of build observability, Book 4, Chapter 9 — Build Observability and Anomaly Detection); and **drift detection** — GitOps controllers already flag when live state diverges from the signed config repo, which turns "someone `kubectl edit`-ed a Deployment" into an alert. The loop closes: findings at runtime feed VEX and prioritization, which feed the next rebuild and redeploy.

The control plane view: a paved road

The architecture above has eight stages and a dozen components. No application team can or should assemble that themselves. The organizing principle — the one that makes this adoptable rather than aspirational — is the **paved road** (Book 1, Chapter 10 — Building a Supply Chain Security Program; Book 4, Chapter 10 — Designing a Secure Build Platform at Scale): a central platform team owns the secure chain, and application teams *inherit the entire thing by using the platform*, at little or no extra effort of their own.

```

flowchart TB
    subgraph platform["Platform team owns – the paved road"]
        GB["Golden base images  
minimal, signed, rebuilt"]
        BP["Build platform  
auto: provenance + SBOM + signing"]
        RG["Internal registry  
auto: scan-on-push, referrers"]
        MS["Metadata store  
GUAC / Dependency-Track / Rekor"]
        GO["GitOps + Helm  
signed config, digest pinning"]
        AC["Admission policy  
auto-verify, central, fleet-wide"]
    end
    subgraph teams["Application teams get – by default"]
        T1["Payments service"]
        T2["Search service"]
        T3["Notifications service"]
    end
    GB --> BP --> RG --> MS
    RG --> GO --> AC
    platform ==>|"inherited security"| teams
    T1 -.->|"push code"| BP
    T2 -.->|"push code"| BP
    T3 -.->|"push code"| BP
    style AC fill:#7a1f1f,color:#fff
    style platform fill:#12314f,color:#fff

```

The division of labor is exact. The platform provides golden bases, the hardened build platform that auto-generates provenance and SBOMs and signs everything, the internal registry that auto-scans and stores attestations by digest, the metadata store, the GitOps machinery, and the admission policy. A team's obligation is to *push reviewed, signed code to a repo the platform builds*. In return they get a signed image with provenance and an SBOM, stored and scanned, deployed via GitOps, and verified at admission — the full chain — without writing a single `cosign` command or Kyverno policy. Security is **inherited, not implemented per team**.

Two design properties make the paved road strong. First, **the secure path is the easy path** — often the *only* supported path. A team that wants to deploy uses the platform because the platform is how you deploy; the security comes along for free, which defeats the usual failure mode where the secure option is the inconvenient option and nobody chooses it. Second, **enforcement is central and uniform**: admission

policy is authored once by the platform/security team (Book 8, Chapter 4 — Policy as Code and Continuous Compliance), distributed to every cluster, and applied identically fleet-wide. There is no per-cluster policy drift, no cluster where the rule is quietly disabled, and — because the gate sits at admission in the API server request path — **no bypass**. An image that skips the paved road and gets pushed straight to a cluster still hits the same gate and still lacks the evidence, so it is still denied. The paved road is a default, not a fence; admission is the fence.

Mapping to SLSA and framework requirements

Auditors, customers, and regulators do not ask "is your chain of custody unbroken?" They ask "what SLSA level do you attest?" and "do you produce SBOMs?" and "can you show verified deployment?" The architecture answers these directly; the mapping is not a retrofit, it is how the components were chosen. (Framework background: Book 1, Chapter 7 — Risk Frameworks and Maturity Models; adoption roadmaps: Book 8, Chapter 2 — Adopting SLSA and S2C2F; regulatory drivers such as EO 14028, NIST SSDF, and the EU Cyber Resilience Act: Book 8, Chapter 1 — The Regulatory Landscape.)

Requirement	Framework anchor	How the architecture satisfies it
Scripted, provenance-emitting build	SLSA v1.0 Build L1-L2	Platform build emits SLSA provenance in-toto attestation for every image (Stage 2; Book 4 Ch 3)
Provenance unforgeable by the build	SLSA v1.0 Build L3	Provenance generated by the <i>platform</i> , not user build steps; hermetic + isolated + ephemeral builds (Stages 2-3; Book 4 Ch 2, 8)
Signed artifacts, verifiable identity	Sigstore / code signing	Keyless signing via Fulcio + workload identity; Rekor transparency log (Stages 2, 5; Book 5 Ch 4-5)
Component transparency	SBOM (SPDX 2.3 / CycloneDX 1.6)	syft-generated SBOM attested per image; centralized, queryable store (Stages 2, 5; Book 3)

Requirement	Framework anchor	How the architecture satisfies it
Vulnerability status, not just presence	VEX (OpenVEX)	VEX statements in the metadata store gate real exploitability (Stage 5; Book 3 Ch 6, Book 2 Ch 7)
Verified, policy-gated deployment	NIST SSDF PW/PS; internal policy	Admission verifies signature + build identity + provenance + SBOM + scan + digest, fleet-wide (Stage 7; Book 6 Ch 5-6)
Tamper-evident record of decisions	Transparency / audit	Rekor inclusion proofs; GitOps signed-commit history; admission audit logs (Stages 5-7)

The point of the table is that **SLSA Build L3 is not a checkbox bolted on at the end**; it is an emergent property of a build platform that is hermetic, isolated, ephemeral, and the sole author of provenance. Likewise "SBOM everywhere" is not a compliance chore but the substrate that makes runtime CVE response a query. The frameworks are satisfied because the architecture is right, not the other way around.

Threat-model validation: showing the chain holds

An architecture is only as good as the attacks it stops. Below, each row is a real class of supply-chain attack (grounded in the incidents of Book 1, Chapters 3-5), the control or controls in this architecture that stop it, and where in the suite that control was built. The design principle to notice: most attacks are stopped by *more than one* control, so a single failed link does not open the door — this is defense in depth realized as a chain where later boundaries re-verify.

```

flowchart LR
    subgraph attacks["Attack enters here"]
        A1["Compromised  
dependency"]
        A2["Malicious build /  
build implant"]
        A3["Poisoned base  
image"]
        A4["Registry tamper"]
        A5["Tag swap"]
        A6["Unsigned /  
unauthorized image"]
        A7["GitOps repo  
compromise"]
        A8["IaC compromise"]
    end
    subgraph controls["Stopped by"]
        C1["Scan + SBOM +  
reachability + VEX"]
        C2["Hermetic + isolated +  
provenance L3"]
        C3["Golden signed base  
+ scan"]
        C4["Signature + verify  
by digest"]
        C5["Digest pinning"]
        C6["Admission denies"]
        C7["Signed commits +  
admission re-verifies"]
        C8["IaC scan + policy +  
least privilege"]
    end
    A1 --> GATE2["Admission backstop"]
    A2 --> GATE2
    A3 --> GATE2
    A4 --> GATE2
    A5 --> GATE2
    A6 --> GATE2
    A7 --> GATE2
    A8 --> GATE2
    style GATE2 fill:#7a1f1f,color:#fff

```

The full mapping, attack to control to chapter:

Attack	What the attacker does	Control(s) in the architecture	Chapter(s)
Compromised dependency	Ships malware in a transitive library (event-stream, ua-parser-js)	SBOM makes the component visible; scan flags the known CVE; reachability + VEX decide if it matters; continuous re-scan catches later disclosure	Book 2 Ch 6-7; Book 3; Book 6 Ch 4
Malicious build / build implant	Injects a payload during the build step (Codecov, the general SolarWinds class)	Hermetic build blocks mid-build network fetch; isolated + ephemeral prevents persistence; SLSA L3 provenance records what actually ran and is checked at admission	Book 4 Ch 2, 3, 8
SolarWinds-class build backdoor	Long-lived implant modifies artifacts between source and release	Provenance ties artifact to exact source commit; reproducibility lets an independent rebuild detect divergence; build isolation denies the implant a home	Book 1 Ch 3; Book 4 Ch 2-3
Poisoned base image	Backdoors a public base everyone pulls FROM	Build pulls only golden, signed bases by digest; bases are scanned and continuously rebuilt; a public poisoned base never enters the chain	Book 6 Ch 3
Registry tamper	Overwrites stored layers/manifest with malicious bytes	Signature is over the digest; verify-by-digest at admission fails for tampered content; attestations bound by referrers cannot be swapped	Book 6 Ch 1-2, 5

Attack	What the attacker does	Control(s) in the architecture	Chapter(s)
Tag swap / mutable-tag attack	Repoints <code>:latest</code> or <code>:v1.2</code> at a malicious image	Deploy and admission use digests only; a Kyverno/policy-controller rule rejects tag references outright	Book 6 Ch 1, 5-6
Unsigned / unauthorized image	Pushes an image built outside the platform	Admission requires a valid signature from the <i>expected build identity</i> ; an image without it — or signed by any other identity — is denied	Book 6 Ch 5-6
GitOps repo compromise	Edits the config repo to deploy a malicious digest	Config commits are signed and reviewed; and admission re-verifies signature + provenance regardless of what GitOps applied, so the malicious image still lacks valid evidence	Book 6 Ch 7; Book 7
IaC compromise	Backdoors a Terraform module / provider to weaken the cluster	IaC misconfiguration scanning + policy-as-code + least-privilege execution identity; plan/apply separation	Book 6 Ch 8
Signing-key theft	Steals a long-lived signing key to forge signatures	No long-lived key exists — keyless signing issues short-lived Fulcio certs; Rekor logs every signing event so forgery is detectable	Book 5 Ch 4-5

Read the "unsigned / unauthorized image" and "GitOps repo compromise" rows together, because they are the architecture's thesis in miniature. An attacker who fully owns the deployment path — the config repo, the CI YAML, even push access to the registry — still cannot get a workload to run, because the last boundary demands evidence they cannot

manufacture: a signature from the *specific* build workload identity, over *this* digest, with matching provenance, logged in Rekor. Every other control is depth; this is the floor.

Adoption roadmap: crawl, walk, run

You do not deploy this architecture on a Tuesday. Turning on enforcing admission across a fleet before your images are signed will halt every deploy in the company and get the whole program cancelled by Wednesday. The order of adoption matters as much as the design, and the sequence has a logic: **visibility before integrity, integrity before verification, verification in audit before enforce**. You cannot verify signatures that do not exist, and you cannot safely enforce a policy you have not first watched run in audit mode against real traffic.

```
flowchart LR
    C["CRAWL – Visibility  
internal registry  
scan-on-push  
minimal golden bases"]
    W["WALK – Integrity  
SBOM generation  
keyless signing  
SLSA provenance"]
    R["RUN – Verification  
admission in AUDIT  
then ENFORCE  
signature + identity"]
    F["FULL – Enforcement  
attestation-gated  
digest-pinned  
policy fleet-wide"]

    C --> W
    W --> R
    R --> F

    style C fill:#12314f,color:#fff
    style W fill:#1f6f43,color:#fff
    style R fill:#6f5a1f,color:#fff
    style F fill:#7a1f1f,color:#fff
```

- **Crawl — visibility and hygiene.** Stand up the internal registry as the chokepoint, turn on scan-on-push, and move teams onto minimal golden bases. Nothing blocks yet. The win is that you now *know* what images exist, what is in them, and what CVEs they carry — the prerequisite for every later gate. This stage alone repays its cost in incident response.

- **Walk — integrity.** Add SBOM generation, keyless signing, and SLSA provenance to the build platform. Still nothing blocks. Every new image now carries verifiable evidence, and the metadata store starts filling with SBOMs and provenance. You are producing the evidence the next stage will require, and you can measure coverage climbing before you depend on it.
- **Run — verification.** Deploy admission policy in **audit** mode first: it evaluates every image and *records* what it would have denied, without denying anything. Watch the audit stream, fix the images and pipelines it flags, and only when the would-deny rate on paved-road images reaches ~zero do you flip individual policies to **Enforce**. Warn, then enforce — always, per policy.
- **Full enforcement.** Tighten to attestation-gated, digest-pinned, fleet-wide policy: require provenance and passing scans, reject mutable tags, and demand the expected build identity. The chain is now closed by construction.

Warn-then-enforce is not a one-time transition; it is how you ship *every* new rule forever. A new policy — "require an SBOM attestation," say — goes out in audit, you watch, you fix the long tail, you enforce. The discipline is what keeps a security control from becoming a company-wide outage.

Handling reality: third-party images, legacy, and the long tail

A pristine internal chain meets an untidy world, and the architecture has to survive contact with it. The tool is **tiered policy** (Book 5, Chapter 10 — Designing Attestation-Based Deployment Gates): different classes of image are held to different, appropriate standards, all enforced at the same gate.

- **Third-party and vendor images.** You cannot demand *your* provenance from an image you did not build. You can demand what is reasonable: pull it through your internal registry (never let workloads pull from arbitrary external registries), scan it, and — where the vendor participates — verify the *vendor's* signature against *their* published identity. A separate policy tier applies to `registry.internal/third-party/*`: signature-by-vendor-identity plus a passing scan, no SLSA L3 provenance required. The gate still gates; the bar is set to what the artifact can actually prove.

- **Legacy workloads.** Systems that predate the platform and cannot be rebuilt get a scoped, time-boxed exemption tier — not a blanket bypass. They are pinned by digest, scanned, and tracked, on an explicit path to migration.
- **Exceptions with expiry.** Every exception — a temporarily unfixable CVE, an unsigned legacy image — is a policy object with an **owner and an expiry date**. It is granted in code (so it is auditable and reviewable), it shows up in metrics, and it *expires*, forcing re-justification rather than becoming permanent debt. An exception that cannot expire is a hole; the process makes holes self-closing.

The tiered gate is the same admission controller with a policy that branches on image class. That is the key architectural move: reality is absorbed by *widening the policy vocabulary*, never by *turning the gate off* for a namespace or a cluster.

Metrics: proving the chain is closed

You manage what you measure, and a supply-chain program lives or dies on whether leadership can see it working (Book 8, Chapter 8 — Metrics, Audits, and Executive Reporting). The metrics that matter are **coverage** metrics — what fraction of the fleet is actually inside the chain — and **response** metrics — how fast the chain reacts when something is found.

Metric	Definition	Target	Why it matters
Signature coverage	% of running images with a valid signature from the expected build identity	→ 100% internal	The floor of the whole model
Provenance coverage	% of running images with a verified SLSA L3 provenance	→ 100% internal	Proves builds are trustworthy, not just present
SBOM coverage	% of running images with an attested SBOM in the metadata store	→ 100%	Enables fleet-wide CVE queries

Metric	Definition	Target	Why it matters
Verified-admission coverage	% of deploys that passed enforcing admission (vs. audit/exempt)	→ 100%	Measures how much of the fleet the gate actually gates
Golden-base adoption	% of images built FROM a golden signed base	→ 100% internal	Shrinks attack surface; enables rebase
Policy coverage	% of clusters under the central admission policy	100%	No cluster is a soft target
Scan/patch MTTR (rebase)	Median time from CVE disclosure to rebuilt, redeployed image	hours-days	The speed of the response loop
Open exceptions	Count of active exceptions, and count past expiry	trending down; zero expired	Measures the size and hygiene of the exception debt

Coverage and MTTR are complementary. High coverage with slow MTTR means the chain is complete but sluggish — a Log4Shell would be *visible* everywhere and *fixed* nowhere fast. Fast MTTR with low coverage means you can patch quickly but only for the fraction of the fleet you can see. You want both, and you want the trend, not the snapshot: a program is healthy when coverage climbs and MTTR falls quarter over quarter.

The single sharpest metric is not on the table because it is a yes/no property, and it is the test the entire architecture exists to pass:

Can an unsigned, unscanned, unauthorized-source, or tag-mutable image reach production?

In this architecture the answer is **structurally no** — not "we would probably catch it," but "the admission gate on every cluster demands a signature from the expected build identity, over a pinned digest, with provenance and a passing scan, and refuses anything that lacks them." The controls make the bad state *unreachable*, which is a stronger guarantee than making it detectable. If you can build the chain to the

point where you can honestly answer "no" to that question, you have built the thing this book set out to build.

Distributed-systems lens

Step back and look at what you have actually constructed, because it is easy to see it as "security tooling" and miss what it is. This reference architecture **is a distributed system** — one you build, run, and operate, with all the properties Book 6 (the Distributed Systems volume) would recognize. Its components are a build platform, an internal registry, a metadata and attestation store, a transparency log, GitOps controllers, and admission engines, deployed across many clusters, many teams, and many regions. And every one of those components is **tier-0 or tier-1**: if the registry is down, no service can deploy; if admission is down or fails open, the gate is gone; if Fulcio, Rekor, or the policy distribution path is unavailable, signing or verification stalls. This architecture therefore inherits every hard problem of distributed systems, and you must design for them explicitly.

- **Availability and blast radius.** The gate sits in the pod-creation critical path on every cluster. Its failure mode is a first-class design decision: fail-closed protects integrity but can wedge the fleet during a control-plane incident; fail-open preserves availability but opens the exact hole the gate exists to close. The right answer is usually fail-closed with aggressive HA — the verifier replicated, its policy and trust roots cached locally on each cluster so an admission decision does not depend on a synchronous call to a remote service — plus a break-glass procedure that is itself audited (Book 8). The registry and metadata store need HA and DR to match their tier-0 status.
- **Consistency of policy and trust roots.** Policy and trust roots (Fulcio's root, Rekor's public key, the set of expected build identities) are replicated state distributed to every cluster. That is a consistency problem. A key rotation or a policy tightening that reaches some clusters before others produces exactly the split-brain you would expect: the same image admitted here and denied there. Policy distribution must be versioned, monotonic, and observable, and rollouts must be staged the same warn-then-enforce way as everything else.

- **Evidence flows artifact-to-gate, asynchronously.** The metadata that a gate consumes is produced far upstream and far earlier, and it must be *present and consistent* when the gate reads it. If an attestation has not yet propagated to the registry a cluster is pulling from, verification fails on a legitimate image — a distributed-systems race, not a security event, that you must design around with read-after-write guarantees on the registry and referrers path.

The reason to endure all of this is the payoff, and the payoff is also a distributed-systems property: **security is inherited by every service through the platform, and enforced uniformly at admission with no bypass.** In a fleet of thousands of services owned by hundreds of teams, you cannot secure the supply chain service by service — the coverage will always have holes where the attacker walks in. You secure it by making the platform produce verifiable evidence at every step and by making one un-bypassable boundary verify that evidence for every workload. That is the concrete, cloud-native realization of the thesis that runs through this entire suite: **produce verifiable evidence at every step; verify at every boundary; centralize the machinery into a platform; enforce it fleet-wide.** Books 1 through 8 built the pieces. This chapter wired them into a system. What you do next is build it.

Key takeaways

- **The chain of custody is the architecture.** No single control secures a cloud-native supply chain; security is the integration of source integrity, build provenance, signing, SBOMs, image hygiene, and verified admission into an unbroken chain where every artifact carries verifiable, digest-bound metadata and every boundary verifies it.
- **Walk the stages, know each edge.** Source (signed commits) → build (SLSA L3 provenance + SBOM + signature, all platform-generated) → golden signed bases → registry (chokepoint: attestations by OCI referrers, scan-on-push) → metadata store (GUAC/Dependency-Track + Rekor) → GitOps delivery (signed config, digest-pinned) → admission (the un-bypassable backstop) → runtime (re-scan, drift).
- **The paved road makes it adoptable.** The platform team owns the whole secure chain; application teams inherit it by pushing reviewed,

signed code. Security is inherited, not implemented per team, and enforced centrally and uniformly at admission.

- **Admission is the floor, everything else is depth.** Even with the pipeline, GitOps repo, and registry fully compromised, an image without a valid signature from the expected build identity — over a pinned digest, with matching provenance — cannot run. The gate trusts the evidence, not the pipeline.
- **Sequence it: visibility → integrity → verification → enforcement,** always warn-then-enforce. Tiered policy and expiring exceptions absorb third-party images and legacy reality by widening the policy, never by turning the gate off.
- **Measure coverage and MTTR,** and hold yourself to the end-state test: an unsigned, unscanned, unauthorized-source, or tag-mutable image must be *structurally* unable to reach production.
- **It is a distributed system.** Every component is tier-0/1 with HA/DR and consistency demands; design the gate's failure mode, the propagation of policy and trust roots, and the artifact-to-gate evidence flow as deliberately as you would design any critical-path service.

Further reading

- **SLSA v1.0 specification** — Build levels, provenance, and the threats each level addresses (slsa.dev/spec/v1.0).
- **Sigstore documentation** — Cosign, Fulcio, and Rekor: keyless signing, certificate transparency for code, and the transparency log (docs.sigstore.dev).
- **in-toto Attestation Framework specification** — predicate/statement structure and the SLSA provenance and SBOM predicate types (github.com/in-toto/attestation).
- **OpenVEX specification and the CISA "Minimum Requirements for Vulnerability Exploitability eXchange (VEX)"** — expressing not_affected/affected status against SBOM components.
- **SPDX 2.3 / 3.0 and CycloneDX 1.6 specifications** — SBOM formats used by syft and the attestation predicates.
- **OCI Distribution and Image specifications, including the Referrers API** — how signatures and attestations attach to an image by digest (github.com/opencontainers).

- GUAC documentation (guac.sh) and OWASP Dependency-Track documentation — ingesting and querying SBOMs, provenance, and VEX at scale.
- Kyverno documentation, "Verify Images," and Sigstore policy-controller documentation — admission-time signature, identity, and attestation verification in Kubernetes.
- Argo CD and Flux documentation on GitOps, and the CNCF "Software Supply Chain Best Practices" white paper (github.com/cncf/tag-security).
- NIST SP 800-218 (Secure Software Development Framework, SSDF) and NIST SP 800-204D (supply-chain security for cloud-native applications) — control mappings behind the framework table.
- Cross-references in this suite: Book 1, Chapter 7 — Risk Frameworks and Maturity Models; Book 1, Chapter 10 — Building a Supply Chain Security Program; Book 3 — SBOMs (esp. Chapters 5-6); Book 4, Chapters 2, 3, 8, 10 — Hermetic Builds, SLSA Provenance, Isolated Build Environments, Secure Build Platform; Book 5, Chapters 4-6, 10 — Keyless Signing, Transparency Logs, in-toto, Attestation Gates; Book 6, Chapters 1-8 — this book; Book 7 — Source, Code, and Insider Threat Security; Book 8, Chapters 1, 2, 4, 8 — Regulatory Landscape, Adopting SLSA/S2C2F, Policy as Code, Metrics and Reporting.